MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING

MÓDULO 2. ENTORNOS DE ANÁLISIS DE MALWARE

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM Universidad Católica de Murcia

TABLA DE CONTENIDO

MÓDULO 2. ENTORNOS DE ANÁLISIS DE MALWARE 1

1. Introducción 7
1.1. ¿De qué trata la asignatura? 7
1.2. ¿Cuál es el enfoque de la asignatura? 7
1.3. ¿Por qué análisis automatizado? 8
1.4. ¿Sustituye el análisis automátizado al manual? 9
1.5. ¿Cuál es el flujo de trabajo de un sistema de análisis automátizado? 9
1.6. Sobre las prácticas voluntarias y el entorno para las evaluaciones 10

2. El entorno de análisis de malware 12
2.1. Ingesta y origen de las muestras 12
2.2. Descarte temprano y preclasificación 13
2.3. Almacenaje de las muestras y su razón 14
2.4. ¿De dónde sale el malware? 14
2.5. Hacia donde va el malware 15

3. Identificación a través de funciones hash 16
3.1. Funciones hash 16
3.2. Funciones hash difusas 17
3.2.1. Algunas funciones hash difusas 18
3.2.2. Otras funciones hash útiles 19
3.3. Conclusiones respecto a las funciones hash 20

4. Reglas YARA 21
4.1. Introducción 21
4.2. Estructura de una regla YARA 22
4.3. Un simple ejemplo 22
4.4. Cadenas 24
4.4.1. Cadenas (o secuencias) hexadecimales 24
4.4.2. Cadenas de texto 25
4.4.3. Expresiones regulares 28
4.5. Condiciones 28
4.5.1. La condición simple 29
4.5.2. Operadores booleanos 29
4.5.3. Repetición de cadenas 30

4.5.4. Operadores de posición relativa de la cadena 30
4.5.5. Discriminar por tamaño de archivo 32
4.5.6. Conjuntos 34
4.6. Módulos 35
4.7. Uso de YARA 36
4.7.1. Invocación básica 36
4.7.2. Algunas opciones interesantes 37
4.7.3. Reglas compiladas 37
4.8. Conclusión 38
4.9. Otras herramientas de detección estáticas 38

5. Sandboxes 40
5.1. ¿Qué es una sandbox? 40
5.2. Características deseables en una sandbox 41
5.3. Aislamiento de red 42
5.3.1. Emulación de infraestructura de red 43
5.3.2. Interposición de proxies, VPN, Tor 44
5.4. Sandboxes comerciales 44
5.4.1. Principales marcas 45
5.4.2. ¿Qué parámetros debemos tener en cuenta en este tipo de soluciones? 45
5.4.3. Opciones y prestaciones de las sandboxes comerciales 46
5.5. Sandboxes Open Source 50
5.5.1. Cuckoo y CAPE2 50
5.5.2. Manalyzer 51
5.6. Handmade Sandbox 52
5.7. Contramedidas frente a la detección de la virtualización 53

6. Reglas Sigma 55
6.1. Introducción 55
6.2. Funcionamiento de SIGMA 55
6.3. Conversión de regla SIGMA a regla específica 59
6.3.1. Preparación del entorno 59
6.3.2. Obtención de las reglas SIGMA 59
6.3.3. Ejemplo de conversión 60
6.4. Creación de una regla SIGMA 62
6.4.1. Metadatos 63

6.4.2. Campos de detección y condiciones 63
6.4.3. Chequear una regla 65
6.5. Etiquetas de las reglas SIGMA 65

7. Reglas Snort 68
7.1. Introducción 68
7.2. El uso de Snort en un entorno de análisis de malware 69
7.2.1. Uso assíncrono 71
7.3. Cabecera de las reglas Snort 72
7.3.1. Acción 72
7.3.2. Protocolo 73
7.3.3. Direcciones IP 73
7.3.4. Puertos 73
7.3.5. Operador de dirección 74
7.3.6. Resumen 74
7.4. Cuerpo de las reglas Snort 74
7.4.1. El mensaje o atributo "msg" 75
7.4.2. "flow" o dirección y estado del tráfico entre cliente-servidor 76
7.4.3. Selección de servicios (o protocolos de aplicación) 76
7.4.4. Contenido y detección 76
7.4.5. Algunos metadatos imprescindibles 77
7.5. Algunos aspectos a tener en cuenta en el uso de Snort 78
7.5.1. Descarte de paquetes por checksum 78
7.5.2. Silenciar mensajes de información 78
7.5.3. Impresión de alertas por pantalla 78
7.6. Otros sistemas de detección de tráfico basados en reglas 79
7.6.1. Suricata 79
7.6.2. Zeek 80

8. El marco de trabajo ATT&CK 81
8.1. Introducción 81
8.2. Tácticas y técnicas 82
8.3. Ejemplos, mitigaciones y detección 84
8.3.1. Ejemplos 84
8.3.2. Mitigaciones 87
8.3.3. Detección 88

8.4. ¿Para qué y cómo se usa ATT&CK? 89

9. Indicadores de compromiso (IOC) 91
9.1. ¿Qué son? 91
9.2. Tipos principales 91
9.3. La Pirámide del Dolor 92
9.4. ¿Para qué se usan? 93
9.5. ¿Cómo extraemos los IoC? 93
9.6. ¿Qué significa "defang"? 94

10. El estándar STIX y TAXII 95
10.1. Introducción 95
10.2. Un lenguaje para modelar amenazas 95
10.2.1. Una advertencia respecto al formato y versiones de STIX 96
10.3. ¿Qué aspecto tiene un paquete STIX? 96
10.4. Componiendo un paquete STIX básico 98
10.4.1. Añadiendo un objeto 100
10.4.2. Estableciendo una relación 101
10.4.3. Creación y uso del bundle STIX 102
10.4.4. Acerca de los UUID 104
10.5. Uso avanzado de STIX 105
10.5.1. SDO (STIX Domain Objects) 105
10.5.2. SRO (STIX Relationship Object) 107
10.5.3. SCO (STIX Cyber-observable Object) 109
10.6. Caso práctico con STIX 109
10.6.1. Descubrimos que el malware es del tipo ransomware y emplea técnicas rookit 112
10.6.2. Modelando el archivo descargado 113
10.6.3. Añadiendo referencias 115
10.6.4. Añadiendo el grupo APT 118
10.6.5. Añadiendo el informe y su URL de consulta 120
10.6.6. Añadiendo un indicador, el hash visto 122
10.6.7. El cuadro final 123
10.7. TAXII 127

11. Cyber kill chain 129
11.1. ¿Para qué sirve en ciberseguridad? 129

11.2. Fases de las Cyber-Kill-Chain.....129
11.3. ¿Cómo usamos una Kill-Chain?.....131

1. INTRODUCCIÓN

1.1. ¿De qué trata la asignatura?

Entornos de análisis de malware es una asignatura dedicada a proporcionarte los conocimientos para poder entender y trabajar con lo que denominamos un **sistema de análisis automatizado de malware**.

En contraposición, estaría el análisis manual de un malware, el cual precisa de la atención y dedicación plena de un especialista y que verás en una asignatura posterior dentro del plan de estudios. Así, la asignatura de **Entornos de Análisis de Malware** posee un propósito múltiple:

- Servir de introducción al malware en general
- Conocer qué hace y cómo opera un sistema de análisis automatizado.
- Conocer que tipos de herramientas tenemos disponibles para insertarlas en nuestro flujo de trabajo.
- Conocer y practicar con los distintos estándar de facto, usados por la industria y laboratorios de soluciones antimalware.

Es decir, en primer lugar, fijaremos en la foto todo lo que debemos saber acerca del malware y posteriormente nos introduciremos en el análisis automatizado para obtener un diagnóstico rápido que nos permita orientarnos acerca de la gran pregunta que toda organización o usuario se hace cuando está a punto de hacer clic ante un binario descargado de Internet o adjunto al correo: *¿Es malware o no?*

Además de ver "la cadena de montaje" del análisis de malware, haremos paradas por todos los componentes de esa cadena haciendo hincapié en el aspecto técnico y, por supuesto, práctico.

1.2. ¿Cuál es el enfoque de la asignatura?

Práctico: terminarás haciendo reglas YARA, SIGMA, SNORT y creando paquetes STIX, detonando binarios en sandboxes y haciendo ejercicios para obtener ciertos datos que, aunque automatizables, debes saber cómo operar manualmente para conocer el sentido que poseen dentro del conjunto de la asignatura.

Orientado al mundo profesional: No vas a tener que montar una solución sandbox. Obtendrás un conocimiento de todo lo que rodea a un **sistema de análisis automatizado** sin perderte en profundidad, pero sin perder de vista los elementos técnicos que la componen, para que sepas evaluar la oferta que ofrece el mercado.

Los conocimientos expuestos en la asignatura te permiten sentarte y ser productivo en entornos profesionales de SOCs, equipos DFIR, inteligencia de amenazas o análisis de

malware desde el primer día.

De forma resumida, una **sandbox** es un sistema operativo (normalmente, **virtualizado**) donde el malware va a ser ejecutado (detonado). Mediante la observación de los cambios que el malware induce en su ejecución a dicho sistema (en caso de existir), se obtiene información para intentar discernir su capacidad maliciosa, infraestructura y funcionamiento. Dado que estos cambios poseen un impacto limitado y no afectan a un **sistema operativo real** (¡o no deben!) por ello de les denomina de esa forma.

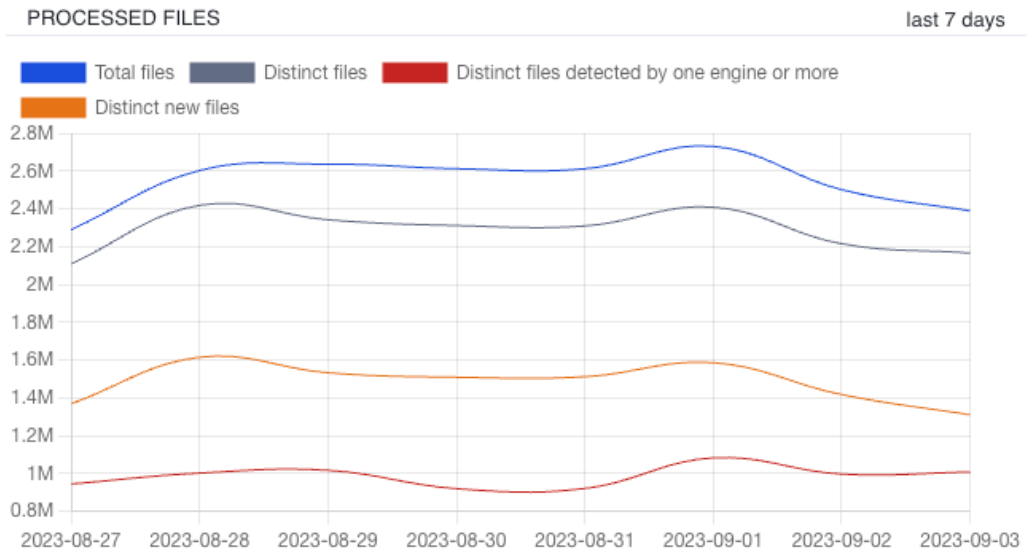
1.3. ¿Por qué análisis automatizado?

Es sencillo de responder. Una organización cualquiera (empresa, oenegé, administración, etc.) recibe una gran cantidad de binarios o adjuntos (documentos) al correo de sus empleados. Si la empresa es grande puede irse a los cientos o incluso miles de archivos.

Por supuesto, necesita saber si son maliciosos o no. ¿Contrata un equipo de analistas de malware expertos en hacer reversing para decidir qué hacer?

La respuesta fácil es: instalan una solución antimalware (más bien, un *Endpoint Detection and Response* o EDR) y ceden el testigo al laboratorio que les provee de dicha solución.

¿Pero qué hace ese laboratorio cuando recibe los cientos o miles de muestras de un cliente que su solución no posee en la base de datos de firmas o cuyo motor de detección es incapaz de inferir una respuesta? ¿Y qué hará cuando tenga miles de clientes?



En un solo día, el servicio Virustotal puede recibir casi 2.8 millones de muestras

Las cuentas salen de forma muy fácil: **no hay analistas suficientes para la ingente**

cantidad de archivos sobre los que se ha de tomar una decisión: Es o no es malware.

Por lo tanto, disponer de un entorno de análisis automatizado es la única herramienta que puede hacer frente al interminable flujo de archivos que han de ser sometidos a un veredicto sobre el que el error puede costar caro.

1.4. ¿Sustituye el análisis automátizado al manual?

Rotundamente, no. Lo complementa. El análisis automatizado no solo es necesario sino fundamental para detectar y clasificar el 99.99% (y quizás bastantes más '9' a la derecha) de muestras que se recogen a diario (recordemos, posiblemente millones).

No obstante, estos sistemas detectan aquello sobre lo que ya se ha escrito o, mejor dicho, descrito mediante reglas fijas, heurísticas o combinaciones de ambos que llevan asociadas señales que el sistema interpreta como malignidad en el comportamiento de una muestra.

Cabe entonces preguntarse ¿Qué ocurre con lo que no detecta o no lo tiene claro? En ese caso, donde no hay carta de navegación sobre la que apoyarse, es necesario el análisis manual y experto por parte de personal instruido para ello.

Hay que añadir también que, en casos muy concretos y precisos de infecciones dirigidas, el malware suele ser único y creado "a medida", por lo que el caso se trata de forma especial y conducido por un equipo multidisciplinar: analista forense, analista de malware, etc. No cabe en casos que requieren delicadeza, la aplicación de entorno automatizados.

1.5. ¿Cuál es el flujo de trabajo de un sistema de análisis automátizado?

El proceso se explicará con detalle en el capítulo correspondiente. No obstante, se darán unas pinceladas para que puedas obtener una visión general de este tipo de sistemas.

El flujo de trabajo comienza por la recepción de muestras. En principio, no se sabe si son maliciosas o no, aunque el origen de estas ya suele ser indicativo de que un usuario o sistema (EDR) sospecha de que algo puede no ir bien. Los orígenes pueden ser variados, ya lo veremos más adelante.

Una vez se reciben las muestras, lo habitual es que el sistema proceda a identificar las muestras mediante un conjunto de funciones hash. Esto es importante por una razón sencilla: no repetir un análisis. Sin embargo, es posible que sí que se quiera repetir un

análisis a posteriori si el anterior no ha sido conclusivo o ha pasado ya un tiempo razonable como para considerarlo obsoleto.

Una vez que la muestra sea "fresca" (es decir: no ha sido vista anteriormente), se procede a realizar un análisis estático con un conjunto de herramientas que identificarán o procesarán la muestra para extraer información útil: fecha de compilación, tamaño, entropía de las secciones de un ejecutable, recursos, metadatos, funciones y librerías importadas o exportadas, etc.

Junto con esta fase (algunas veces antes, conjuntamente o después) se realiza una pasada por las reglas de identificación que posee el laboratorio. En concreto nos referimos a la más popular de todas: YARA. Habitualmente, los laboratorios poseen un conjunto de estas reglas (siempre en crecimiento) que los analistas proporcionan para identificar binarios y familias enteras de estos por ciertas características que los identifican unívocamente (o al menos esa es la intención).

La siguiente fase es la denominada **detonación** de la muestra que no es otra cosa que la ejecución de esta en un entorno de análisis preparado para la toma de datos (telemetría). Lo habitual es realizarlo en sistemas operativos virtualizados, pero nada impide su detonación en sistemas reales; aunque el proceso de restauración es más laborioso y costoso.

Finalmente, se compone (también de forma automática) un informe para su distribución y consulta y, en el caso adecuado, el disparo de una alerta si se trata de un ejemplar malicioso, pero no identificado (por ejemplo, una nueva familia de malware) o visto anteriormente y que haya dado positivo en ciertas reglas.

Como vemos, es un proceso industrializado. Aunque con ciertas variaciones y muy pendientes también de los avances en aprendizaje computacional (machine learning) y otros campos prometedores de la inteligencia artificial, todos poseen en común numerosos elementos aquí descritos.

Como se ha mencionado, en el capítulo correspondiente veremos con detalles cada una de estas fases.

1.6. Sobre las prácticas voluntarias y el entorno para las evaluaciones

Lee el apéndice dedicado a montar el entorno de pruebas. La idea es que uses una máquina virtual con cualquier distribución Linux para no contaminar tu sistema personal. Así, todo queda recluido en un sistema virtualizado.

Las prácticas voluntarias **no se corrigen y no influyen absolutamente nada en tu nota**. Evidentemente, sí **se pueden preguntar dudas acerca de éstas durante el periodo de la asignatura**.

Las prácticas voluntarias sirven para plasmar lo presentado aquí, en el temario, y para que aprendas "tocando". Son básicas y te permiten pasar un primer escalón antes de enfrentarte a las evaluaciones. Más rigurosas y complejas.

Las encontrarás en el apéndice dedicado, en el orden de aparición de su respectivo capítulo en el temario.

Finalmente. Recuerda que las prácticas son voluntarias pero **totalmente recomendables**.

2. EL ENTORNO DE ANÁLISIS DE MALWARE

Tal y como comentamos en el primer capítulo, el flujo de trabajo de un entorno automatizado de análisis de malware es aproximadamente algo similar al esquema siguiente:

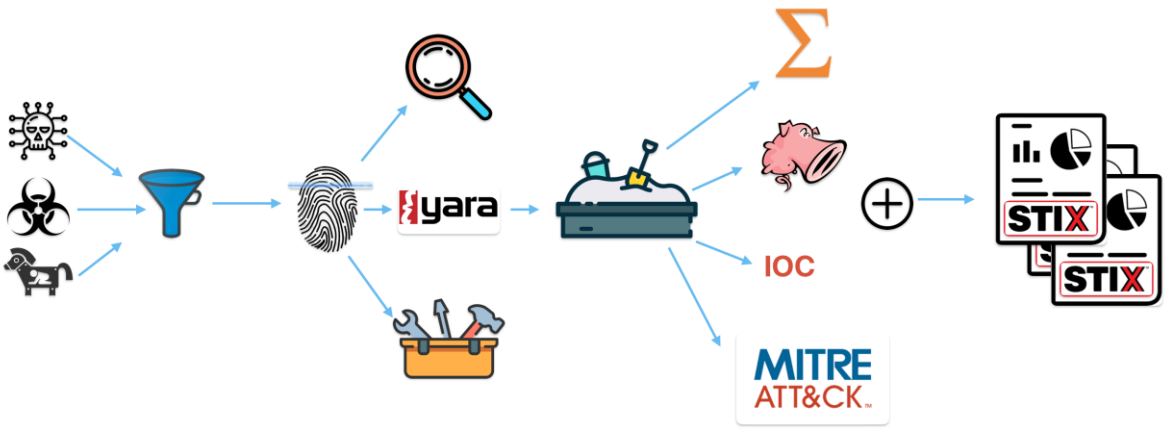


IMAGEN 1 FLUJOGRAMA DE UN ENTORNO DE ANÁLISIS AUTOMATIZADO DE MALWARE

En los siguientes capítulos iremos desgranando todas las fases que componen el entorno de análisis, desde la recolección o ingesta de las muestras, el primer filtrado en base a la identificación a través de varias funciones hash, la aplicación de herramientas sobre las que obtengamos los primeros datos que porta el objeto de estudio, su detonación en la sandbox, la aplicación de nuevas reglas que ayuden a contrastar que se ha obtenido de su ejecución sobre los diferentes objetos que la muestra ha ido proporcionando. Finalmente, la elaboración del informe que, invariablemente, también es automatizado.

2.1. Ingesta y origen de las muestras

Precisamente, estos sistemas nacen de la necesidad de ingestar los cientos de miles e incluso millones de muestras que pueden caer en el paraguas de los laboratorios de las compañías de ciberseguridad.

A esa capacidad para recolectar y procesar esa cantidad de muestras lo denominamos **ingesta**.



La ingesta tiene dos fases o componentes principales: un sistema de recolección o colector y un gestor de colas.

El **sistema de recolección** es un simple programa o script que va visitando recursos en el que hay malware y los deja en un repositorio o almacén hasta que el **gestor de colas** aparece para recogerlos y llevarlos al procesamiento. No hay un estándar de facto ni es una tecnología que cumpla con unos cánones.

2.2. Descarte temprano y preclasificación

Debido a que las muestras nos pueden llegar de diversas fuentes, puede ocurrir que vengan duplicadas. Es decir, que dos o más fuentes nos entreguen la misma muestra.

En ese caso, no nos va a interesar esa muestra así que la idea es tener un sistema que la descarte de forma temprana, antes de que pueda verla algún otro sub-sistema. Eso nos va a ahorrar recursos para no perder el tiempo.

Lo ideal es que se trate de un sistema que calcule el hash (a ser posible que no posea colisiones prácticas) de la muestra al llegar y tenga en caché un listado de los hashes ya analizados previamente o al menos de un tiempo atrás (semanas, meses...). En caso de hallar el hash en memoria la muestra se descarta completamente y tan solo se deja (opcionalmente) un evento registrando el descarte (hora, procedencia, hash, motivo).

El siguiente sub-sistema sería de preclasificación. Simplemente se trata de un componente que detecte el tipo de muestra y la encole en el lugar adecuado. Es fácil de entender ¿Qué hace una muestra de malware para Android esperando ser detonada en una sandbox para Windows?

La preclasificación hace eso, detecta el tipo mediante una herramienta, api o librería (libmagic). Con esa detección el sistema ya sabe dónde debe depositar la muestra.

2.3. Almacenaje de las muestras y su razón

Entre la recolección y de donde toma las muestras el gestor de colas para procesarlas se usará un componente de almacenaje de muestras. Habitualmente, se tratará de servidores on-premise o en la nube (Amazon S3 y servicios similares).

Almacenar muestras posee una problemática: el espacio.

A medida que el sistema va procesando muestras, cabría preguntarse si aportan valor al laboratorio una vez analizas. Y la respuesta es curiosa:

¿Aportan valor ahora o lo aportarán luego?

Extraña pregunta ¿verdad? El problema está en que en el momento del análisis una muestra puede dar un falso negativo (es malware, pero no es detectado como tal) y posteriormente dar positivo; ya sea porque un analista se ha interesado por ella o porque el sistema ha actualizado las reglas de detección u otros componentes y ahora si lo detecta correctamente.

Finalmente, en el mundillo del malware y sus cazadores está el concepto de **retrohunting**. Se trata simplemente de aplicar nuevas reglas a muestras antiguas para poder comparar los resultados.

Imagina que sale una nueva muestra en 2023. Se estudia, se extraen unas reglas YARA y luego se quiere aplicar esas reglas sobre un corpus de muestras de años atrás para ver si el grupo cibercriminal ya estaba en activo antes de dicho año. Evidentemente, si no tienes muestras guardadas ahí se para tu investigación.

2.4. ¿De dónde sale el malware?

El origen es variado. Empezando porque el malware sale del sistema en el que ha sido creado cabría preguntarse a donde llega. Correos con phishing, infecciones procedentes de análisis forenses, etc. Recordemos las cifras que dábamos al principio: millones al día.

Habitualmente, un laboratorio de malware se nutre de dos factores principales: su propia cosecha y acuerdos de intercambio de muestras con el resto de los laboratorios.

La “cosecha propia” es la diseminación de **honeypots** (sistema que aparenta ser real, para que sea atacado y luego estudiar los cambios), cuentas trampa de correo electrónico, arañas (programa para navegar de forma autónoma) que van visitando la web, etc.

Si el laboratorio posee una solución de detección es probable, si la licencia lo permite o el usuario opta a subir la muestra, que también obtenga nuevas muestras gracias a esa funcionalidad.

Sin embargo, lo habitual es cerrar acuerdos entre compañías y laboratorios (que pueden ser comerciales, universidades o administraciones, etc.) e intercambiarse muestras a través de una plataforma en la que todos están de algún modo obligados a ingresar unas cuotas mínimas de muestras que serán diseminadas a los partícipes.

2.5. Hacia donde va el malware

Una vez le llega su turno, el gestor de colas toma la muestra y la deposita en la siguiente fase de la cadena: su identificación y análisis estático preliminar. Lo habitual es que traslade una copia y deje el original con su ficha de identificación (procedencia, peso, hora de llegada, estado del análisis, fecha del análisis...) pendiente de que se termine el informe.

3. IDENTIFICACIÓN A TRAVÉS DE FUNCIONES HASH

En esta fase, aun no se ha detonado la muestra. Primero, antes de gastar recursos computacionales se verá si merece la pena detonarla o se trata de un ejemplar ya estudiado.



Hay que comentar que algunos sistemas tienen esta detección de duplicidad en el recolector, el cual una vez recibe la muestra ya se pasa por ciertas funciones hash y el propio recolector las descarta si ya están incorporadas en el sistema; una especie de identificación temprana. Todo dependerá de cómo tengamos orientada nuestra arquitectura y de cómo queramos balancear el gasto de nuestros recursos.

Algunas de las técnicas y herramientas comentadas en esta sección también pertenecen al ámbito de la asignatura: "Técnicas de análisis de malware", que se dará más adelante. No obstante, el enfoque que se le da aquí será siempre el orientado a la automatización y complementario a aquella asignatura.

3.1. Funciones hash

Como ya las habrás visto en la asignatura de criptografía, no vamos a repetir aquí la teoría detrás de las funciones hash. Simplemente diremos que se requieren dos cualidades cuando se usan en estos sistemas y que coinciden con lo deseable en las propiedades de las funciones hash: que la clave se calcule rápido y que no haya colisiones.

Las propiedades comentadas se traducen fácilmente a los intereses que buscamos en un sistema que debe procesar millones de muestras: rapidez de cálculo -> coste de servidores, colisiones -> que dos archivos con el mismo hash en realidad sean distintos.

¿Qué función hash utilizamos?

Como sabemos, tenemos un gran abanico de funciones disponibles y precisamente por ello, no solo emplearemos una sino un grupo de estas para una misma muestra. El porqué es sencillo. Nosotros almacenamos información de la muestra en el sistema, pero a la vez, quienes se alimentan de dicha información (El SOC, analistas de inteligencia de amenazas, equipos DFIR, etc.) necesitan cruzarla. Si solo utilizáramos una función hash (por ejemplo, SHA2) y el equipo está estudiando información externa de muestras que están en otra función hash (pongamos la separadísima MD5), nunca sabrían que en realidad esas muestras ya se poseen y han sido estudiadas.

Por lo tanto, no hablamos de una función hash sino de un grupo de estas.

Funciones hash criptográficas

No vamos a extendernos aquí, se usan las habituales, desde funciones hash que han dejado de ser seguras (se usan por costumbre) para entornos donde se necesita seguridad: MD5, SHA1…hasta las que aun si lo son: SHA2, etc.

Que una función hash criptográfica deje de ser segura no significa que no pueda usarse para obtener el hash de un conjunto de datos (un archivo, una captura de red, etc.) para compararlo con otro. Simplemente, la función ha dejado de ser segura y no se recomienda donde la seguridad sea un requisito.

Es decir, si nos basáramos solo en funciones hash inseguras los creadores de malware explotarían este hecho intentando encontrar colisiones con archivos no maliciosos para no ser detectados.

3.2. Funciones hash difusas

Las funciones hash que hemos visto toman un conjunto de entrada (un archivo, por ejemplo) y su resultado en un resumen (*digest*) que es lo que acostumbramos a ver (una cadena de caracteres hexadecimales).

Eso puede no ser suficiente, debido entre otras cosas a las colisiones de algunas funciones hash, pero mejor aún, ¿podríamos detectar que una muestra es una variación con ligeras modificaciones? Es lo que solucionan las funciones hash difusas.

Este tipo de funciones hash están pensadas para detectar variaciones o alteraciones en un archivo. Dado que estas variaciones podrían causar que el hash cambiase, una pequeña variación en una muestra ya identificada podría generar otro hash y pasar desapercibida.

¿Cómo funcionan?

Las funciones hash habituales que solemos ver con frecuencia (md5, sha1, ...) toman el conjunto de datos de entrada como un todo. Las funciones hash difusas, de forma genérica, "trocean" por bloques la entrada y van creando partes del hash final. Por ello, si las variaciones son solo en una parte concreta veremos cómo solo cambia una parte del hash final.

Esto es extremadamente útil en el caso de identificación de muestras. Imagina que los autores del malware cambian tan solo la configuración que porta el malware y lo vuelven a liberar. Los hashes habituales cambiarían completamente, con lo que creeríamos que la muestra es completamente nueva. Sin embargo, esto no pasa con los hashes difusos, cambiarían una parte, pero no el todo. Por ello, sabríamos que se trata de malware de la misma familia o con una mínima variación.

3.2.1. Algunas funciones hash difusas

Existe un buen arsenal de funciones de este tipo. Vamos a enumerar las más usadas.

Ssdeep

Es la que hemos usado en el ejemplo anterior. Es una de las primeras funciones de esta clase, pionera en su género cuando hablamos de implementaciones prácticas. Tal y como reza en su propia documentación, no es la mejor función, pero si una de las más rápidas, algo que casa con los intereses que tenemos en nuestro sistema de análisis. Fiabilidad sin sacrificar rendimiento.

Sdhash

Basada en ssdeep, añade un repertorio de técnicas y algoritmos para tener en cuenta no solo los fragmentos o secuencia de bytes sino la procedencia de estos dentro del archivo. Es decir, que tiene en cuenta como se estructura el archivo o secuencia de datos (bytes).

Esto último no es una diferencia pequeña, se trata de una técnica poderosa que nos va a tener en cuenta que, si una secuencia idéntica se ha obtenido de una parte en un archivo A, y en otra en un archivo B, veremos cómo los hashes finales muestran esa coincidencia.

TLSH

Procedente del personal del laboratorio de Trend Micro, este hash es igualmente muy usado por la industria antimalware desde su presentación. Es quizás no tan conocido como ssdeep, pero al ser una solución muy orientada al malware (cambios pequeños en dos archivos idénticos) está siendo bien acogida y popular.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

3.2.2. Otras funciones hash útiles

Imphash

En el mismo espíritu de TLSH, no podemos dejar de pasar la oportunidad para comentar la función hash **imphash**.

Como todos sabemos, los ejecutables importan librerías y funciones externas para poder funcionar. La idea es calcular un hash de las importaciones que realizan y compararlo con otros. De esta forma, se puede obtener una familia de archivos ejecutables que poseen un grupo de importación de funciones externas idéntico.

Eso sí, si varían en una sola función o el orden de importación los hashes serán distintos (no es una función hash difusa).

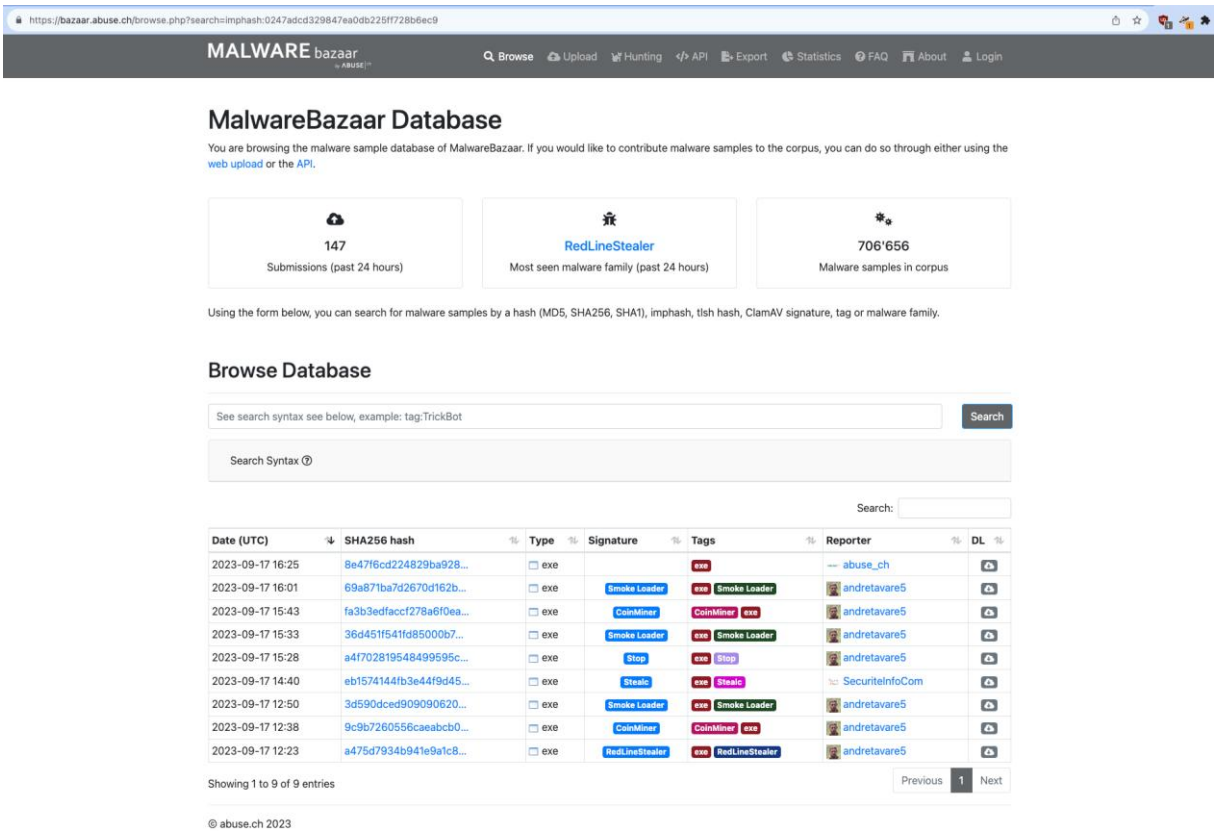


IMAGEN 2 GRUPO DE MALWARE CON DISTINTOS HASHES, PERO IDÉNTICO IMPHASH

En la imagen de arriba podemos ver ilustrado el concepto que presenta imphash. Esos ejecutables con diferentes (observa el hash SHA256) sin embargo, su imphash es idéntico, lo que significa que están relacionados.

Authentihash

Es el hash criptográfico usado por Microsoft para su tecnología Authenticode. Básicamente, es el hash del ejecutable sin las cabeceras y el certificado que se le añaden una vez es firmado.

3.3. Conclusiones respecto a las funciones hash

Como vemos, existe una gran riqueza de funciones hash. Nos hemos dejado bastantes más, pero prácticamente tomarían una gran parte del temario que es necesaria ocupar con otros conceptos.

Existen funciones hash incluso para los recursos que porta un ejecutable consigo mismo (icono de la aplicación, imágenes, etc.) y no solo identificando unívocamente la imagen sino la similitud con otras. Imagina el caso en el que se troyaniza una aplicación móvil y se cambia el color del icono para intentar llamar la atención.

4. REGLAS YARA

4.1. Introducción

YARA surgió precisamente de la necesidad de poner orden en el caos que era manejar la gran cantidad de muestras que cada vez más llegaban a los servidores de los laboratorios.

Los analistas de malware se daban cuenta que muchas de las muestras que analizaban eran variantes de una misma familia, por lo que cuando abrían el desensamblador la sensación que tenían era de perder el tiempo: incluso los parámetros de configuración del malware estaban en el mismo lugar, con lo que un simple script era capaz de extraerlos sin necesidad de detonarla.

¿Pero cómo identificas un grupo de muestras, qué tienen en común? Para empezar, cadenas de texto o secuencias (patrones) de bytes únicos en esa familia.

A estos patrones, le aplicas condiciones: donde aparecen, tamaño del archivo, punto de entrada del ejecutable, etc. y ya tienes una regla YARA.

Una vez tienes un conjunto de reglas, puedes aplicarlas sobre las muestras que te vayan llegando y con ello, puedes detectar familias enteras de malware que es similar entre ellos.

YARA es un estándar en la industria. No solo debes conocerla, sino que es **IMPRESCINDIBLE** que sepas manejar la herramienta y sobre todo la composición de reglas con soltura. De hecho, en las entrevistas de puestos para ciberseguridad la batería de preguntas sobre YARA es un clásico. Por esto mismo, le vamos a dar importancia a su peso en la asignatura y te animo a profundizar en ello, incluso más allá de lo que se enseñe durante la materia.

Vamos a ver qué son, como se usan y su sintaxis básica.

4.2. Estructura de una regla YARA

Antes de nada, una regla YARA tiene este aspecto:

```
rule silent_banker : banker
{
    meta:
        description = "This is just an example"
        threat_level = 3
        in_the_wild = true

    strings:
        $a = {6A 40 68 00 30 00 00 6A 14 8D 91}
        $b = {8D 4D B0 2B C1 83 C0 27 99 6A 4E 59 F7 F9}
        $c = "UVODFRYSIHLNWPEJXQZAKCBGMT"

    condition:
        $a or $b or $c
}
```

Es, básicamente, un lenguaje de programación declarativo (de hecho, puedes compilar reglas yara).

La primera línea es título de la regla y podemos escoger el que queramos (obviamente, que sea único). En nuestro caso, la regla se llama “silent_banker”. Tras los dos puntos le sigue una serie de tags o etiquetas. Luego veremos para que sirven, pero podemos adelantar que sirven, esencialmente, para tener variantes de una misma regla o grupos de reglas.

La sección de meta (metadata) es para describir la regla. Podemos poner los campos que queramos en el formato que ves: clave, valor. No tiene ninguna utilidad a la hora de detectar o que la regla se active. Piensa que es información para humanos. Eso sí, podemos utilizar grep o alguna herramienta similar para procesar la salida de YARA.

4.3. Un simple ejemplo

Vamos a ver un ejemplo muy sencillo para ilustrar lo comentado:

```
rule simple: bash_scripts
{
  meta:
    author = "David Garcia"
    description = "This simple rule detect a shebang on scripts"

  strings:
    $text_string = "/bin/env sh"

  condition:
    $text_string
}
```

ILUSTRACIÓN 1 REGLA YARA

La regla es sencilla, los metadatos mínimos y solo posee una cadena de texto. La condición es mínima también, simplemente le decimos que si aparece la regla de un positivo.

Al ejecutar la regla, vemos esto:

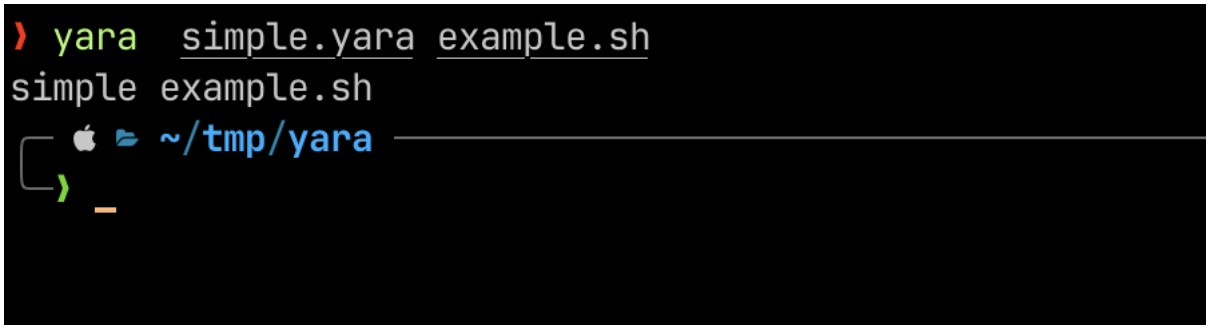


ILUSTRACIÓN 2 EJECUCIÓN DE UNA REGLA YARA

Una vez se ejecuta “yara <reglas> <archivos>”, da positivo y exhibe que regla ha sido la que ha causado la detección.

El archivo ‘example.sh’ posee la siguiente línea:



ILUSTRACIÓN 3 EJECUCIÓN DE UNA REGLA YARA (II)

A continuación, vamos a ver las distintas secciones clave (strings, condition) que debe contener una regla YARA.

4.4. Cadenas

Vamos a la parte importante. **Strings** es la parte declarativa respecto a lo que queremos que se detecte. Ahí va el patrón o patrones que nos interesa para que YARA los detecte.

No nos dejemos llevar por la palabra "string". El contenido a detectar es variado. En primer lugar, hay varios tipos de strings:

4.4.1. Cadenas (o secuencias) hexadecimales

Son las cadenas de bytes en formato hexadecimal. Un ejemplo sencillo: si quisiéramos detectar ejecutables que posean la secuencia \x41\x41\x41\x41 la regla yara sería:

```
rule detect_41 {

strings:
        $hex_string = { 41 41 41 41}

condition:
        $hex_string
}
```

No necesitamos explicitar que son bytes en hexadecimal. YARA es capaz de detectarlo automáticamente al encontrar la secuencia dentro de las llaves "{ }". Es similar a un array en C. Tampoco tenemos que escapar el digito, se da por hecho que es hexadecimal.

La mayor parte de las reglas poseen este tipo de secuencias. Sobre todo, debido a que las cadenas en el malware suelen estar cifradas u ofuscadas.

Existe un potente conjunto de operadores que multiplican la efectividad de las cadenas de bytes:

Comodines:

> Si la cadena incluye un '??' entonces hace de comodín y cualquier valor aplica. Por ejemplo, podemos tener {41 ?? 41 41} y entonces valdría para detectar la secuencia 41 41 41 41 y 41 42 41 41...o 41 A0 41 41. También funciona con un solo interrogante: 41 4? 41 41, lo que daría validez desde 40 a 4F.

Negación:

> Justo lo contrario, el uso del carácter '~' (virgulilla) haría que se negase la expresión. Es como decir: "No no empiecen o terminen por...". Ejemplo: {41 ~4? 41 41} haría que la expresión 41 31 41 41 diera positivo, pero no la 41 41 41 41. Otro ejemplo {41 ~41 41 41} excluiría explícitamente un 41 en dicha posición.

Rangos:

No es exactamente intuitivo, pero es muy potente. Se trata de indicar un rango de valores. La sintaxis es esta [comienzo, final]. Ejemplo: {41 [1-8] 41 41}. ¿Crees que significa que admite valores en ese byte concreto desde el 01 al 08? Pues no, para eso estarían los comodines. En realidad, dado que en ciertos patrones puede haber grupos de bytes que cambien y no nos interesan, podemos indicar a YARA que los ignore. Es decir, las secuencias: 41 00 4A 16 41 41 y 41 23 15 4F 31 32 41 41 darían positivo. Lo importante es que después de 1 a 8 bytes se vuelva a encontrar el 41 41.

Los rangos pueden incluso no tener límites: [4-] no hay límite superior, [-10] no hay límite inferior, [-] sin límites. O fijo: [6] que es exactamente seis bytes.

Alternativas:

O variaciones. Se trata de indicar dos o más grupos de secuencia de bytes. La sintaxis es similar a las expresiones OR: { 41 (41 | 42) 41 41} Se explica sola, o detecta un 41 en ese byte o un 42, ambos darán positivo.

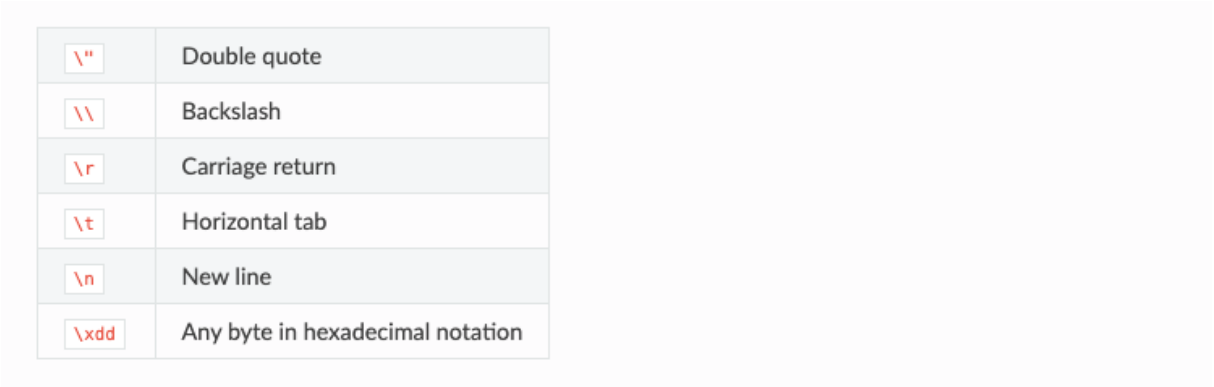
Las alternativas pueden complicarse y combinarse con los operadores anteriores tanto como queramos: {41 (41 4? 43 | ?? ~40 41 | 30 [4-6] 32) 41 41}

4.4.2. Cadenas de texto

Aquí ya sí entramos en las cadenas de texto tal y como las conocemos, caracteres englobados en un par de comillas.

$string_de_texto: “cadena que queramos detectar”

Las cadenas pueden contener secuencias de escape, son similares a las del lenguaje C:



\"	Double quote
\\	Backslash
\r	Carriage return
\t	Horizontal tab
\n	New line
\xdd	Any byte in hexadecimal notation

IMAGEN 3 SECUENCIAS DE ESCAPE EN YARA (WEB DE YARA)

Así:

$string_de_texto: "quiero detectar esta palabra: \"hola\""

Tendría en cuenta como caracteres a detectar las comillas que engloban o cierran la palabra hola.

Por defecto, YARA es case-sensitive, lo que quiere decir que discrimina las mayúsculas de las minúsculas. Algo que podemos solucionar indicando con la palabra clave "nocase" que se detecte tanto mayúsculas como minúsculas:

$string_de_texto: "cadena que queramos detectar" nocase

Detectaría tanto la cadena en sí como "Cadena Que QUERAMOS Detectar".

Otro modificador sería wide, que nos permite detectar la cadena cuando se está usando dos bytes por carácter. Para compatibilizar la detección de cadenas con caracteres habituales y las extendidas con representación de dos bytes podemos añadir ascii.

$string_de_texto: "cadena que queramos detectar" wide ascii

Uno de los cifrados triviales es el uso de XOR. Si recuerdas, la operación XOR es un OR-Exclusivo. Es decir, si tengo los bits 0x010101 y hago XOR con 0x000111 obtengo el siguiente resultado: 0x010010. En una cadena, la operación XOR transforma el byte correspondiente al carácter, lo que resulta en un galimatías para la vista humana. Un ejemplo de pasar una cadena a través de la operación XOR con el byte al azar de 0x4B:

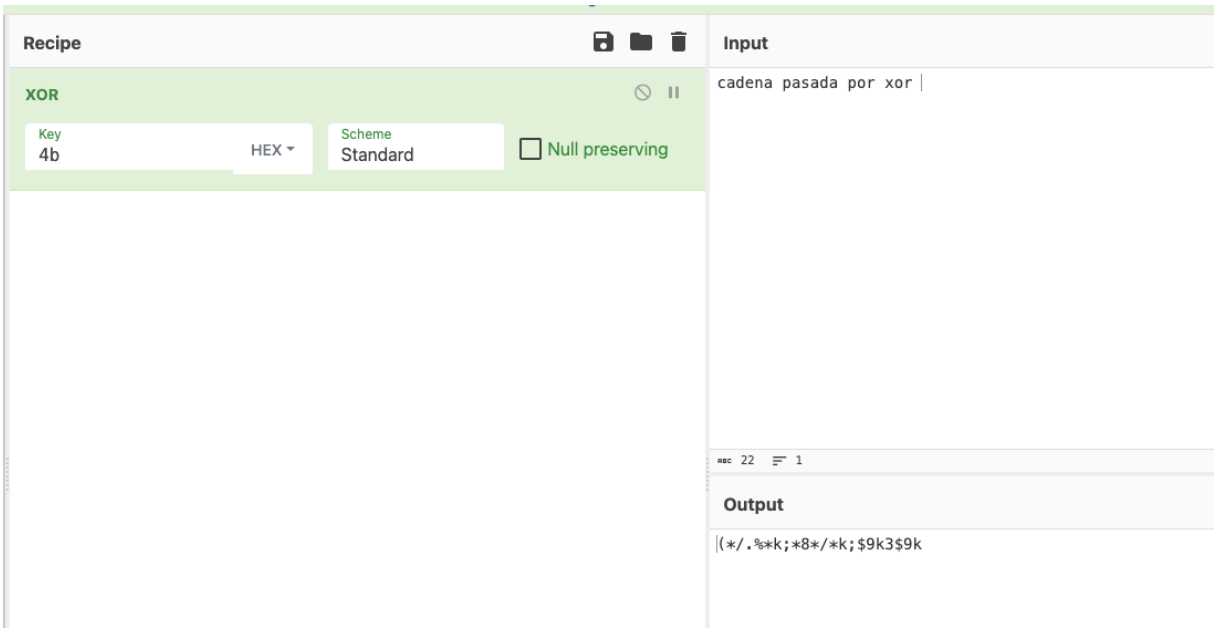


IMAGEN 4 CADENA XOREADA

Como vemos en el output, el resultado es ilegible.

Pues bien, YARA cubre esa posibilidad simplemente indicando xor como operador:

$string_de_texto: “cadena que queramos detectar” xor

Detectaría sin problemas la cadena XOReada con cualquier byte desde 0x00 a 0xFF.

En algunos casos también podemos encontrarnos con cadenas codificadas en base64, algo bastante común. En tal caso, disponemos del operador base64:

$string_de_texto: “cadena que queramos detectar” base64

Detectaría la cadena:

Y2FkZW5hIHF1ZSBxdWVyYW1vcyBkZXRlY3Rhcg==

También disponemos del operador base64wide que emplea la codificación base64, pero teniendo en cuenta lo comentado para el operador wide.

En algunos casos, queremos detectar una palabra exacta, no como subcadena. Por ejemplo, tenemos la cadena: www.dominiointeresante.com. Si queremos detectar la palabra “interesante” esa cadena anterior nos daría un positivo, pero no es lo que andamos buscando.

Si queremos que YARA nos detecta la palabra y solo esa palabra (sin que forme parte de una cadena) entonces usaremos el operador fullword.

$string_de_texto: "interesante" fullword

4.4.3. Expresiones regulares

YARA soporta expresiones regulares, algo que no sorprende ya que se trata de un lenguaje especializado en el dominio de la detección de patrones.

Para usar una expresión regular tan solo tenemos que agruparla entre barras inclinadas. Un ejemplo:

```
$md5_hash = /md5: [0-9a-fA-F]{32}/
```

Esa expresión regular detectaría cadenas del tipo: "md5: 3ae6368cd5dd77fa158c8fc5b20cd908".

Las expresiones regular son en si mismas un lenguaje más e increíblemente potentes. No podemos extendernos aquí en el uso de estas. Cualquier tutorial PCRE (Perl Compatible Regular Expression) lo explicaría mucho mejor.

En Ciberseguridad en particular y en informática en general abunda el uso de **expresiones regulares**. No solo en caso de tratar con malware o inteligencia de amenazas. Por ejemplo, encontrar una cadena determinada en un archivo de millones de líneas de eventos, puede ser complicado a veces si la cadena posee variaciones, etc. En tal caso, las expresiones regulares son imprescindibles. Aprende a usar expresiones regulares. Agradecerás este consejo.

4.5. Condiciones

Una vez hemos definido qué queremos detectar, ahora veremos donde y como con las **condiciones**.

El qué queremos detectar ya lo hemos estado definiendo anteriormente con las variables, esas asignaciones que hemos visto de cadenas, por ejemplo: $cadena1 = {00 01 4? 34 56}

Ahora, en las condiciones, podremos hacer referencia a todas esas cadenas que hemos ido definiendo y las aplicaremos a las reglas lógicas que YARA nos facilita.

En **condition** debes hacer uso de todas las cadenas que pusiste en **strings**. Si YARA encuentra una variable que no se usa en condition dará error (unreferenced string).

4.5.1. La condición simple

Que aparezca la cadena, sin más.

Observemos la siguiente regla:

```
rule simple_1 {
        strings:
                $cadena = “cadena a encontrar”
        condition:
                $cadena
}
```

Ya está. Si “cadena a encontrar” se encuentra en el archivo sobre el que estamos pasando YARA, dará un positivo.

Hay que observar cómo se referencia la variable que contiene la cadena. Sin ninguna expresión u operador que lo acompañe. Basta con referenciarla para dar a entender que **se trata de que sea positivo con tal de que la cadena aparezca una o más veces**.

4.5.2. Operadores booleanos

Los tres grandes clásicos: **and, or, not**. Como ya sabes, se trata de una conjunción, disyunción y negación.

No hace falta mucha explicación. Nos permite generar condiciones con estos operadores lógicos de este tipo:

```
strings:
        $cadena = “cadena a encontrar”
        $no_debe = “esto no debe encontrarse”
condition:
        $cadena and not $no_debe
```

Esa condición implicaría que **debe** aparecer $cadena **y no debe** aparecer $no_debe.

```
strings:
        $cadena1 = “cadena a encontrar”
        $cadena2 = “otra cadena a encontrar”
condition:
        $cadena1 or $cadena2
```

Aquí, relajamos la condición y basta encontrar cualquiera de las dos cadenas en el archivo.

4.5.3. Repetición de cadenas

Dado que muchas veces un mismo patrón se repite a lo largo del archivo, esta sintaxis nos permite indicar a YARA que cuente las veces o repeticiones de las cadenas a detectar y solo de un positivo en caso de que el número de repeticiones sea el indicado.

Para ello, contamos también con los operadores de comparación (idénticos a los usados en lenguajes de programación). Un ejemplo:

```
strings:
        $cadena1 = “cadena a encontrar”
        $cadena2 = “otra cadena a encontrar”
condition:
        $cadena1 == 3 $cadena2 > 5
```

En esta condición se debe encontrar la $cadena1 exactamente tres veces y $cadena2 más de cinco veces.

Recordemos que tenemos los siguientes operadores: ==, !=, <, >, <=, >=.

4.5.4. Operadores de posición relativa de la cadena

Muy usados, los operadores de posición nos permiten indicar a YARA exactamente dónde debe encontrar la cadena. Es decir, su posición relativa al comienzo de un archivo o la dirección virtual de un volcado de memoria o incluso la misma memoria virtual de un proceso vivo.

Básicamente, indicamos con at la posición deseada o con in en un rango determinado.

Ojo. El primer byte o carácter en un archivo es la posición 0

```
rule position
{
  strings:
    $cadena1 = "posición"

  condition:
    $cadena1 at 1
}
```

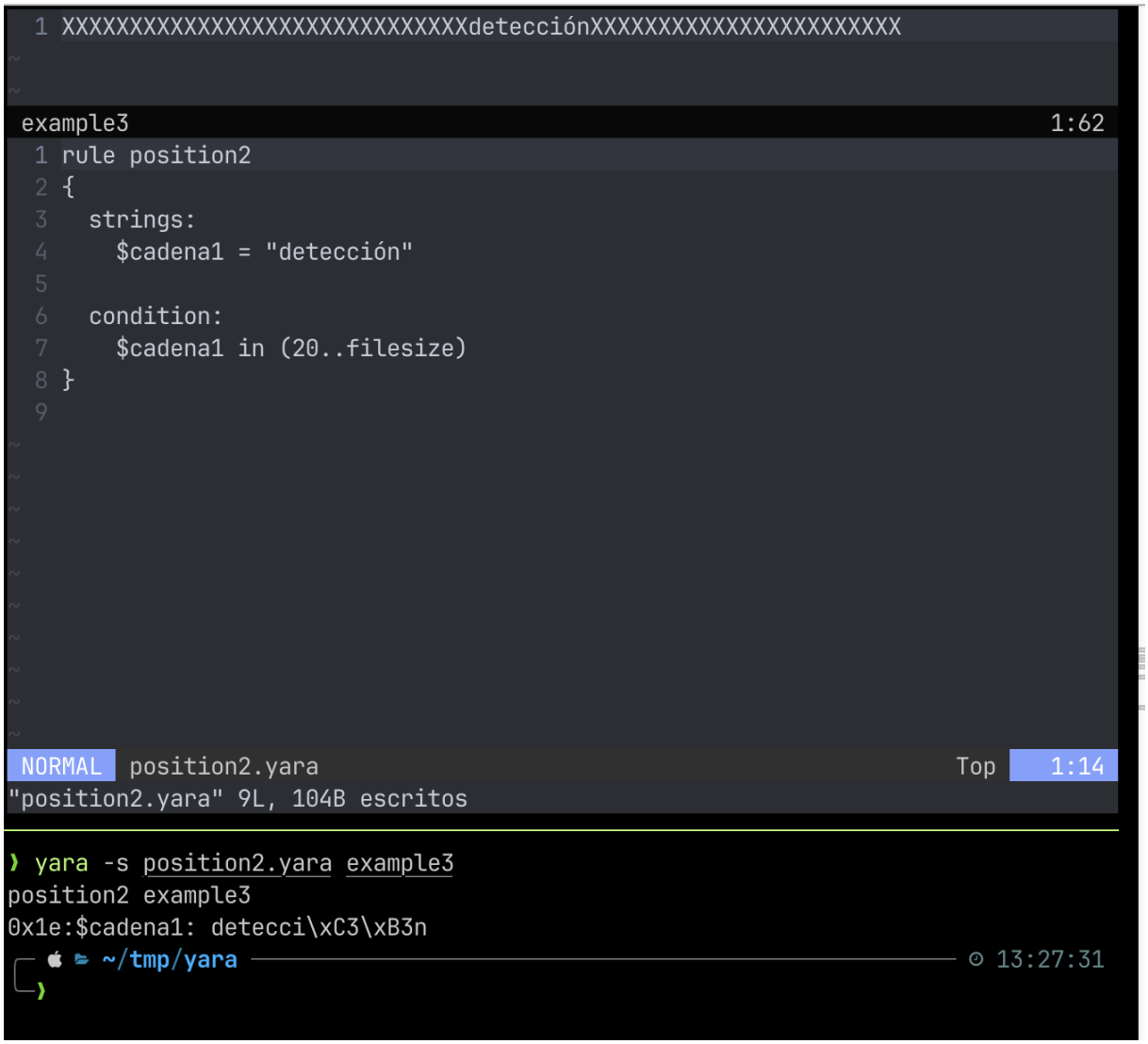
La regla anterior detectará la cadena $cadena1 en este archivo:

```
❯ cat example2
XposiciónXXXX
❯ yara -s position.yara example2
position example2
0x1:$cadena1: posici\xC3\xB3n
 ~/tmp/yara
❯ _
```

Ahora quitamos el primer carácter, el que está en la posición 0 y ya no lo detectará:

```
❯ cat example2
posiciónXXXX
❯ yara -s position.yara example2
 ~/tmp/yara
❯ _
```

Los rangos con el operador in son fáciles de entender con un ejemplo. En la imagen debajo tienes el archivo en cuestión, la regla con el operador in y su rango y abajo la detección.



Si la palabra "detección" hubiese estado al principio del archivo y nunca detrás de 20 caracteres no se hubiese detectado nada.

filesize es un operador que posee el tamaño del archivo. Es útil cuando definimos rangos o queremos descartar archivos por su tamaña. Más en la siguiente sección.

4.5.5. Discriminar por tamaño de archivo

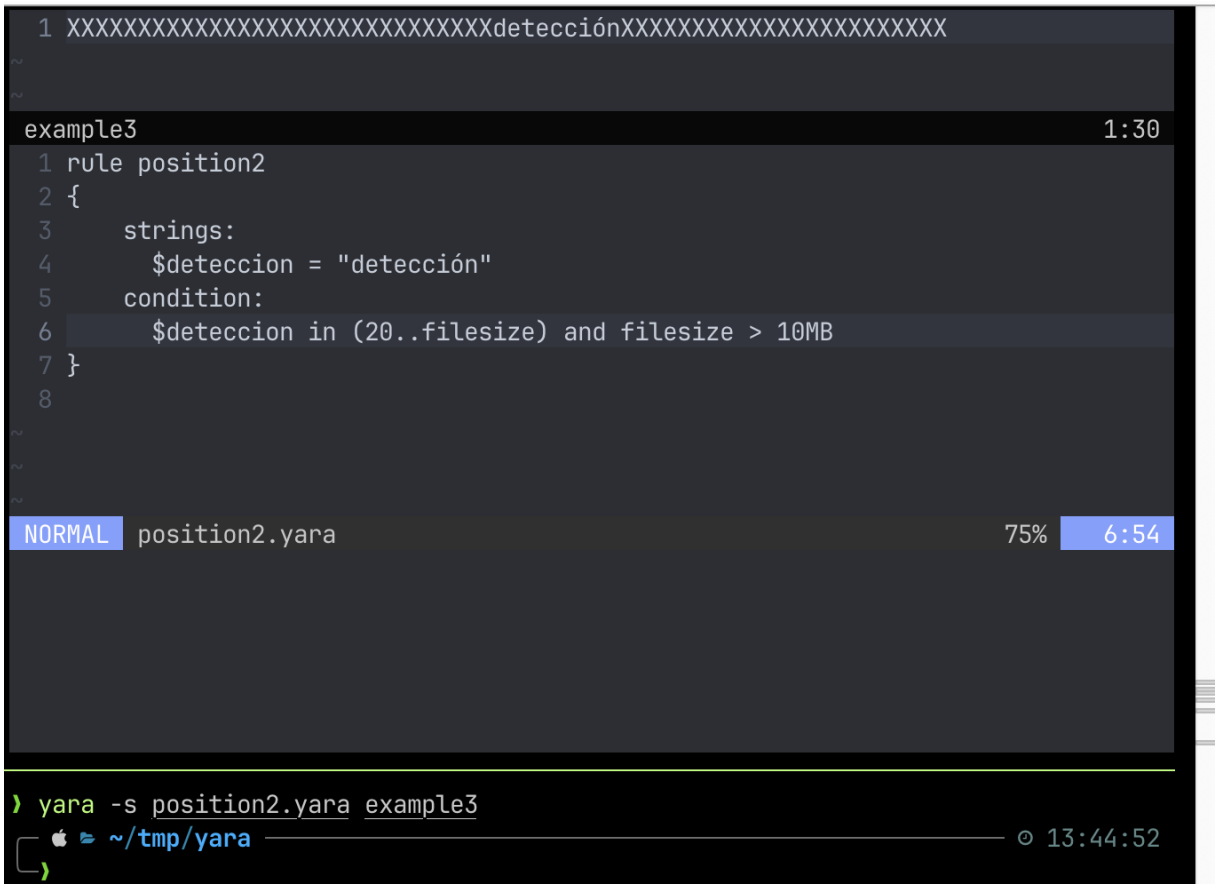
Las familias de malware poseen un tamaño característico. No es, por supuesto, una regla fija, pero habitualmente nos encontramos con tamaños que no suelen variar demasiado entre especímenes de una misma cepa.

En esto, nos puede llegar a sorprender que haya cepas de 40Kb y otras familias que se ocultan en archivos ISO con cientos de megas. Típico de troyanos en aplicaciones crackeadas.

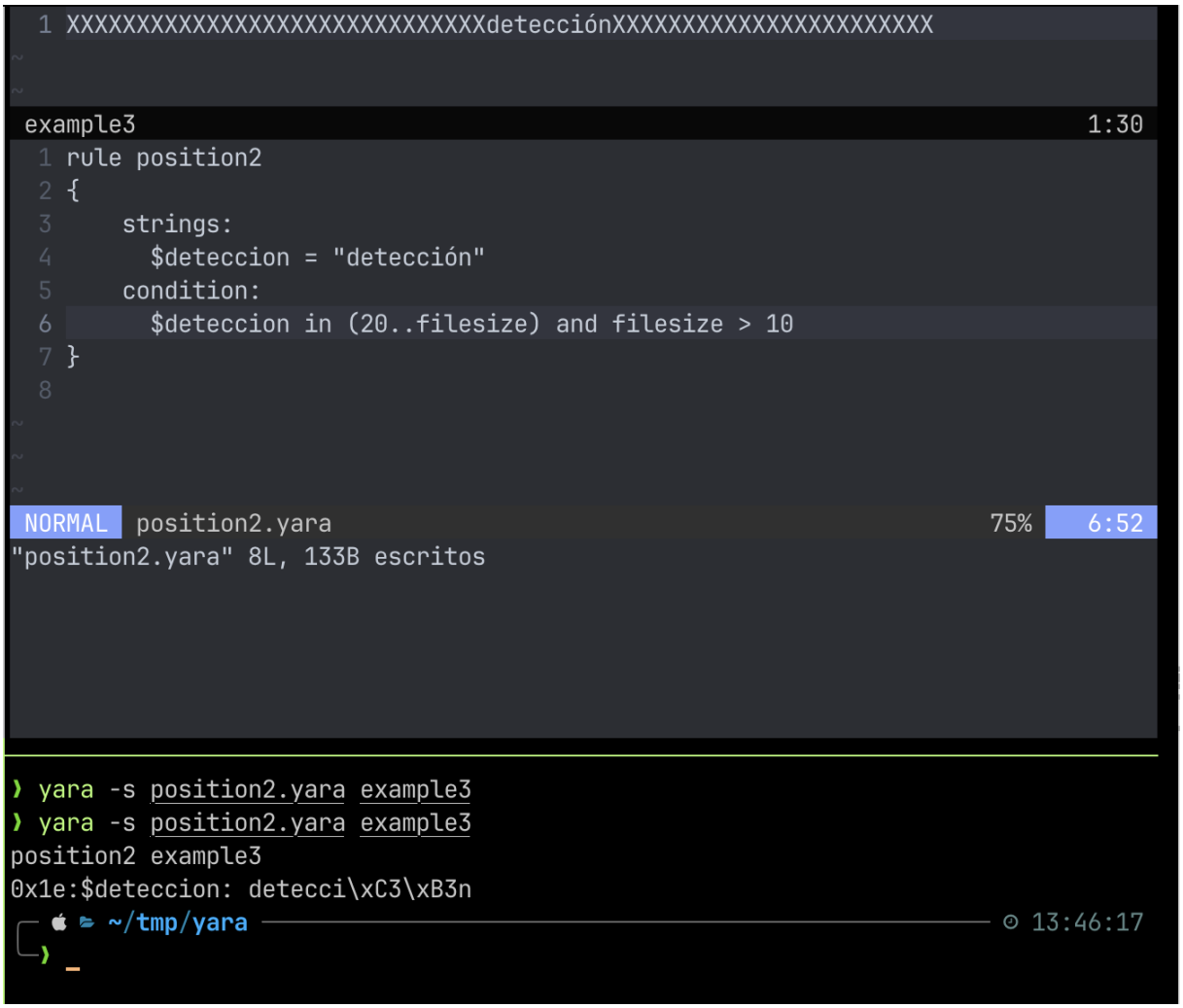
Como hemos aplicado antes en la sección de los operadores de posición. Es posible usar la palabra clave filesize que YARA traducirá inmediatamente al tamaño del archivo objeto de estudio.

Esto es de suma utilidad, no solo para indicar el rango como hemos visto antes sino para discriminar los análisis en base al tamaño del archivo.

Un ejemplo:



No detectará el archivo puesto que es muchísimo menor de 10 megabytes, pero si quitamos el MB (YARA lo interpretará como 10 bytes) entonces...



Por supuesto, lo podemos combinar con cualquier otro operador.

4.5.6. Conjuntos

Los conjuntos nos permiten granularizar aún más las detecciones indicando el número exacto de detecciones de todas las cadenas que hemos ido definiendo. La sintaxis nos permite indicar los conjuntos agrupándolos en paréntesis, como, por ejemplo: ($cadena1, $cadena2, $cadena3).

Cuando los tengamos agrupados, usamos el operador of para decirle a YARA cuantas de esas cadenas debe detectar. Esto lo podemos hacer mediante un entero o las palabras: any, all o none.

Así, podemos decir que para que haya un positivo se debe detectar la presencia de todas las cadenas:

all of ($cadena1, $cadena2, $cadena3)

O tan solo de dos:

2 of ($cadena1, $cadena2, $cadena3)

O tal vez, tan solo nos basta la presencia de una de ellas para que lo consideremos un positivo:

any of ($cadena1, $cadena2, $cadena3)

Disponemos de un atajo para no tener que referenciar todas las cadenas que hayamos definido. Esto lo hacemos con el operador them:

all of them

También podemos negar, en vez de todas o algunas de ellas, queremos que no detecte ninguna. Es decir, que como encuentre una de las cadenas entonces no de positivo en ningún caso. Esto lo haremos con el operador none:

none of them

4.6. Módulos

YARA posee un sistema de extensión a través de módulos. Primero vamos a ver su uso a través de los módulos que ya trae preconfigurados.

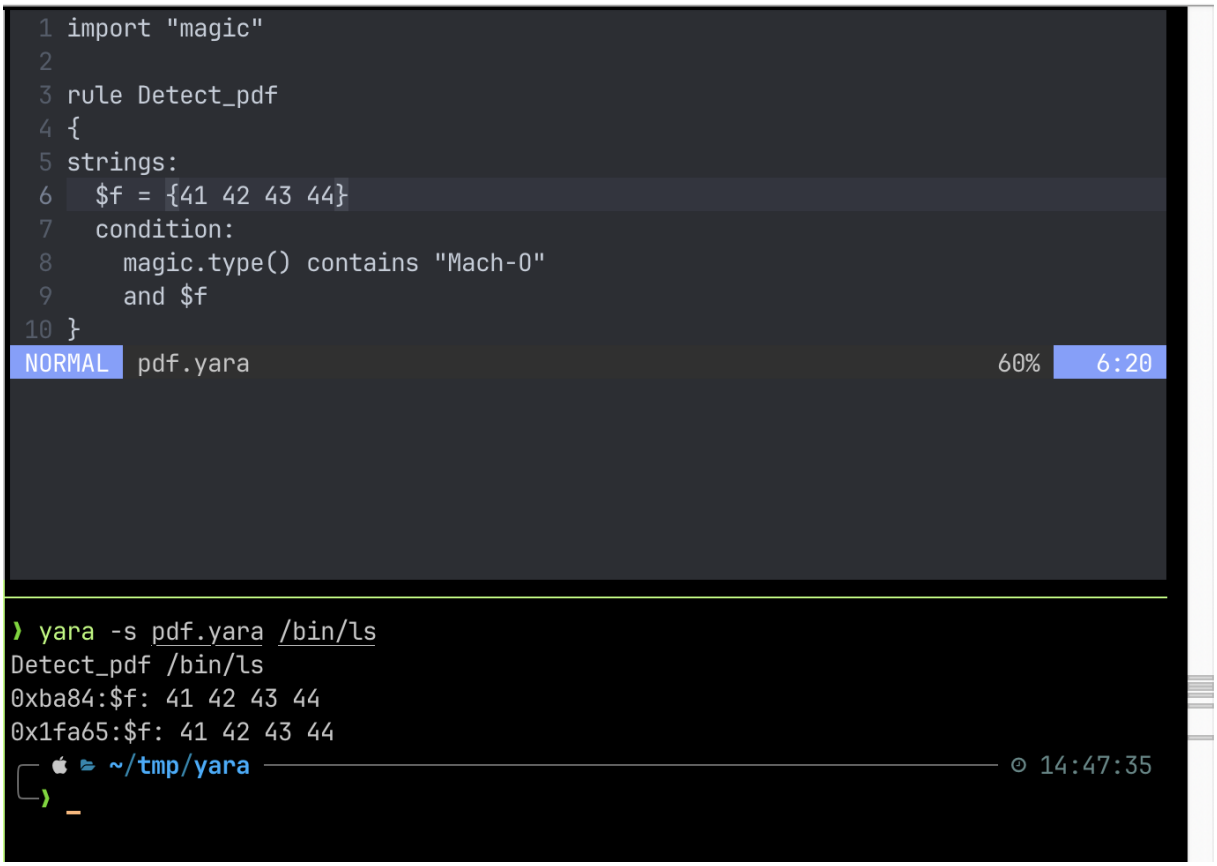
YARA posee varios módulos de serie. Por ejemplo, el módulo PE (Portable Executable, el formato ejecutable de los sistemas Windows) nos permite utilizar operadores y atajos propios de los archivos PE, mientras que el módulo ELF hará lo propio para ejecutables Linux.

La sintaxis para importar un módulo es sencilla:

import “PE”

Esto lo haremos justo al inicio de la regla, antes incluso de la declaración o nombre de la regla. Veamos un ejemplo donde usamos el módulo “magic” (la “magia” de un archivo es una secuencia de bytes característicos y fijos)

Como vemos en la regla, primero importamos el módulo (siempre con comillas) y en las condiciones, además de la cadena de bytes, la condición es que el tipo de archivo sea un ejecutable para MacOS.



4.7. Uso de YARA

YARA puede usarse desde la línea de comandos, invocando su ejecutable. También de forma programática a través de su API (lenguaje C). Otro de los usos más extendidos es a través de lenguajes de script vía wrappers, siendo Python uno de los más populares.

Para usarlo desde Python basta con instalar el módulo `yara-python` con, por ejemplo, pip.

Para las prácticas usaremos el ejecutable de línea de comandos. Comentaremos los usos principales.

4.7.1. Invocación básica

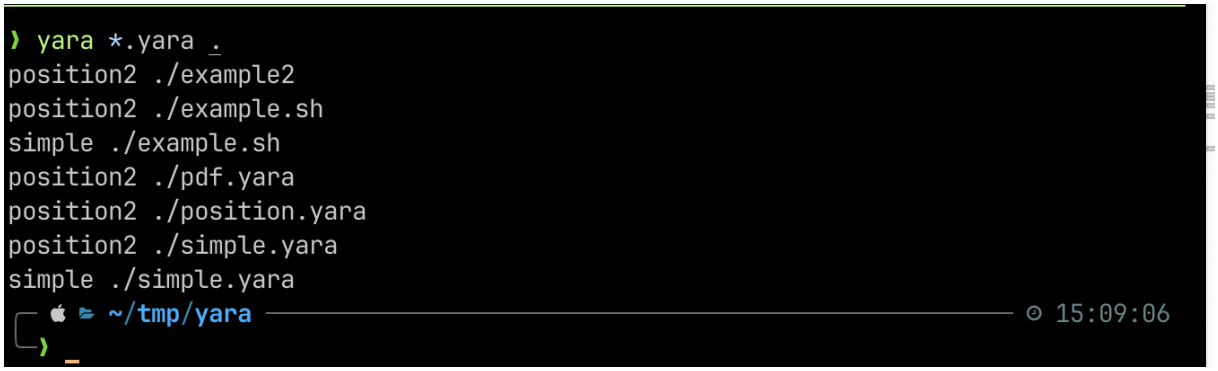
En primer lugar, YARA solo necesita saber dónde están las reglas y sobre que archivos ha de efectuar el escaneo de patrones.

Como hemos visto ya en algunos ejemplos, la sintaxis es:

```
yara <reglas a incluir> <directorio o archivo>
```

Lo habitual es que las reglas se incluyan con *.yara y el directorio sea un simple path o

ruta.



4.7.2. Algunas opciones interesantes

Como toda herramienta de línea de comandos, YARA posee varias opciones. Entre las más interesantes, podemos encontrar las siguientes:

-r Añade recursividad, por si tenemos los archivos en una jerarquía de carpetas.

--scan-list Designamos un archivo de donde tomará rutas para escanearlos

-s Mostrará en pantalla las cadenas que han resultado positivas

-g Muestras las tags que hayamos asignado a la regla (si es que tiene)

-t <tag> Solo ejecutará las reglas que tenga el tag e ignorará el resto

-i <nombre> Igual que tag, pero con el nombre de las reglas

-c Muestra el número de reglas positivas

4.7.3. Reglas compiladas

YARA puede compilar las reglas y es algo que harás si estás al cargo de un sistema de análisis automatizado. El porqué es fácil de concluir: velocidad es igual a rendimiento y mejor rendimiento es ahorro de costes.

El problema es la seguridad. Básicamente: **NO ADMITAS NUNCA REGLAS COMPILADAS DE TERCEROS**.

Compilado significa que no sabemos qué hará puesto que no vemos el código fuente sino

un binario al que tendríamos que aplicar ingeniería inversa para averiguarlo. Lo que se traduce en que no vamos a perder el tiempo con reglas compiladas que nos llegue de un tercero. Solo usaremos las compiladas por nosotros mismos.

Esto lo haremos con la utilidad de línea de comandos: yarac.

La sintaxis es simple:

yarac <regla a compilar> <nombre del binario de salida>

Es buena práctica tener nuestras reglas fuente en un directorio y en otro las compiladas. Cuando YARA se ejecute, tomará las reglas compiladas y usará estas. Nosotros nos quedaremos con los archivos fuente por si tenemos que leerlos o modificarlos y recompilarlos en caso necesarios.

Para ejecutar YARA con reglas compiladas le indicamos como hemos visto anteriormente:

yara -C <regla compilada> <archivos objeto de estudio>

4.8. Conclusión

YARA es una de las herramientas más potentes que existen en el área de detección y clasificación de muestras. De ahí la importancia que posee en los entornos de análisis de malware.

Es, como se ha recalcado, imprescindible que aprendas a manejarte de forma sobresaliente en el uso y escritura de reglas YARA. Muchos investigadores y laboratorios publican reglas de forma altruista, por lo que es común intercambiarlas e integrarlas en los sistemas de detección propios.

Además de lo mostrado aquí, te animo a proseguir investigando y aprendiendo su uso con los siguientes recursos:

https://yara.readthedocs.io/en/stable/index.html

https://github.com/InQuest/awesome-yara

https://github.com/Yara-Rules/rules

4.9. Otras herramientas de detección estáticas

No podemos dejar de mencionar que el abanico de herramientas disponibles para diseccionar en estático una muestra es inmenso. Los hay de todo tipo. Desde detección

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

de que tipo de compilador ha sido usado hasta medir la entropía de las secciones de un ejecutable o detectar si posee un packer y de qué tipo.

No obstante, todas estas herramientas se van a ver en el módulo específico para análisis de malware: "Técnicas de análisis de malware", donde ya se ha comentado se centrará en el análisis manual de una muestra.

Para evitar el solapamiento, simplemente se comentará que dichas herramientas se usan, pero de forma automatizada. Los datos que se saquen de estas formarán parte de la "ficha" de la muestra y serán útiles para realizar clasificaciones o búsquedas.

Por ejemplo, la herramienta **exiftool** se emplea para extraer los metadatos de múltiples tipos de archivo. Dicha herramienta tanto es usada por el analista de malware de forma manual como integrada en sistemas automatizados.

5. SANDBOXES

Ya tenemos las muestras identificadas por su hash, clasificadas previamente y puestas en la cola correspondiente que vendrá, en la mayoría de los sistemas, bajo el criterio de su tipo (ejecutable Windows, ejecutable Linux, pdf, Android...)

Como su propio nombre indica, la razón de una **sandbox** es que la **detonación** (ejecución) del malware no impacte colateralmente sobre un sistema o red en producción. Es decir, es de pura lógica que si el malware se ejecuta llegaremos a presenciar el efecto de su **payload**. Es evidente que no queremos infectar un sistema real e incluso uno de la misma red.

payload es la "carga" maliciosa de un malware. Por ejemplo, si se trata de un minero, el payload será cuando se active su función de minado en el sistema infectado. Si se trata de un ransomware, entonces el payload es la función de cifrado de los archivos secuestrados.

Ojo, también existen sandboxes en el propio sistema operativo e incluso dentro de un programa. Por ejemplo, Windows posee la virtualización de procesos, lo que impiden que salga fuera de su contexto de ejecución. En Linux se posee el concepto de chroot, que explota hábilmente Android desde sus inicios para crear una separación entre aplicaciones.

Luego, dentro de las propias aplicaciones también podemos encontrar sandboxes para aislar hilos o subprocesos particulares. Por ejemplo, la del navegador Chrome.

5.1. ¿Qué es una sandbox?

Es un sistema para la detonación controlada de malware; algo que ya hemos dicho. Pero también es un **sistema observable**. Esto es fundamental, dado que monitorizar los cambios que se efectúen en el sistema es la clave que nos va a proporcionar el comportamiento que realiza la muestra objeto de estudio.



Es evidente que la ejecución de un programa va a alterar el estado inicial en el que se encuentra un sistema operativo (descartando los cambios ajenos al proceso objeto de estudio, esto es, los propios del sistema operativo u otros procesos).

La observabilidad de un sistema se trata de eso. Se tiene la foto inicial o **snapshot** (el sistema limpio), se detona (activando toda la instrumentalización) y finalmente, se detiene el proceso y se para el sistema, obteniendo una foto final.

La comparación de la "foto" inicial con la final, nos dará los cambios efectuados. El estudio de estos cambios y su clasificación nos dará la clave para decidir si se trata de malware o es benigno...de momento.

Un **snapshot** es un estado guardado de un sistema en un momento dado. Es común en las máquinas virtuales tener la funcionalidad de poder guardar snapshot y poder usarlas posteriormente para volver a dicho estado de ejecución.

Importante. Como estamos hablando de un sistema automatizado, podrías pensar que se usan componentes muy diferenciales a los que usaría un analista incluso aficionado. Nada más lejos de la realidad. De hecho, muchas de las herramientas son las mismas, solo que el analista las suele lanza a mano o con un pequeño script y un sistema como el que estamos describiendo posee una aplicación que orquesta todos los componentes para mantener un ritmo de análisis adecuado.

De hecho, la gran mayoría de sandboxes no son otra cosa en el fondo (hablando de sistemas operativos habituales) que un hipervisor corriendo sistemas operativos virtualizados como las que usarás (y ya habrás usado) a lo largo del curso.

5.2. Características deseables en una sandbox

Aislamiento

En primer lugar, antes de todo, no deseamos afectar a otros sistemas o redes, como hemos dicho. Al final se trata de **aislamiento**.

El aislamiento significa que todo lo que interaccione con el exterior de la sandbox ha de ser controlado. Por lo tanto, el sistema de sandbox no debe alterar nada de su exterior.

¿Qué hay de la red? podrás preguntarte. Y es una buena pregunta, porque el malware puede necesitar conexión a Internet para proseguir con la ejecución. Bien, pues en este caso hay que darle acceso, pero un acceso muy restringido, controlado y monitorizado. Vamos a dedicarle más adelante su propio apartado para explicarlo a fondo.

Restaurabilidad

Básicamente, si el sistema no puede volver a su punto inicial (sistema limpio) no nos sirve. Pero es que, además, si puede hacerlo, pero tarda demasiado tiempo no nos es rentable (de nuevo: millones de muestras).

Por lo tanto, necesitamos un sistema que podamos restaurar de forma rápida. Habitualmente, con ese nivel de muestras pendiente de detonación lo que se hace es tener múltiples sistemas concurrentes que puedan ir tomando muestras de la cola mientras otros sistemas están detonando o están en modo restauración.

Observabilidad

De este ya hemos hablado, pero vamos a recalcar este aspecto deseable. Si tenemos una plataforma que nos deja ejecutar una muestra, pero es difícil o no nos permite monitorizar a fondo los cambios que la muestra produce, no nos va a servir de mucho.

Esto lo podemos encontrar en software cerrado o tecnologías hardware de las que no se dispone de mucha información.

Necesitamos datos, observar los cambios, ver qué eventos se producen, que altera en el disco, que artefactos aloja en RAM, en que procesos se inyecta, que tráfico produce y cómo podemos grabarlo para reproducirlo luego...en fin, multitud de sensores que debemos desplegar en nuestra sandbox y que, si el sistema no nos permite actuarlos, como hemos comentado, no podremos disponer de información esencial para nuestros análisis.

5.3. Aislamiento de red

Es un error que el malware pueda transmitir sin cortapisas hacia Internet. Existen múltiples razones, algunas ya las hemos comentado:

- Utilizar una dirección IP "normal" hará que a los pocos análisis los creadores de malware la detecten, la bloqueen o peor, manipulen la respuesta.

- Una IP conocida señalará la identidad, tarde o temprano, de quien está detrás de las detonaciones.

- Es posible que se modifique el contenido o se bloqueen las respuesta dependiendo del bloque IP geolocalizado.

- Muchas infraestructura son de un solo uso. Es decir, una vez se genera la petición y llega al servidor, no volverá a responder.

¿Qué hacer entonces?

En primer lugar, lo ideal es que no tenga acceso como hemos dicho. Pero claro, entonces no sabríamos que "se hablan" entre el malware y su centro de control, así como otras actividades de red.

Existen varias opciones y algunas compatibles entre sí.

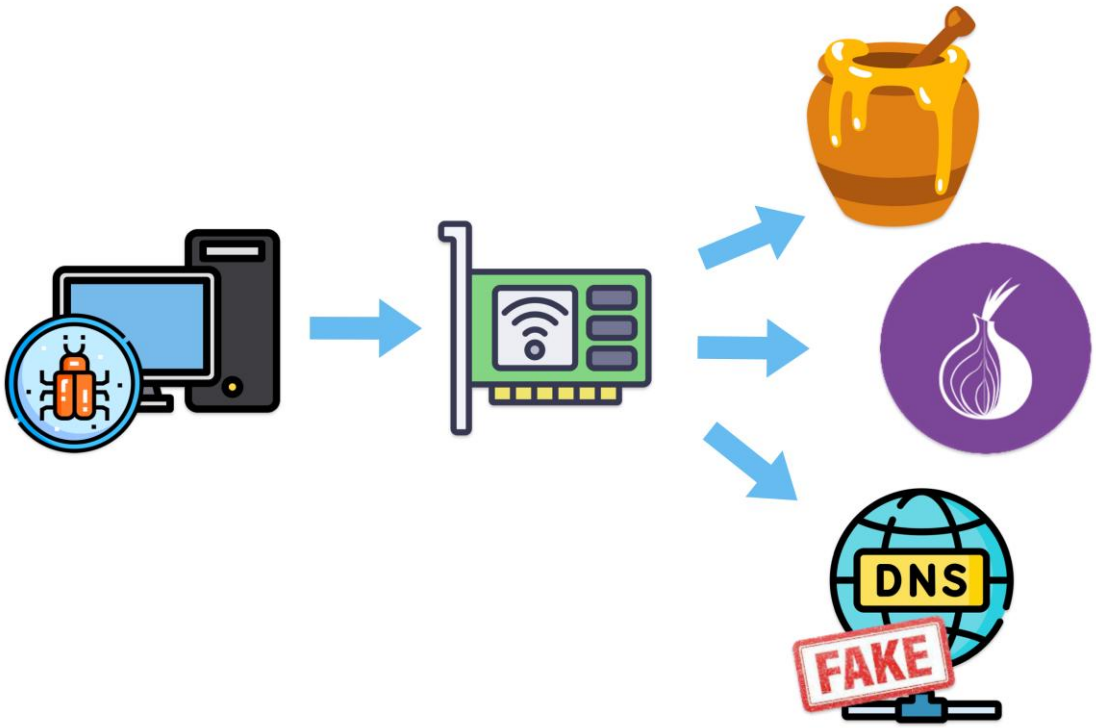


ILUSTRACIÓN 4 MODOS DE OCULTACIÓN DEL ORIGEN Y ANONIMATO

5.3.1. Emulación de infraestructura de red

Es posible emular cierta infraestructura de red. Por ejemplo, servidores DNS que responderán a una dirección IP controlada por nosotros. En esa dirección podemos optar a instalar un software tipo honeypot. Esto serviría para malware muy genérico (y sencillo) pero no engañaríamos al malware sofisticado.

No obstante, nos puede servir si no queremos optar a procesar la respuesta del servidor en una primera fase y detonar después si observamos o tenemos interés en la conversación malware-servidor.

Para registrar y manipular peticiones DNS, existen varias herramientas de este tipo, sobre todo si buscas por "fake dns".

Un ejemplo: https://github.com/Crypt0s/FakeDns

Respecto a los honeypot, la carta es abundante:

https://github.com/paralax/awesome-honeypots

Existe mucha variedad. Desde un simple script que manipule las respuestas y levante un servidor web donde se necesite que siempre responda HTTP 200 OK a entornos completos en la nube donde el malware creerá que está dentro de la red de una empresa.

Todo depende del presupuesto y el tiempo del que dispongamos.

5.3.2. Interposición de proxies, VPN, Tor

Aquí abrimos un poco la puerta al malware respecto de las comunicaciones, pero de forma controlada en el sentido de que no se revele el origen.

Las opciones son clásicas, pero la idea siempre es la misma: ocultar el origen.

La idea es centralizar las peticiones en un nodo que obrará de proxy (y que suele ser la máquina que efectúa la captura de tráfico a pcaps) y redirigirá el tráfico de manera conveniente.

- Si se necesita responder desde un país o zona concreta levantará un proxy en dicha zona.
- Si es posible navegar vía Tor, recurrirá a dicha red para ocultar el origen. Aunque es bastante probable que el malware o mejor dicho, el centro de control banee la red Tor.
- Si el malware realiza peticiones de un solo uso, entonces deberemos recalcular dicha petición, cambiar de IP o parámetros que tome del sistema para regenerar una nueva petición fresca.

5.4. Sandboxes comerciales

Existe un gran surtido de sandboxes comerciales. Conscientes de la necesidad de análisis rápidos, masivos y fiables (algo que suele ser contradictorio), las empresas han invertido una gran cantidad de medios en producir servicios para el consumo de análisis automatizados. En este sentido, hay variedad de soluciones que compiten entre ellas para ofrecer servicios de distinto calado.

Lo habitual, si entras a formar parte del equipo de Inteligencia de Amenazas o Ingeniería Inversa y Análisis de Malware de una empresa, es que obtengas acceso a una cuenta profesional en una de estas soluciones comerciales.

No nos vamos a parar solamente en ofrecer un escaparate; ni por asomo es la intención. De esta sección te tienen que quedar las cosas claras respecto a **saber evaluar una solución comercial respecto de las demás** y **determinar qué características ofrecen y cuáles se adaptan a tus necesidades**. Tareas que te encontrarás en tu puesto de trabajo.

5.4.1. Principales marcas

Intezer - https://intezer.com/
Joe Sandbox - https://www.joesecurity.org/joe-sandbox-reports
Hybrid Analysis - https://www.hybrid-analysis.com/
Any.RUN - https://app.any.run/submissions/

Hay muchas más, algunas sin reporte público ni cuenta gratuita, por lo que no nos interesa ya que no podrás, siendo alumno, interactuar con ellas.

Recuerda también que Virustotal posee un interfaz de subida de archivos en el que detonarán la muestra, si procede, en varias soluciones sandbox. Recuerda también que subir un archivo a este servicio supone que lo vas a compartir con toda la industria antimalware. Piensa mucho si vas a subir un archivo a este tipo de servicios porque puede contener información confidencial.

5.4.2. ¿Qué parámetros debemos tener en cuenta en este tipo de soluciones?

Una sandbox comercial vende y ofrece varias cosas:

- Privacidad de resultados.
- Tiempo de procesamiento.
- Versatilidad de opciones de configuración y sistemas disponibles.

Muchas de estas sandbox tienen interés en que los usuarios suban archivos para análisis. Puesto que otra línea de negocio es intercambiar muestras nuevas (que les llega de los usuarios) y producir feeds de inteligencia.

Ser cliente de pago presupone que los análisis y muestras que subas serán solo tuyos y privados. No obstante, revisa la letra pequeña de los EULA.

El **tiempo de procesamiento** es el coste que les produces cuando subas una muestra: el

consumo de CPU. Al final lo tienen que repercutir si o si a tu costa, obviamente. El caso es que no es lo mismo detonar una muestra con un límite de un minuto que configurar la detonación a 300 segundos. De hecho, es algo que los creadores de malware saben y emplean en muchas muestras para no ejercer ninguna acción hasta que no pase cierto tiempo.

Respecto de las opciones y prestaciones, las veremos en la siguiente sección.

5.4.3. Opciones y prestaciones de las sandboxes comerciales

Distintas versiones de sistemas operativos

En primer lugar, habitualmente se posee un sistema base ligero. Por ejemplo, un Windows 7. Como cliente, esto no te interesa, a ellos sí puesto que es más económico restaurar y arrancar una imagen de un sistema operativo anterior.

Una versión antigua o no soportada es ligera para la sandbox, pero no estará actualizada, por lo que es posible que mucho malware actual no llegue ni a funcionar. Necesitas asegurarte de que puedes tener acceso a distintas versiones: anteriores, actuales, de 32 bits (hay algunos bichos curiosos...) y por supuesto, con diferentes idiomas dado que existe malware que discrimina la ejecución en base al idioma detectado. Al final se trata de montar un escenario realista, donde el malware se sienta cómodo y crea estar corriendo en un sistema real.

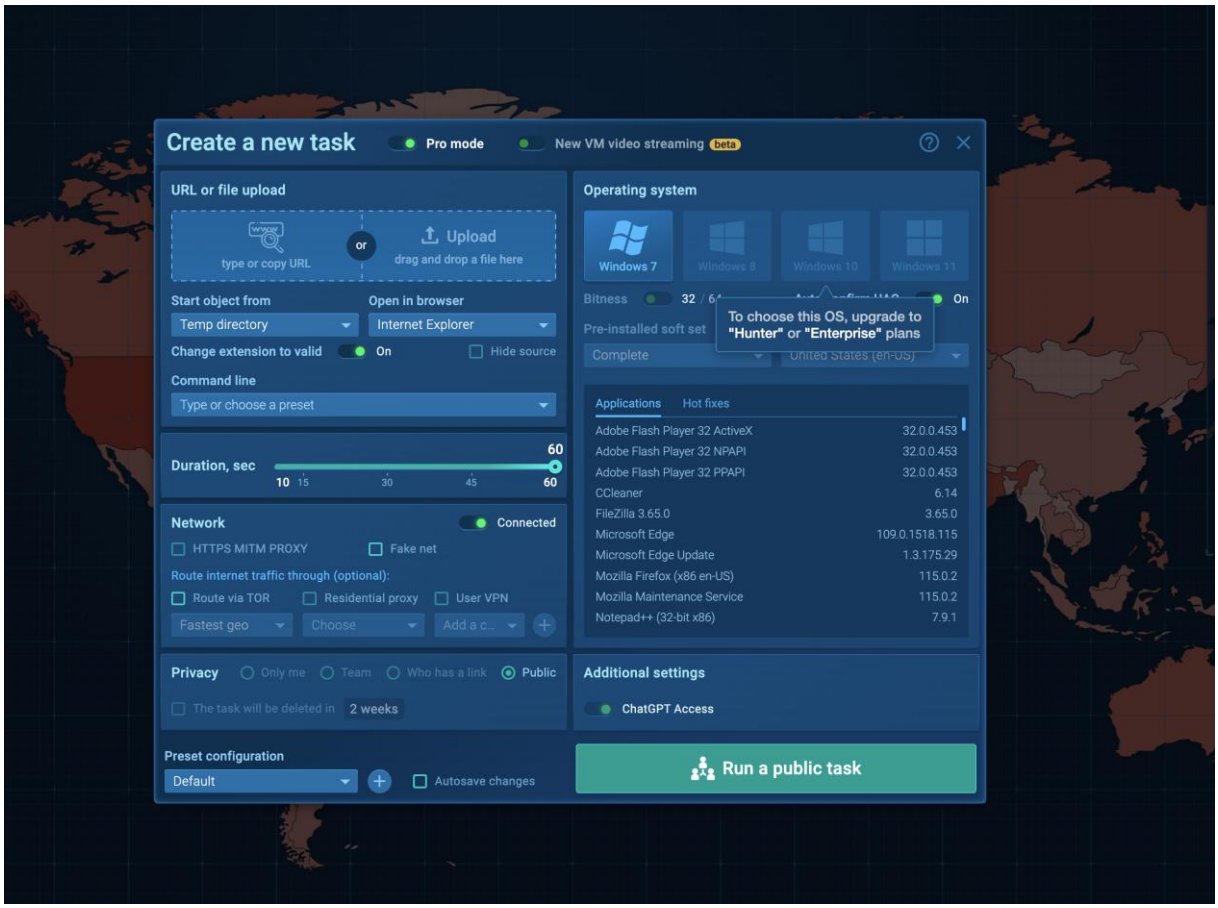


ILUSTRACIÓN 5 SÍ QUIERES UNA VERSIÓN DE WINDOWS MÁS ACTUALIZADA DEBES SER CLIENTE PRO

Adicionalmente, y como puede verse en la imagen, se puede tener la posibilidad incluso de configurar qué software viene preinstalado en la máquina para darle más realismo, pero también para poder experimentar con malware que puede explotar software adicional (samba, etc.) y observar su comportamiento.

Disponibilidad de acceso vía API

Las interfaces web son muy bonitas y vistosas. Pero al final, una sandbox en un sistema automatizado es un componente más; un eslabón de la cadena. Es decir, no vas a detonar manualmente a través de la interfaz, será algo excepcional o porque el analista quiere echar un vistazo antes de "operar".

Por lo tanto, es fundamental disponer de una API para insertar la sandbox dentro de la cadena. El estudio de las posibilidades de la API ofrecida por la sandbox resulta crucial, porque no te dará más prestaciones que las que ofrece.

Debes fijarte bien en los **valores de admisión o ratio de ingesta**. Esto es, el caudal de muestras que admite. Si te dicen que te limitan la subida a una muestra por minuto, debes ser consciente de que no podrás subir más de 1.440. Una cifra que puede quedarse corta rápidamente.

Otra cosa que debes estudiar es **¿Cómo recuperas el informe de análisis? ¿Qué formatos soportan?** Esto es, cuando subes la muestra no obtienes el informe inmediatamente. La muestra se encola y luego se detona, finalmente se produce un informe. Cuando subes la muestra, habitualmente, te devuelven un valor (token) para recuperar posteriormente el informe. Debes analizar qué tiempos suelen manejar en el sentido de que no te interesa que lances una muestra y tarde horas en ser analizada.

Respecto al formato de salida, deberían ofrecer varios: un STIX, json, xml, csv, etc. Cuantos más mejor. Si poseen STIX, es una ventaja puesto que ya posees gran parte del trabajo hecho (más de STIX en su propio capítulo). Hay sandboxes que solo tienen un formato y si te va a costar mucho procesarlo (no debería ser problema, no obstante).

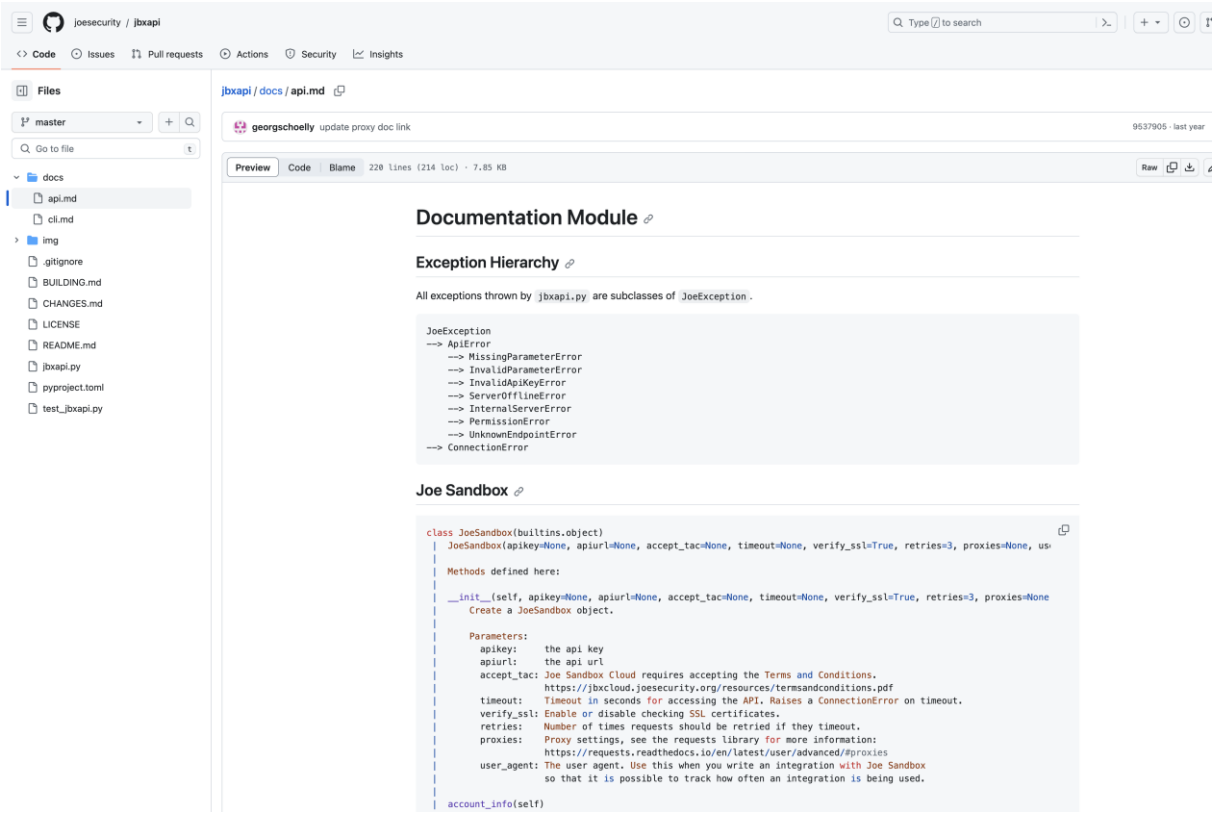


IMAGEN 5 DETALLE DE LA DOCUMENTACIÓN DE LA API PYTHON DE JOESANDBOX

Cuota

Ya hemos hablado de la admisión, pero otro parámetro es **¿Cómo cuentan los consumos?** ¿Es por muestra, análisis, consumo de API? Es algo que se deja bastante claro, pero como siempre, debes evaluar que se adapta a las necesidades de tu sistema.

También debes estar atento a si la cuota que te sobra puedes acumularla o no. Me explico: si es por subidas mensuales y no las gastas, es posible que lo no consumido se pierda; una lástima. Si es por cuota de llamadas de API sin límite, igual te interesa comprar

paquetes según la vayas consumiendo.

Hay que estar atento también a como se puede dividir esa cuota. ¿Puedes tener un equipo, generar múltiples tokens? Piensa que si tienes gente trabajando contigo querrás que la API pueda ser utilizada por más de una persona o instancia del sistema.

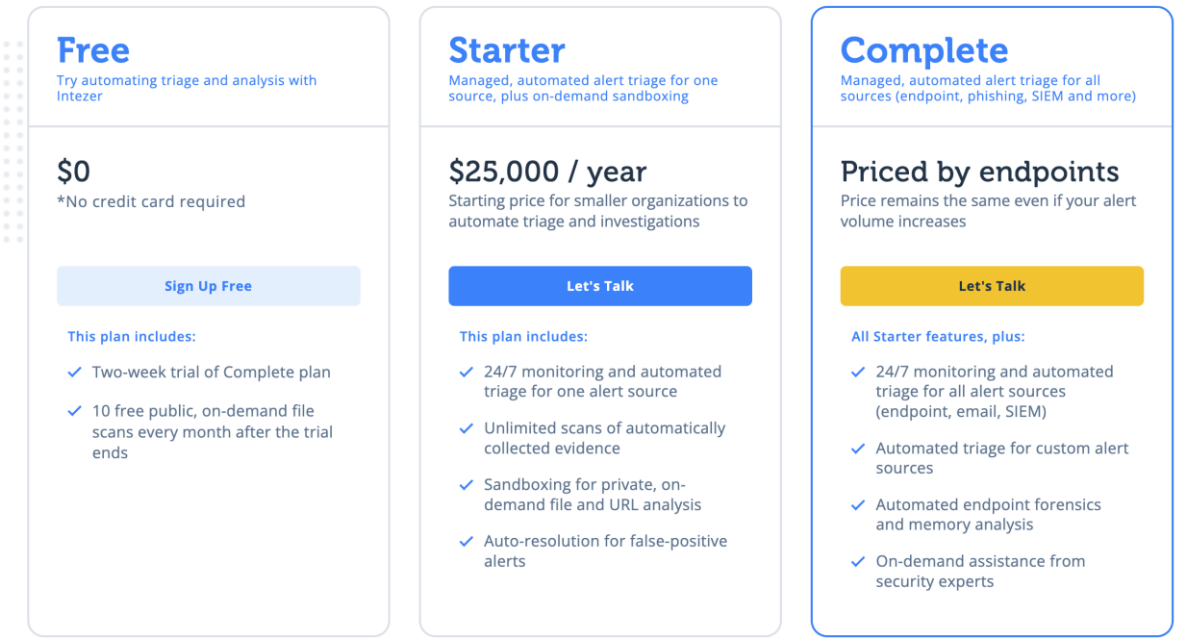


IMAGEN 6 DE 0 A 25.000 DÓLARES. EL PRECIO SE PUEDE IR DE LA MANO.

Reanálisis

Esta característica es útil, aunque quizás no vital. Existe la posibilidad de que un malware haya sido analizado y posea ciertos resultados: sospechoso, etc. Puede incluso ocurrir que la infraestructura necesaria para su funcionamiento estuviese caída. ¿Sería interesante volver a detonar la muestra?

Por supuesto, pero es una característica que no todas las sandbox poseen: guardar un informe y la propia muestra y ofrecer la posibilidad de hacer un reanálisis para comparar resultados.

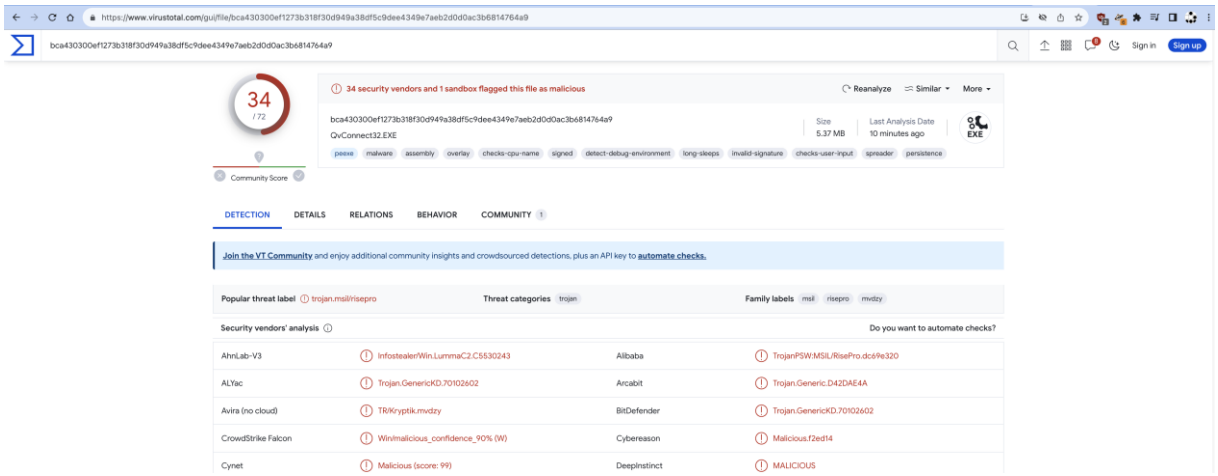


IMAGEN 7 DETALLE DE VIRUSTOTAL, DONDE PUEDE VERSE LA CAPACIDAD DE "REANALIZAR" LA MUESTRA

5.5. Sandboxes Open Source

5.5.1. Cuckoo y CAPE2

No hay mucha elección. La sandbox más conocida y popular es Cuckoo, proyecto archivado en 2021 según el repositorio de Github. Sus autores anunciaron una reescritura del proyecto, pero a fecha de la elaboración de estos materiales no se tiene noticia.

De ese proyecto surgieron varios forks. El más conocido es CAPEv2. CAPE funciona bastante bien, pero al necesitar de Cuckoo y al estar congelado se complican los avances.

Desplegar y arrancar una instancia funcional de CAPE no es trivial. Las dependencias y versionado de las distintas herramientas y librerías usadas hacen difícil rebasar su escalón de entrada. No obstante, es un sistema excelente para presupuestos ajustados y por supuesto, para aficionados al análisis de muestras.

Existe una instancia pública (aquí) para uso particular (previo registro) y gratuito. También, como en el resto de sandboxes, es posible consultar los informes de las detonaciones de otros usuarios.

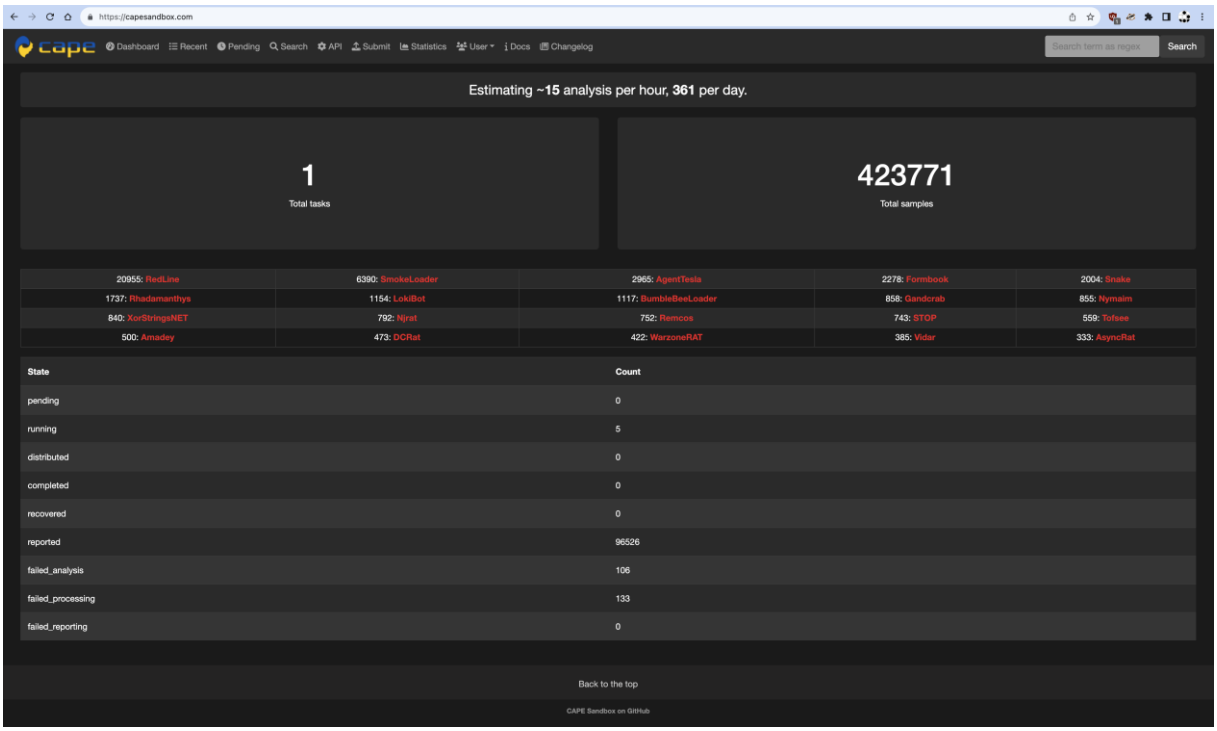


ILUSTRACIÓN 6 INTERFAZ WEB DE LA INSTANCIA PÚBLICA DE CAP

Esperemos que el desarrollo prosiga y se pueda alcanzar más funcionalidades y prestaciones de esta gran sandbox y única con licencia libre.

5.5.2. Manalyzer

Tenemos interfaz web y repositorio con el código del proyecto (licencia GPLv3).

No se podría hablar de sandbox propiamente porque solo realiza funciones de análisis estático, pero para un primer análisis de descarte rápido previo paso a otra sandbox nos puede servir.

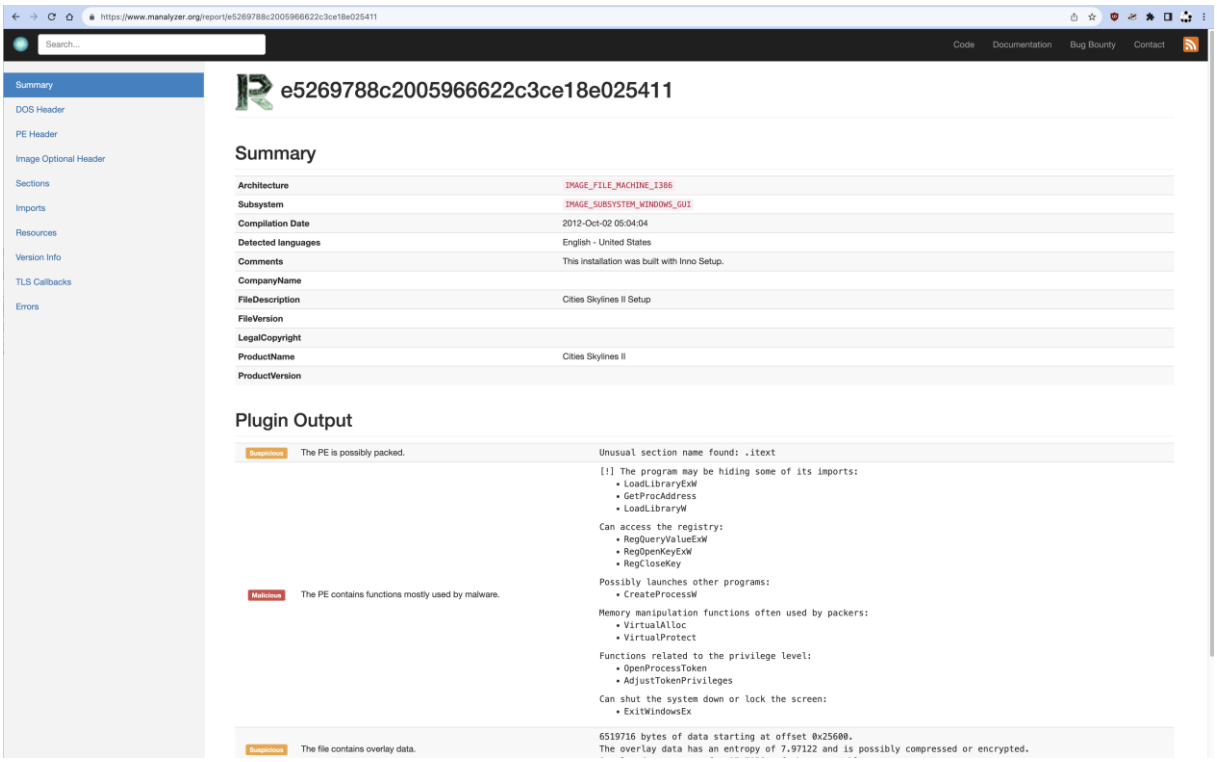


ILUSTRACIÓN 7 INFORME PÚBLICO DE MANALYZER

Es precisamente por lo mencionado por lo que puede sernos útil una herramienta de este estilo. Por ejemplo, supongamos que tenemos una cuota de análisis en una sandbox comercial que no queremos malgastar porque son números bastante apretados.

Si tenemos un sistema que descarte en estático las muestras que no nos interesa analizar, antepondríamos un componente de este tipo para hacer el estático en "casa" y enviar fuera aquello que se sale de nuestras posibilidades, ahorrándonos un buen bocado de cuota.

5.6. Handmade Sandbox

La detonación es una de las partes más importantes del flujo de trabajo. Pero es la más compleja y costosa en términos de recursos computacionales (dejando al margen el almacenamiento de los más que posibles millones de muestras).

Hasta ahora, hemos presentado soluciones comerciales que nos permiten derivar el problema de la detonación a un tercero a cambio de una suma de dinero. También hemos visto como existen (en realidad, solo hay una) soluciones open-source, limitadas pero suficientes para un entorno modesto.

Existe una tercera opción, utilizada cuando apenas existían sandboxes, ni comerciales ni open-source. Básicamente consiste en instrumentalizar y automatizar una máquina virtual con un sistema repleto de scripts y programas ejecutados en secuencia para

extraer los cambios e interacciones, así como eventos, producidos por la detonación. Que no es distinto a las sandbox ya comentadas.

El principal problema es que se necesita actualizar constantemente las herramientas desplegadas en la imagen del sistema operativo virtualizado. Además, se debe afinar mucho como se aísla de la red, captura de datos y sobre todo, "tunear" el sistema para que el huésped (el malware) no se dé cuenta de la virtualización y aborte o derive la ejecución y no dispongamos de telemetría alguna.

De hecho, la detección de virtualización es uno de los grandes problemas y desafíos de las sandbox de todo tipo y de esto hablaremos en la siguiente sección.

5.7. Contramedidas frente a la detección de la virtualización

Existe una gran cantidad de malware que no se ejecuta o realiza una derivación cuando detecta que se está ejecutando en una máquina virtual.

Los hay de dos tipos: abortan la ejecución y no se ejecutan o se ejecutan pero realizan una función completamente diferente y ausente de acciones maliciosas para no levantar sospechas.

Por supuesto, esto es otra carrera entre creadores de malware e investigadores y analistas. En el que los últimos intentan mitigar la detección y los primeros descubren un nuevo método para detectarla.

Existen numerosos papers, presentaciones, herramientas, etc. Un largo reguero de recursos que esta lucha ha dejado y sigue dejando y del que, por supuesto, podemos recurrir para equilibrar la lucha de nuestro lado.

Una excelente herramienta es pafish. Creada por Alberto Ortega. Con ella podremos descubrir algunas de las técnicas más empleadas por el malware para recurrir a la detección de entornos de análisis. La usaremos para precisamente lo contrario; desplegar contramedidas para evitar la detección e ir comprobando con la herramienta cuáles son efectivas o no.

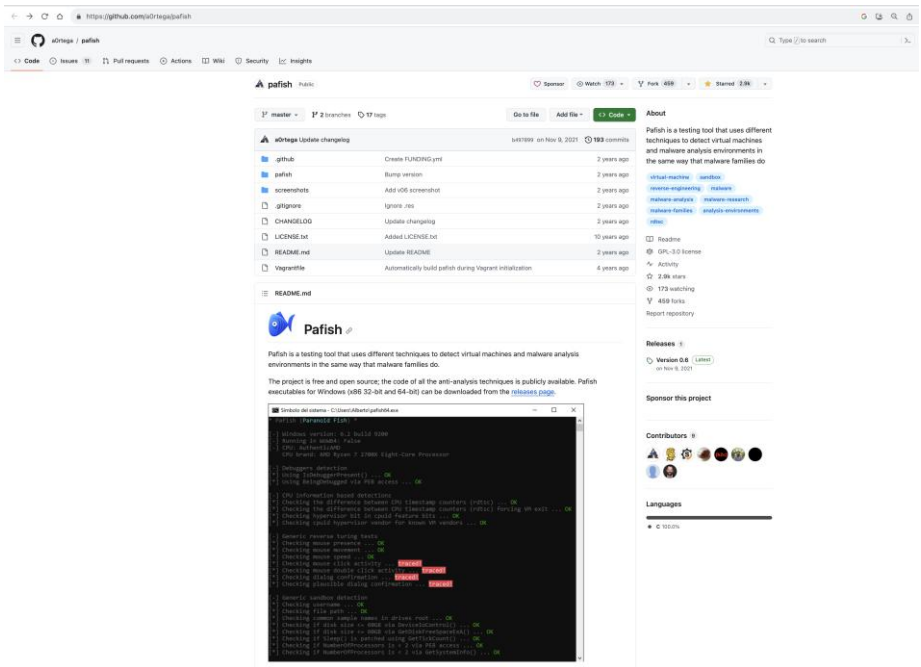


IMAGEN 8 REPOSITORIO DE CÓDIGO DEL PROYECTO PAFISH

En este sentido, aunque lo verás más adelante en otras asignaturas, no queda más remedio que detectar si el malware posee dichas capacidades de, a su vez, detección y parchearlo para que no las emplee (naturalmente, existen rutinas para detectar este parcheo…), algo que estaría fuera del dominio de las sandbox por su complejidad.

Uso de sistemas reales

Respecto a la virtualización, algunas sandboxes emplean sistemas operativos de forma nativa en máquinas con hardware especial para restaurar el sistema operativo en pocos segundos. Una opción para diversos tipos de muestras difíciles de analizar de otra forma.

6. REGLAS SIGMA

6.1. Introducción

Más reglas. En esta ocasión se trata de las reglas SIGMA, que podríamos decir que son el equivalente a las reglas YARA (indicadas para archivos) para archivos de eventos, registros o más conocidos como "logs". Su uso está muy extendido en los sistemas SIEM (Security Information Event Management) que "sufren" quienes trabajan en los SOCs.

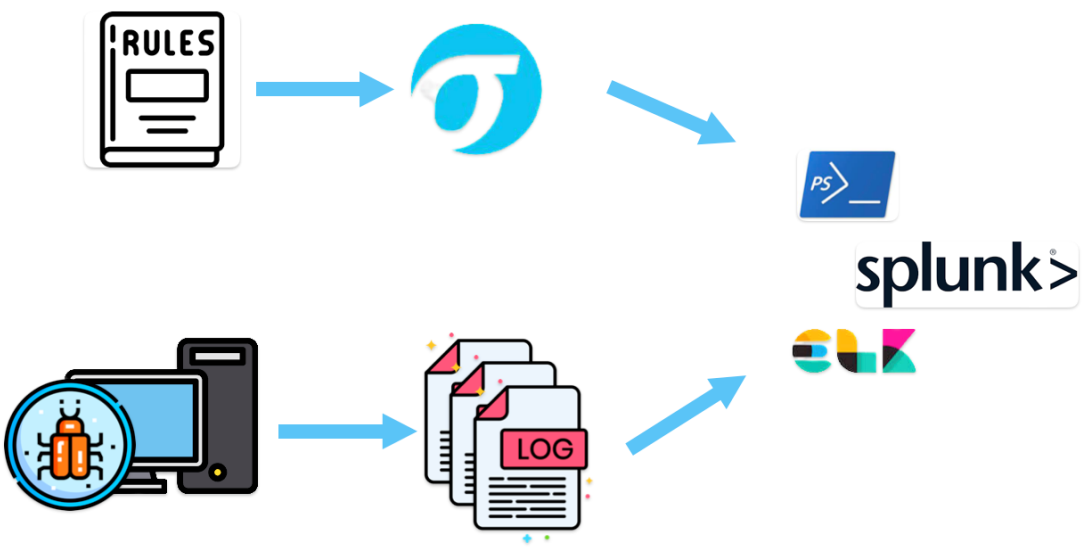
Sin embargo, antes de que pensemos que se trata de un sistema en el que vamos cargando las reglas y "disparándolas" sobre un conjunto de ejecutables, las reglas SIGMA son un animal completamente distinto a YARA en la filosofía de funcionamiento.

En primer lugar, las reglas SIGMA son genéricas. No existe el concepto de "pasar por reglas SIGMA". Antes debemos convertir o transformar la regla SIGMA a un formato de salida. Ahora veremos como...

La razón por el cual se diseñó SIGMA es que los analistas tenían que reescribir una regla de detección de interés a varios formatos, puesto que existen multitud de productos y cada cual tiene su propio lenguaje de reglas. SIGMA viene a dar solución a ese problema, por lo que aspira a: "escribir una regla y convertir a varios formatos". No soporta ni todos los formatos de entrada ni posee formatos de salida para todos los productos empleados en el tratamiento de eventos de seguridad.

6.2. Funcionamiento de SIGMA

Como hemos dicho, SIGMA "traduce" reglas genéricas en el lenguaje de destino que soporte la herramienta que esté usando el analista. Si nos interesa una regla concreta, usamos una utilidad de sigma (sigma-cli), convertimos la regla y la integramos en nuestro sistema de detección.



Imaginad, por ejemplo, que queremos detectar eventos de Windows. Descargamos las reglas SIGMA que están disponibles en el repositorio oficial de reglas SIGMA. Obtengo las siguientes:

```
❯ ll sigma/rules/windows
total 0
drwxr-xr-x@   32 davidgarcia  staff   1,0K 11 oct 18:03 builtin
drwxr-xr-x@   14 davidgarcia  staff   448B 11 oct 18:03 create_remote_thread
drwxr-xr-x@   11 davidgarcia  staff   352B 11 oct 18:03 create_stream_hash
drwxr-xr-x@   14 davidgarcia  staff   448B 11 oct 18:03 dns_query
drwxr-xr-x@   14 davidgarcia  staff   448B 11 oct 18:03 driver_load
drwxr-xr-x@    7 davidgarcia  staff   224B 11 oct 18:03 file
drwxr-xr-x@   92 davidgarcia  staff   2,9K 11 oct 18:03 image_load
drwxr-xr-x@   46 davidgarcia  staff   1,4K 11 oct 18:03 network_connection
drwxr-xr-x@   18 davidgarcia  staff   576B 11 oct 18:03 pipe_created
drwxr-xr-x@    5 davidgarcia  staff   160B 11 oct 18:03 powershell
drwxr-xr-x@   29 davidgarcia  staff   928B 11 oct 18:03 process_access
drwxr-xr-x@ 1039 davidgarcia  staff    32K 11 oct 18:03 process_creation
drwxr-xr-x@    3 davidgarcia  staff    96B 11 oct 18:03 process_tampering
drwxr-xr-x@    3 davidgarcia  staff    96B 11 oct 18:03 raw_access_thread
drwxr-xr-x@    6 davidgarcia  staff   192B 11 oct 18:03 registry
drwxr-xr-x@    8 davidgarcia  staff   256B 11 oct 18:03 sysmon
drwxr-xr-x@    5 davidgarcia  staff   160B 11 oct 18:03 wmi_event
 ~/tmp/sigma
```

Vamos a afinar un poco, vemos lo que hay en “file”, las reglas dedicadas a eventos sobre el sistema de archivos:

```
❯ ll sigma/rules/windows/file
total 0
drwxr-xr-x@   7 davidgarcia  staff   224B 11 oct 18:03 file_access
drwxr-xr-x@   4 davidgarcia  staff   128B 11 oct 18:03 file_change
drwxr-xr-x@  14 davidgarcia  staff   448B 11 oct 18:03 file_delete
drwxr-xr-x@ 149 davidgarcia  staff   4,7K 11 oct 18:03 file_event
drwxr-xr-x@   4 davidgarcia  staff   128B 11 oct 18:03 file_rename
❯ ll sigma/rules/windows/file/file_delete
total 96
-rw-r--r--@ 1 davidgarcia  staff   811B 11 oct 18:03 file_delete_win_cve_2021_1675_print_nightmare.yml
-rw-r--r--@ 1 davidgarcia  staff   1,1K 11 oct 18:03 file_delete_win_delete_backup_file.yml
-rw-r--r--@ 1 davidgarcia  staff   633B 11 oct 18:03 file_delete_win_delete_event_log_files.yml
-rw-r--r--@ 1 davidgarcia  staff   795B 11 oct 18:03 file_delete_win_delete_exchange_powershell_logs.yml
-rw-r--r--@ 1 davidgarcia  staff   809B 11 oct 18:03 file_delete_win_delete_iis_access_logs.yml
-rw-r--r--@ 1 davidgarcia  staff   622B 11 oct 18:03 file_delete_win_delete_powershell_command_history.yml
-rwxr-xr-x@ 1 davidgarcia  staff   750B 11 oct 18:03 file_delete_win_delete_prefetch.yml
-rw-r--r--@ 1 davidgarcia  staff   777B 11 oct 18:03 file_delete_win_delete_teamviewer_logs.yml
-rw-r--r--@ 1 davidgarcia  staff   841B 11 oct 18:03 file_delete_win_delete_tomcat_logs.yml
-rw-r--r--@ 1 davidgarcia  staff   1,0K 11 oct 18:03 file_delete_win_sysinternals_sdelete_file_deletion.yml
-rw-r--r--@ 1 davidgarcia  staff   882B 11 oct 18:03 file_delete_win_unusual_deletion_by_dns_exe.yml
-rw-r--r--@ 1 davidgarcia  staff   1,4K 11 oct 18:03 file_delete_win_zone_identifier_ads_uncommon.yml
 ~/tmp/sigma
```

Nos fijamos en las que están relacionadas a la vulnerabilidad conocida como "PrintNightmare" (info).

Veamos que contiene la regla SIGMA, que estando en formato YAML es posible verla con un editor de texto:

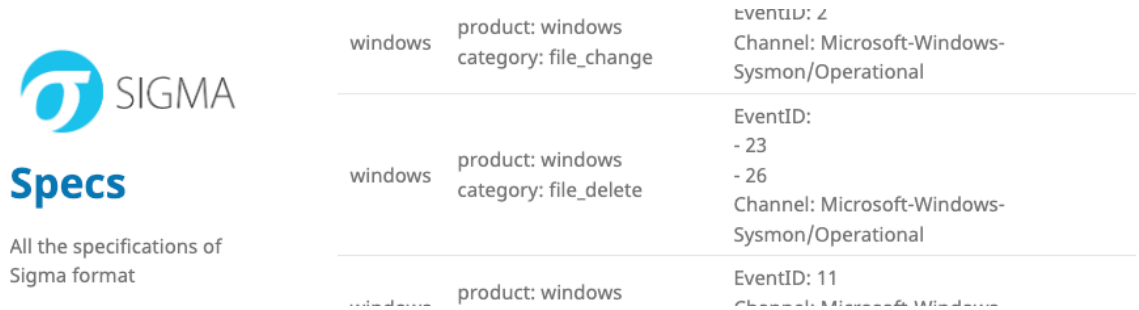
```yaml
title: Potential PrintNightmare Exploitation Attempt
id: 5b2bbc47-dead-4ef7-8908-8cf73fcbecbf
status: experimental
description: Detect DLL deletions from Spooler Service driver folder. This might be a potential exploitation attempt of CVE-2021-1675
references:
    - https://github.com/hhlxf/PrintNightmare
    - https://github.com/cube0x0/CVE-2021-1675
author: Bhabesh Raj
date: 2021/07/01
modified: 2023/02/17
tags:
    - attack.persistence
    - attack.defense_evasion
    - attack.privilege_escalation
    - attack.t1574
    - cve.2021.1675
logsource:
    category: file_delete
    product: windows
detection:
    selection:
        Image|endswith: '\spoolsv.exe'
        TargetFilename|contains: 'C:\Windows\System32\spool\drivers\x64\3\'
    condition: selection
falsepositives:
    - Unknown
level: high
```

Luego entraremos en detalle sobre cómo interpretar y escribir reglas SIGMA, ahora fijémonos en que posee metadatos acerca de la regla y una pequeña sección dedicada al evento en sí:

```
logsource:
    category: file_delete
    product: windows
detection:
    selection:
        Image|endswith: '\spoolsv.exe'
        TargetFilename|contains: 'C:\Windows\System32\spool\drivers\x64\3\'
    condition: selection
```

En primer lugar, se define de "donde" vamos a tomar eventos, es decir, cuál será la fuente que vamos a procesar. Como podemos ver es la sección "logsource". Bajo ésta, indicamos que el producto es "windows", eso hace que SIGMA seleccione los eventos del sistema operativo vía (Sysmon) y que estamos interesados en el evento particular "file_delete", bastante autodescriptivo: el borrado de archivos.

Ese evento está documentado también en la sintaxis de las reglas SIGMA, en su sitio web:



Ahora vamos con las condiciones que debe poseer dicho evento. Es decir, filtraremos todos los eventos de borrado de archivos con las condiciones que imponemos.

Las condiciones son sencillas:

```
detection:
    selection:
        Image|endswith: '\spoolsv.exe'
        TargetFilename|contains: 'C:\Windows\System32\spool\drivers\x64\3\'
    condition: selection
```

La etiqueta "selection" es una convención. Lo que nos interesa son las directrices que están bajo ella y que son equivalentes a los campos que encontraríamos en un evento detallado por Sysmon.

"Image" indicamos que se trata de la ruta a un archivo ejecutable y la partícula "|endswith" es que dicha ruta termine con "\spoolsv.exe".

La siguiente condición es que la ruta al archivo que sea borrado contenga la cadena descrita. Básicamente, dará como positivo todo aquello que vaya a la ruta indicada, posea cualquier otra partícula de ruta y termine en "\spoolsv.exe".

Con ello, ya tendríamos nuestra regla SIGMA. Pero recordemos que estas reglas son genéricas y abstractas, no podemos "detectar" nada con ellas, debemos convertirlas a un lenguaje particular.

6.3. Conversión de regla SIGMA a regla específica

6.3.1. Preparación del entorno

SIGMA se divide en varios subproyectos para que pueda ser utilizado de la forma más conveniente. Así, disponemos de una utilidad de línea de comandos, una librería Python que podemos utilizar para construir utilidades basadas en ella llamada PySigma (de hecho, la utilidad de línea de comandos está basada en la librería) y diferentes reglas en el repositorio oficial.

Lo primero será instalar sigma-cli, que es la utilidad de línea de comandos. Como está desarrollada en Python, simplemente hacemos:

pip install sigma-cli

¿Estamos listos? No. Dado que SIGMA es modular, debemos instalar 'backends' que nos proporcionarán la funcionalidad de conversión al sistema deseado. Por ejemplo, queremos instalar el backend para ElasticSearch o mejor, la pila ELK (Elastic, LogStash, Kibana). Hacemos:

sigma install elasticsearch

Podemos obtener una lista de backends (y otros complementos para sigma-cli) con:

sigma plugin list

Podréis ver que hay una buena cantidad de backends disponibles.
Ahora bien, ya lo tenemos todo, pero nos falta algo importante: las reglas.

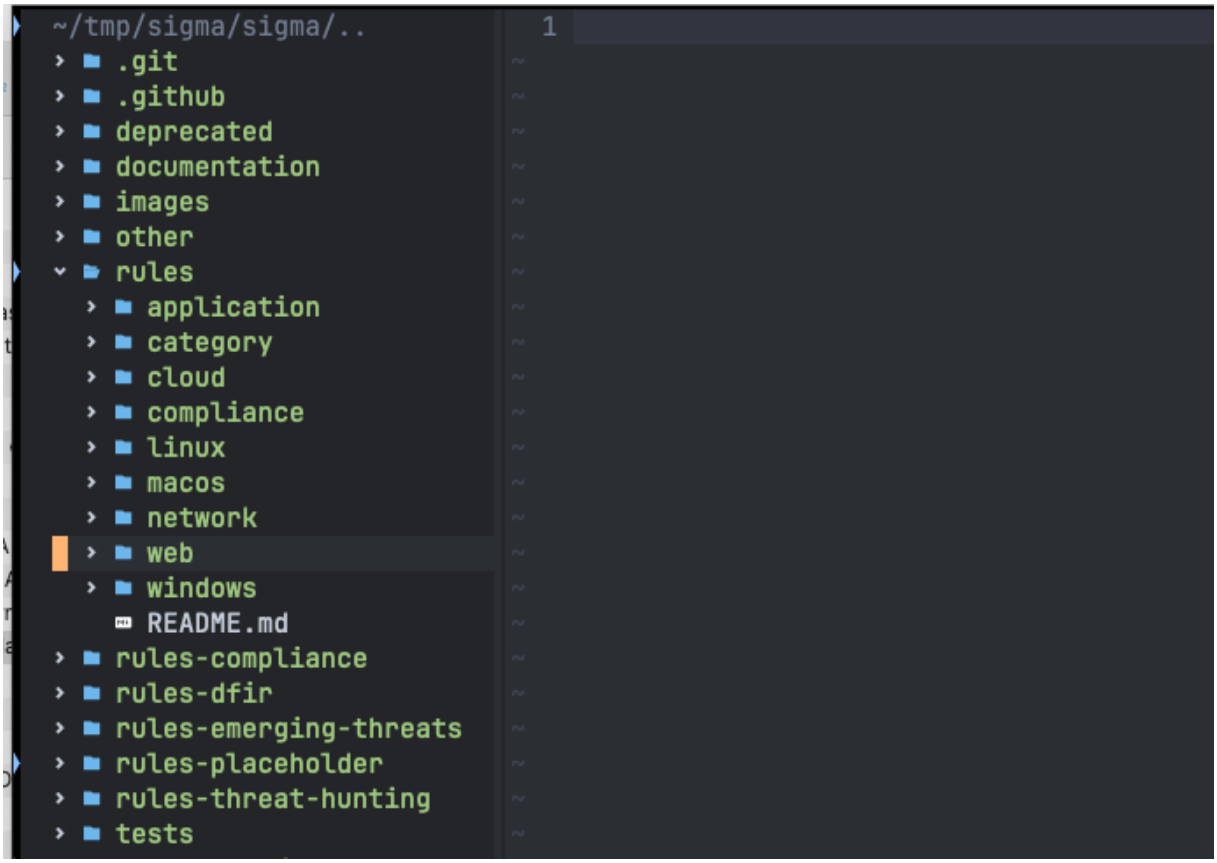
6.3.2. Obtención de las reglas SIGMA

Existe un repositorio oficial, pero en realidad la comunidad comparte constantemente reglas, por lo que podemos tener múltiples fuentes.

Para practicar con las reglas, las clonamos desde Github con git:

git clone https://github.com/SigmaHQ/sigma

Visualizando el árbol del directorio podemos ver las categorías de las reglas con una estructura muy autodescriptiva:

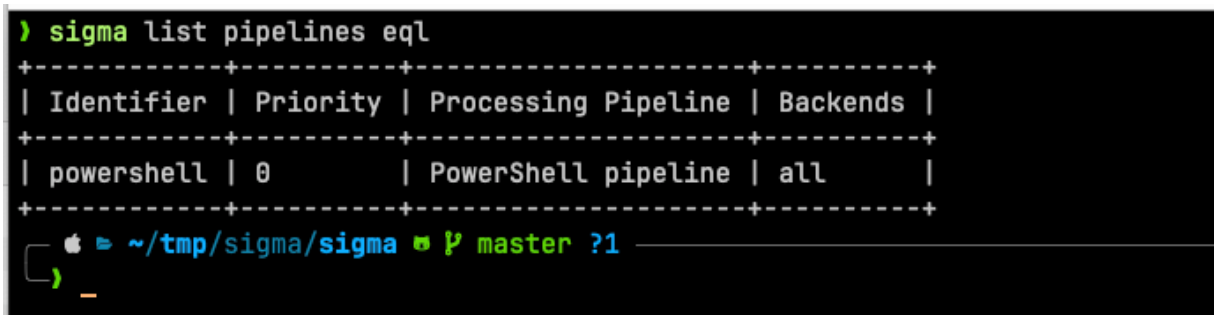


6.3.3. Ejemplo de conversión

Ahora supongamos que queremos convertir una regla al backend que hemos instalado para ElasticSearch.

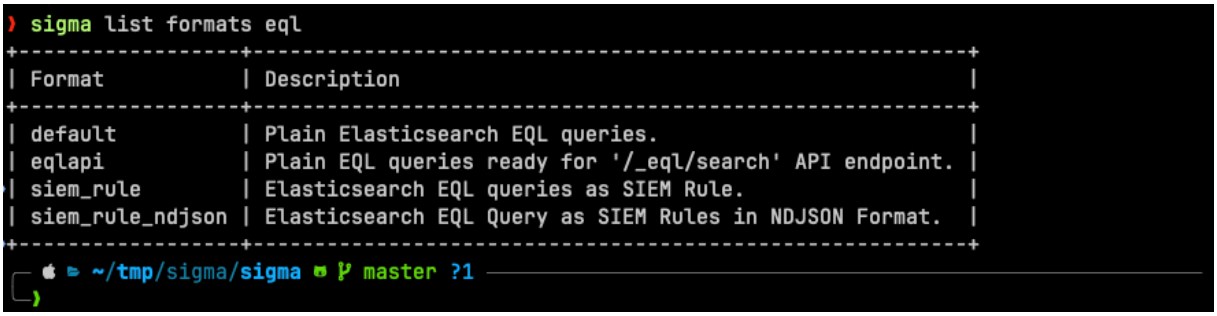
El proceso a seguir se compone de varios elementos. En primer lugar, usaremos el comando 'convert' de sigma-cli. Con la opción '-t eql' le decimos a sigma-cli que seleccione el backend de elasticsearch.

También necesitamos una 'pipeline', que será la traducción entre la regla genérica SIGMA y nuestro backend. Para listar los pipelines disponibles hacemos:



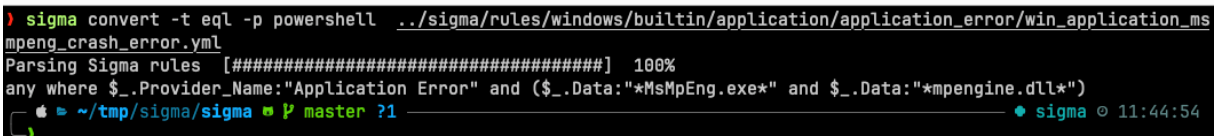
Disponemos de 'powershell', el lenguaje de programación usado por el intérprete de comandos Windows del mismo nombre. Para seleccionarla, disponemos de la opción de línea de comandos '-p'.

Finalmente, elegimos el formato de salida que queremos que tenga la regla. Para ver los disponibles:



Tenemos cuatro salidas o formatos disponibles para el backend 'eql' de ElasticSearch. Perfecto, seleccionaremos el que mejor se acondicione a nuestras necesidades (va a depender de que método queremos o forma de inyectarle finalmente la regla a ElasticSearch)

Vamos a ver el formato por defecto:



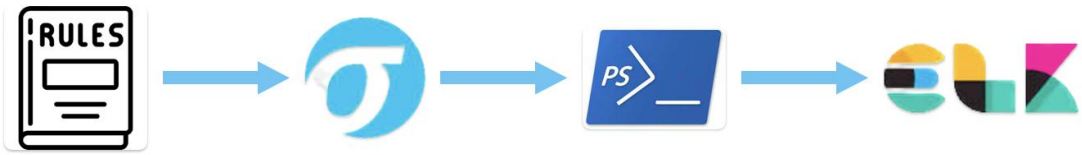
Es sencillo, una simple query o consulta en el lenguaje de consultas de ElasticSearch.

Supongamos que queremos una regla para almacenar en el sistema o stack ELK, vamos con 'siem_rule'. Aplicaremos la opción de línea de comando '-f':

```
❯ sigma convert -t eql -p powershell -f siem_rule  ../sigma/rules/windows/builtin/application/application_error/win_a
pplication_msmpeng_crash_error.yml | jq
Parsing Sigma rules  [####################################]  100%
{
  "name": "SIGMA - Microsoft Malware Protection Engine Crash",
  "tags": [
    "attack-defense_evasion",
    "attack-t1211",
    "attack-t1562.001"
  ],
  "consumer": "siem",
  "enabled": true,
  "throttle": null,
  "schedule": {
    "interval": "5m"
  },
  "params": {
    "author": [
      "Florian Roth (Nextron Systems)"
    ],
    "description": "This rule detects a suspicious crash of the Microsoft Malware Protection Engine",
    "ruleId": "545a5da6-f103-4919-a519-e9aec1026ee4",
    "falsePositives": [
      "MsMpEng might crash if the \"C:\\\" partition is full"
    ],
    "from": "now-5m",
```

Ahí la tenemos. Lista para integrar en nuestro stack ELK.

Vamos a hacer un resumen de las opciones:



Cogemos una regla, aplicamos 'sigma-cli', las opciones serán:

-t <backend>
-p <pipeline>
-f <formato_salida>

También podemos aplicar '-o <nombre_archivo>' para que guarde la salida en un archivo.

6.4. Creación de una regla SIGMA

Las reglas SIGMA son bastante sencillas de componer. Poseen gran cantidad de metadatos debido sobre todo a que, al ser genéricas, tienen que portar dicha información para que el usuario de estas pueda seleccionar con los datos disponibles las reglas adecuadas a su caso y herramienta (backend).

El formato utilizado es YAML. Básicamente, es un formato que estructura la información con claves-valor y que da jerarquía usando indentación, similar a como se estructura el

código Python.

Dispones de la especificación SIGMA en este enlace.

6.4.1. Metadatos

Como hemos dicho, explican concretamente qué y para qué sistemas está ideada la regla, además de muchos más datos importantes para tanto ayudar a encontrar lo que estás buscando como verificar si es adecuada para tus propósitos.

Por un lado, están los metadatos comunes que identifican una regla y su autor:

'title': Título, que debe ser descriptivo respecto de qué detecta la regla. (obligatorio)
'id': Es una cadena que debe tener formato UUIDv4. Aunque SIGMA señale que es un formato no obligatorio, es lo recomendable y el que usa la industria en la práctica (obligatorio)

'author': Creador o creadores de la regla.
'status': El estado de desarrollo en el que se encuentra la regla: 'stable', 'test'...
'date': fecha de creación de la regla.
'modified': fecha de la última modificación.
'description': Explicación más detallada (respecto del título) de la regla.
'license': No es muy usado, ya que la mayoría de las reglas se comparten.

'references': Campo opcional pero muy interesante que ayuda a obtener información del porqué de la regla. Por ejemplo, artículos de blogs, publicaciones de laboratorios, etc.

'logsource': Campo muy importante. Aquí se indica la tripleta: categoría, producto y servicio. Hay una referencia orientativa de los valores que puede portar (obligatorio). Es el campo usado por SIGMA para realizar la trasformación acorde al backend. Repetimos: ES MUY IMPORTANTE dominar este campo cuando creamos reglas.

Dos campos que suelen ponerse al final (opcionales, pero útiles), son:

'falsepositives': Lo que el autor/es de la regla piensan que podría causar falsos positivos debido a operaciones en el sistema que podrían pasar por maliciosas, pero no lo son.

'level': Es la clásica graduación de riesgo: informativo, bajo, medio, alto y crítico (las pondremos en inglés, claro)

6.4.2. Campos de detección y condiciones

Esto es más parecido a las reglas YARA, cuando vimos la parte de las "cadenas" y "condiciones". Es decir, primero señalamos que queremos detector y luego las

condiciones, apoyándonos en conectivas lógicas y otros operadores.

Detección

Para empezar, podemos emplear expresiones regulares o comodines tales como '*' o '?'. El último para cuando queremos decir "cualquier carácter" y el anterior es como la shell de Linux: cualquier cadena.

Así, por ejemplo, 'driver_*.sys' podrá contener o dar positivo en las cadenas: driver_init.sys, driver_sector.sys, etc.

En nuestra regla SIGMA, añadiremos las detecciones bajo el campo 'detections'. Es habitual ir separando grupos de detecciones bajo un mismo nombre, así, nos quedaría una estructura similar a esta:

```
detection:
    selection:
        Image|endswith: '\spoolsv.exe'
        TargetFilename|contains: 'C:\Windows\System32\spool\drivers\x64\3\'
    otros:
        CommanLine|contains: 'LocalCommand'
```

Si os fijáis, es muy habitual que si solo hay una sección se llame 'selection', es convención. Los nombres son arbitrarios. Pero luego nos apoyaremos en ellos cuando creemos las conectivas lógicas de las condiciones.

Existen numerosos campos a los que podemos hacer referencia. Usad la especificación de SIGMA para localizar los campos de nuestro interés.

Condiciones

Muy similares (ya lo hemos comentado) a YARA. Aquí hacemos referencias a los grupos de detecciones o al grupo entero si solo tenemos uno. De hecho, la regla más simple de detección sería:

```
detection:
    selection:
        Image|endswith: '\spoolsv.exe'
        TargetFilename|contains: 'C:\Windows\System32\spool\drivers\x64\3\'

    condition:
        selection
```

Si tuviésemos dos grupos, por ejemplo:

```
detection:
    selection:
        Image|endswith: '\spoolsv.exe'
    file:
        TargetFilename|contains: 'C:\Windows\System32\spool\drivers\x64\3\'


    condition:
        selection or file
```

Podemos ir aplicando las conectivas lógicas, tales como AND, OR, not, etc. Disponemos de un amplio apartado de reglas de condición aquí.

6.4.3. Chequear una regla

Disponemos de un útil comando en la herramienta 'sigma-cli' que nos permite chequear que una regla no posea errores. Usaremos:

sigma check <regla>

```
❯ sigma check test.yml
Parsing Sigma rules  [####################################]  100%
Checking Sigma rules  [####################################]  100%

=== Summary ===
Found 0 errors, 0 condition errors and 0 issues.
No rule errors found.
No condition errors found.
No validation issues found.
```

Si existen errores, la propia herramienta nos presentará un informe indicándolo.

6.5. Etiquetas de las reglas SIGMA

Hemos querido darle importancia a este apartado que vas a usar tanto para componer una regla SIGMA como para extraer inteligencia de aquellas que han hecho otros. Especialmente esto último.

Cuando realizamos labores de inteligencia de amenazas (threat intelligence) una pieza de malware o un grupo de indicadores de compromiso aislados no nos dicen nada; o al

menos, no mucho.

La idea es que cuando interrelacionamos todos los datos de los que disponemos, podemos obtener información. Una gran ayuda es la clasificación y caracterización de los datos. Por ejemplo, son muy populares las clasificaciones ATT&CK (hablaremos a fondo más adelante).

Por ello, muchos creadores de reglas se toman la molestia de anotar las mismas con esta útil información.

El apartado correspondiente es el de 'tags' (etiquetas). Veamos una regla con un par de etiquetas:



Lolbin Ssh.exe Use As Proxy

Detect usage of the "ssh.exe" binary as a proxy to launch other programs

Sigma Integrated Rule Set (GitHub) - frack113, Nasreddine Bencherchali

Copy rule | Download | Other files

```
title: Lolbin Ssh.exe Use As Proxy
id: 7d6d30b8-5b91-4b90-a891-46cccaf29598
status: experimental
description: Detect usage of the "ssh.exe" binary as a proxy to
launch other programs
references:
    - https://lolbas-project.github.io/lolbas/Binaries/Ssh/
    - https://github.com/LOLBAS-Project/LOLBAS/pull/211/files
    - https://gtfobins.github.io/gtfobins/ssh/
    - https://man.openbsd.org/ssh_config#ProxyCommand
    - https://man.openbsd.org/ssh_config#LocalCommand
author: frack113, Nasreddine Bencherchali
date: 2022/12/29
modified: 2023/01/25
tags:
    - attack.defense_evasion
    - attack.t1202
logsource:
    category: process_creation
    product: windows
detection:
    selection_parent:
        #ParentCommandLine: '"C:\Windows\System32\OpenSSH\sshd.
        exe" -R'
        ParentImage: 'C:\Windows\System32\OpenSSH\sshd.exe'
    selection_cli_img:
        Image|endswith: '\ssh.exe'
    selection_cli_flags:
        - CommandLine|contains: 'ProxyCommand='
        - CommandLine|contains|all:
            - 'PermitLocalCommand'
            - 'LocalCommand'
    condition: selection_parent or all of selection_cli_*
falsepositives:
    - Legitimate usage for administration purposes
level: medium
```

Aquí, el autor relaciona un positivo en esta regla con dos categorías ATT&CK. En concreto, la de evasión de defensas y la técnica con el id t1202. Estas dos etiquetas mapean directamente al comentado framework con este contenido:

https://attack.mitre.org/tactics/TA0005/

https://attack.mitre.org/techniques/T1202/

La primera es una **táctica** empleada por grupos APT, la segunda es una **técnica** en concreto. Una es el "porqué" y la otra el "cómo". Es decir, empleas una inyección de comandos para...evadir defensas.

Las etiquetas están clasificadas en la siguiente documentación de SIGMA. No solo existen etiquetas para ATT&CK, aunque estas son las más útiles también encontramos etiquetas para CVEs, CAR y otros frameworks.

7. REGLAS SNORT

7.1. Introducción

SNORT es un Sistema de detección de intrusos o por sus siglas en inglés, NIDS.

Un NIDS es un programa que *sniffa* (escucha) el tráfico de una red o redes y realiza un filtrado basado en, sorpresa, reglas.

Hay que señalar que aunque su función primordial y razón de existencia es esa, los NIDS también suelen ser usados para capturar tráfico de red (al estilo de WireShark o tcpdump) y depurar su funcionamiento o porque nos interesa registrar la actividad de red con una motivación no relacionada con la ciberseguridad: por ejemplo, guardar las conversaciones por red de una red de sensores.

Instalación

Snort es un programa que está disponible en casi cualquier sistema de distribución de paquetes de un sistema Linux. También se encuentra disponibles en binario compilados para Windows y Mac.

Si usas Linux, busca el paquete snort e instálalo como cualquier otro paquete. Prueba a que esté disponible como utilidad de línea de comandos para efectuar las prácticas y evaluaciones.

Reglas de pago

Muchos productos de este tipo (Suricata, Zeek, etc.), poseen reglas con licencia open source y son libres para compartir. No obstante, muchos proyectos poseen un feed de pago que distribuye de forma privativa reglas 'bleeding-edge' procedentes de su propia sandbox o de terceros.

Esto permite dotar al sistema de un conjunto de reglas profesionales mientras que cualquier estudiante, empresa pequeña o aficionado tiene acceso libre al sistema IDS toda vez que el proyecto consigue financiación y posibilidad el desarrollo y continuidad.

Especificación de reglas

Al igual que los otros sistemas que hemos ido viendo, Snort tiene su propia especificación de reglas.

Las reglas Snort se dividen en dos secciones principales: la cabecera y el cuerpo de la regla:

```
alert tcp $EXTERNAL_NET 80 -> $HOME_NET any
(
    msg:"Attack attempt!";
    flow:to_client,established;
    file_data;
    content:"1337 hackz 1337",fast_pattern,nocase;
    service:http;
    sid:1;
)
```

IMAGEN 9 CABECERA Y CUERPO DE UNA REGLA SNORT BÁSICA

7.2. El uso de Snort en un entorno de análisis de malware

Si has llegado hasta aquí sin haberte saltado los capítulos anteriores creo puedes estar imaginándote cómo se usa. De hecho, suele decirse que YARA es a los binarios lo que SIGMA a los eventos y Snort al tráfico de red.

Sí, nos sirve para inspeccionar el tráfico capturado de la detonación de un malware y, junto con las respectivas reglas Snort, detectar actividad de red maliciosa. Principalmente veremos tráfico de comunicación hacia un centro de control (C2), descarga de archivos de configuración del malware, subida de material robado (leaks) o descarga de malware adicional entre otros muchos tipos de tráfico malicioso.

Y precisamente porque venimos estudiando este tipo de esquemas en capítulos anteriores, vamos a ver como tendría cabida Snort en nuestro sistema, conjuntamente con un sistema con reglas procedentes de SIGMA:

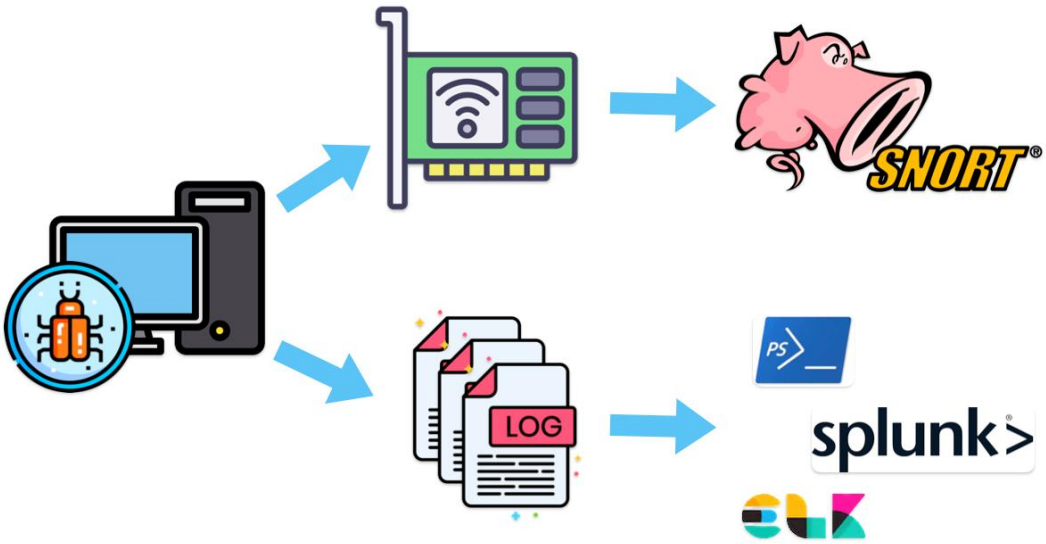


IMAGEN 10 ESQUEMA DEL FLUJO DE TRABAJO CON SNORT INTEGRADO

Como vemos, uno se encarga de los logs y otro del tráfico de red. Recordemos aquí que SIGMA no es un sistema que forme parte de la detonación, sino un formato de reglas genéricas que transformamos en reglas concretas de un backend. Dicho backend puede ser ELK, scripts de PowerShell o Splunk, entre muchos otros, es el componente activo.

Sin embargo, Snort sí posee un papel activo. Es un componente más de la cadena. Tomará el tráfico de un interfaz de red, lo examinará y al momento nos dirá si una o más reglas se han activado.

Además, si recuerdas cuando hablamos del aislamiento de red en las sandbox, Snort entraría aquí:

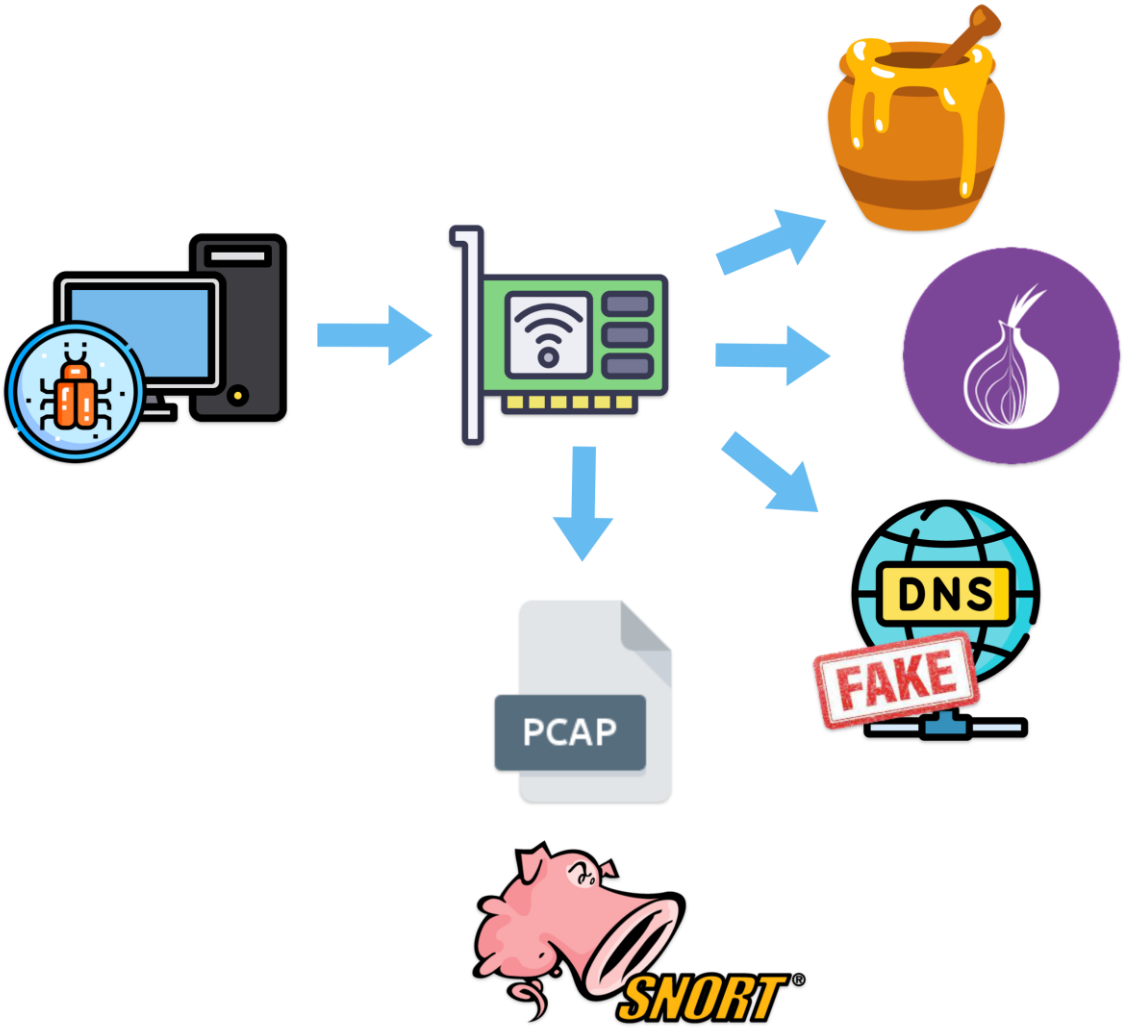


ILUSTRACIÓN 8 SNORT DENTRO DEL ESQUEMA DE RED DE LA SANDBOX

7.2.1. Uso assíncrono

Otra posibilidad de uso de Snort es el examen de un fichero Pcap. Un archivo Pcap es una grabación de todos los paquetes que han circulado, en ambos sentidos, a través de una red. Es decir, podemos capturar el tráfico con un software de captura de paquetes (wireshark, tshark, tcpdump o incluso con el propio Snort en modo sniffer) porque queremos ese archivo Pcap para almacenarlo o examinarlo en varias herramientas.

El método asíncrono es usado de manera preferente por sus ventajas:

- Es posible almacenar el archivo Pcap resultante y reutilizarlo para varias herramientas.
- No es necesario que Snort forme parte activa del sistema, por lo que podemos aislarlo en otra cola de trabajo en la que controlar y administrar sus recursos.

La desventaja es obvia: no dispondremos de alertas de tráfico de manera temprana.

El esquema de trabajo asíncrono no difiere mucho del anterior, simplemente es que podemos ir acumulando archivos pcaps y posteriormente pasarlos por un Snort:

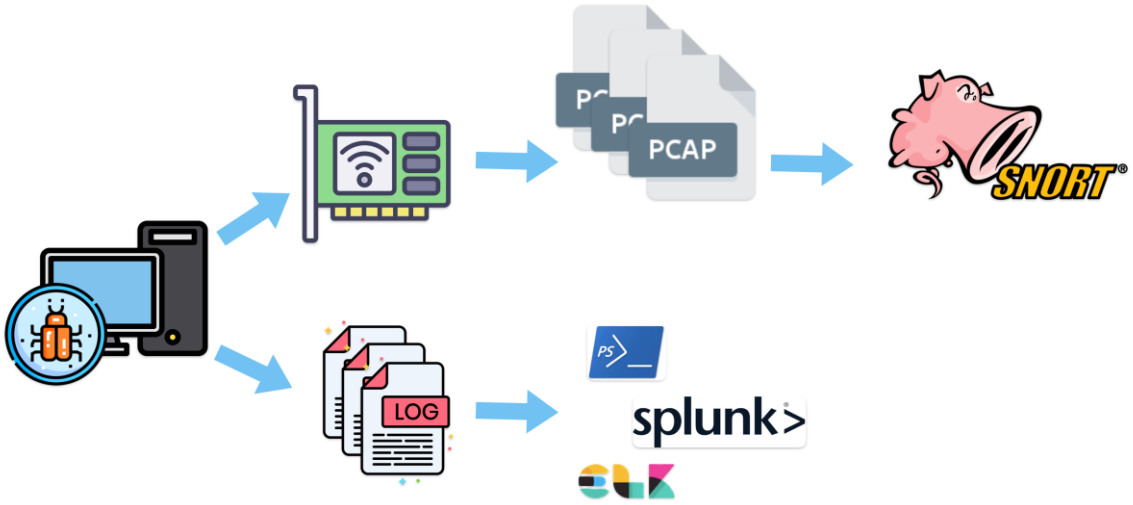


IMAGEN 11 MODO ASÍNCRONO DE SNORT

7.3. Cabecera de las reglas Snort

La cabecera posee cinco componentes o directivas. No todas son obligatorias. El esquema es bastante sencillo y además es muy intuitivo. Básicamente es como decir:

<que debe hacer Snort> <protocolo a vigilar> <IP><puerto><sentido del tráfico> <IP><puerto>

Vamos a ir viendo cada componente por separado.

7.3.1. Acción

Va siempre la primera. Es obligatoria. Existen hasta ocho tipos de acciones, pero en ciberseguridad en un altísimo porcentaje solo se usa "alert":

Como su propio nombre indica, genera una alerta. Es la alerta que vamos a usar cuando una regla de positivo.

El resto de las reglas son más útiles en otros campos y como mucho, en ciber, cuando se está manipulando el tráfico para engañar a un malware rescribiendo paquetes, dropeándolos (descartándolos) o bloqueándolos.

7.3.2. Protocolo

Ojo aquí que no solo se trata de los clásicos de capa 4: tcp y udp; aunque desde luego, son, de lejos, los más usados. Es posible indicar algunos protocolos de capas superiores (de hecho son muy interesantes las reglas basadas en ellos), como "http", "ssl", "smtp", "ftp", etc.

No tiene nada más. Especificar el protocolo que queremos emplear y listo.

7.3.3. Direcciones IP

No te dejes llevar por el título. Cuando hablamos de direcciones IP sería más correcto hablar de "direccionamiento" IP, puesto que puedes hacer referencia a rangos de redes CIDR o incluso variables (que puedes definir en la configuración de arranque de snort)

Es decir, puedes poner tanto 192.168.1.1 para una sola IP o 192.168.1.0/24 para un rango completo. Lo que sea de tu interés en esa regla.

Lo habitual es ver las variables típicas $EXTERNAL_NET y $HOME_NET o $LOCAL_NET (los nombres son arbitrarios, pero son los más usados) o el "any" -> "any".

Lo de las variables son bastante útiles porque las definiremos acorde a la red de detonación de la sandbox (que, como no podría ser de otra forma, está aislada) en snort.conf y a partir de ahí cuando migremos un sistema solo tenemos que cambiar el valor de estas variables.

Es posible no obstante, que tengas una regla para ciertos bloques de red específicos porque quieres vigilar que exista tráfico hacia una red concreta por algún motivo, por ejemplo, vigilar servidores bullet-proof.

7.3.4. Puertos

Sería el puerto usado por el protocolo a vigilar. Aquí tenemos bastante flexibilidad y es muy usado.

Si no nos interesa un puerto concreto o mejor dicho, nos da igual el puerto, pondremos "any".

Si es un puerto concreto se indica tal cual, por su número.

Podemos indicar un rango de puertos así:

alert tcp $LOCAL_NET any -> any :1024

Esa regla crearía una alerta a toda comunicación saliente de la red de la sandbox hacia cualquier otra dirección y que utilice un puerto destino menor a 1024 inclusive.

También disponemos de listas de puertos e incluso listas de rangos o todo mezclado:

alert tcp $LOCAL_NET any -> any [80, 443:512]

También podemos negar la condición con un signo de exclamación delante del puerto:

alert tcp $LOCAL_NET any -> any [!486, 443:512]

Esto indicaría todos los puertos del 443 al 512 a excepción del 486.

También podemos especificar una variable definida por nosotros:

alert tcp $LOCAL_NET any -> any $MIRAI_MALWARE

Por ejemplo, podríamos definir la variable anterior a los puertos típicos que utiliza el malware Mirai cuando realiza escaneos en busca de sistemas para infectar.

7.3.5. Operador de dirección

No tiene mucha complicación. Con ello expresamos el sentido del tráfico. Solo hay dos:

"->" de origen a destino. Bastante intuitivo.

"<>" en ambas direcciones. Tanto origen-destino como viceversa. Útil cuando queremos vigilar tanto las conexiones entrantes como salientes.

7.3.6. Resumen

Como vemos, tenemos un amplio abanico de posibilidades pero en realidad, solo utilizaremos un subconjunto; por lo que facilita aún más la tarea.

Ahora veremos las opciones que tenemos en el cuerpo de la regla, que sí son más variadas y se abren las posibilidades, sobre todo debido a la gran cantidad de contenido que puede haber en un paquete de tráfico y las opciones de inspección de campos tan variadas.

7.4. Cuerpo de las reglas Snort

No tendría sentido enumerar una a una todas las opciones de las que disponemos. Para eso ya está la documentación oficial. Lo que haremos es, en primer lugar señalar la estructura (cómo funcionan) y definir las combinaciones habituales o más importantes en

nuestra tarea de análisis automatizado.

7.4.1. El mensaje o atributo "msg"

Sí o sí, la regla va a tener un mensaje definido por nosotros. Es un campo cuyo valor determina que mensaje va a aparecer en las alertas de Snort. Sin él, difícilmente sabríamos que nos quiere decir una alerta, por lo que éste debe tener tres cualidades: **conciso, descriptivo y unívoco**.

Conciso porque nadie va a leer un mensaje de alerta de 1.000 palabras, hay que ir al grano.

Descriptivo, porque de nada sirve un mensaje que no transmita lo que la regla detecta.

Unívoco se explica por sí mismo: no queremos que dos mensajes se parezcan y confundamos que regla ha sido disparada.

El campo mensaje es descripto con el atributo "msg". Ejemplo:

```
alert udp $EXTERNAL_NET 3344 -> $HOME_NET 3345
(
msg:"MALWARE-BACKDOOR Matrix 2.0 Client connect";
flow:to_server;
content:"activate";
metadata:ruleset community;
classtype:misc-activity;
sid:161;
rev:10;
)
```

Fijémonos en dicho mensaje: es corto, describe que está pasando y no hay otro igual; aunque sí que hay uno parecido, pero no es el mismo: "MALWARE-BACKDOOR Matrix 2.0 Server access".

El mensaje nos dice que ha detectado una conexión del cliente (el nodo en nuestra red) conectándose a un servidor que corresponde al malware Matrix, versión 2.0.

Como vemos, tiene todos los atributos que buscamos y que deberíamos respetar cuando estemos confeccionando una regla.

Recuerda, el mensaje debe ser: conciso, descriptivo y unívoco.

7.4.2. "flow" o dirección y estado del tráfico entre cliente-servidor

Este atributo le indica a la regla el estado del tráfico y la dirección de la conexión. Muy, muy ligada al operador de dirección en la cabecera de la regla Snort.

La dirección es bastante descriptiva:

Si tenemos "from_" la conexión es "desde" y si es "to_" es "hacia". Básicamente: origen y destino.

Las dos opciones son, por supuesto: "server", "client". Del mismo modo: servidor o cliente.

Piensa que se trata desde donde se inicia la **petición** y a dónde va la **respuesta**.

Respecto del estado las habituales son:

"stateless" y "established". Estas dos se traducen en "me da igual el estado de la conexión" y "conexión establecida".

7.4.3. Selección de servicios (o protocolos de aplicación)

Mediante el atributo **service**, Snort nos permite indicar que protocolos de aplicación estamos interesados. Podemos indicar uno solo o una lista de ellos. Ejemplo:

service: http

service: http, ftp, ssl

7.4.4. Contenido y detección

El atributo **content** es el que lleva la carga de la regla. Además es el atributo más importante es cuanto abrir mucho la detección aumenta los falsos positivos y cerrarla los negativos. Si has leído la sección de YARA, ejerce la misma función que las cadenas.

Puedes indicar una cadena ascii o caracteres hexadecimales. Además de la cadena de detección, puedes acompañarla con varios operadores para modificar la detección. Por ejemplo, el **nocase** para que no sea sensible a mayúsculas.

También puedes indicar un **offset** numérico y un límite de lectura ("no detectas más de...") con **depth**. Son atributos muy a tener en cuenta para afinar la detección de **content**.

Es posible (y bastante habitual) tener varias clausulas **content** apiladas.

También podemos usar expresiones regulares con **pcre** o (mejor) **regex**.

Para la detección existen numerosos campos adicionales que podemos usar para indicarle a Snort donde y como debe mirar, pero sería extenso y no tendría cabida aquí.

7.4.5. Algunos metadatos imprescindibles

Hay ciertos metadatos que nos hablan acerca de la regla. Son muy útiles puesto que sin ellos podríamos perdernos entre tanta regla (el archivo de reglas community está sobre las 4.000).

sid: Es un número que en teoría debe ser único, Snort se reserva el primer millón. A partir de ahí puedes poner el tuyo en tus reglas. Tiene consideración local, es decir: tus reglas, tus números.

gid: No suele emplearse. Indica la procedencia de la regla cuando esta se activa: reglas estándar, procedentes de librerías compiladas, reglas de fábrica, etc.

rev: Básicamente es un número incremental de versión. Si hacemos algún cambio a la regla pasará de n a n+1. Simple.

reference: Aunque parezca un campo libre, tiene cierto formato. Nos habla acerca del porqué de la regla. Por ejemplo, si la regla detecta cierto exploit para una vulnerabilidad, su campo metadata vendrá dado por:

reference: cve,2023-11338

Si es una entrada de blog o similar que se puede visitar en una URL:

reference: url,attack.mitre.org/techniques/T1014

Es decir, el formato genérico es:

reference: <tipo de fuente>,<fuente>

classtype: Atributo muy informativo y clasificatorio. Nos indica qué clase de ataques detecta la regla. Existe una tabla con los posibles valores aquí. Va acompañada de una prioridad que puede resultar demasiado rígida y que no tiene en cuenta contexto alguno. No obstante, tenemos el atributo **priority** para dotar a la regla de la prioridad que queramos del 1 al 2^32-1.

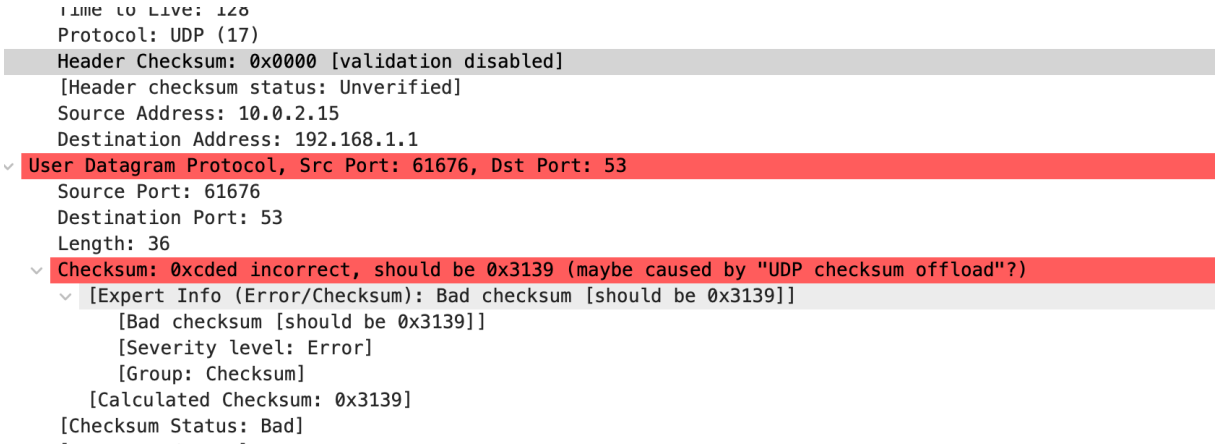
7.5. Algunos aspectos a tener en cuenta en el uso de Snort

7.5.1. Descarte de paquetes por checksum

Snort descarta paquetes cuya checksum sea errónea. Esto es debido a cómo funciona ese cálculo. En vez de estar hecho por el sistema operativo, el paquete sale de la CPU a la tarjeta de red sin ese checksum calculado (para librar a la CPU del cálculo), por eso en caso todas la capturas PCAP ese checksum saliente es nulo.

Cuando el paquete llega a la tarjeta de red, el hardware especializado de la tarjeta lo calcula e inserta en el paquete (justo donde estaba a nulo), pero Snort (bueno, libpcap) no lo ve.

Un ejemplo:



Debes tener esto en cuenta porque si no, no vas a poder inspeccionar paquetes salientes si el checksum se calcula en el hardware de la tarjeta de red.

En 'snort' línea de comandos (que emplearás para hacer reglas y probarlas) se debe utilizar la opción "-k none" para que admita checksum a nulo.

7.5.2. Silenciar mensajes de información

Snort, cuando se lanza por la línea de comando, da bastante información respecto del procesamiento de la configuración, estado, etc.

Para silenciar estos mensajes pon la opción "-q" (de quiet)

7.5.3. Impresión de alertas por pantalla

Snort tiene varias opciones de configuración respecto que hacer con una alerta. Cuando queramos que imprima por pantalla éstas, disponemos de multitud de formatos.

Las opciones de configuración se operan con la bandera "-A". Tenemos:

-A none: no hará nada.
-A cmg: alerta y da datos respecto a dicha alerta.
-A alert_*

Donde el asterisco tenemos varias opciones:

```
❯ snort --help-modules | rg alert
alert_csv (logger): output event in csv format
alert_fast (logger): output event with brief text format
alert_full (logger): output event with full packet dump
alert_json (logger): output event in json format
alert_syslog (logger): output event to syslog
alert_talos (logger): output event in Talos alert format
alert_unixsock (logger): output event over unix socket
alerts (basic): configure alerts
```

Ahí podemos optar a poner el formato de salida de nuestra preferencia. Pruébalos para ver el que mejor se adapta a tus necesidades o preferencias.

7.6. Otros sistemas de detección de tráfico basados en reglas

Snort, aunque siendo el más popular, no es el único IDS que podemos emplear para el registro y escrutinio de red. Existen más alternativas que son usadas en entornos automatizados de análisis. Vamos a exponer de forma breve dos de ellas: Suricata y Zeek (antiguamente, Bro Scanner).

7.6.1. Suricata

Suricata es, al igual que Snort, un proyecto open-source nacido de la OSIF. Es bastante usado y es una alternativa completa a Snort.

Es bastante popular el conjunto de reglas para Suricata publicado por Emerging Threats. Disponible aquí. Además del conjunto de uso libre, Emerging Threats publica, bajo pago, un feed profesional de reglas más actualizado.

La mayoría de las reglas para Suricata son compatibles con Snort, ya que utilizan el mismo lenguaje para definirlas.

7.6.2. Zeek

Zeek es otro IDS cuyo nombre cambió y antiguamente era popular bajo la denominación de Bro Scanner.

Aun no siendo tan popular como Snort o Suricata, el poder de Zeek reside en un potente lenguaje de programación para las reglas.

No es muy empleado en soluciones automatizadas de análisis de malware, en parte, porque no es compatible con el conjunto de reglas de Snort o Suricata.

8. EL MARCO DE TRABAJO ATT&CK

8.1. Introducción

Ya lo hemos mencionado en apartados anteriores. ATT&CK, procede de MITRE, una organización estadounidense sin ánimo de lucro creada en 1958 a partir de un proyecto del conocido MIT. Desde entonces se ha encargado de crear y definir estándares y marcos de trabajo para la industria. En las dos últimas décadas ha estado involucrada en proyectos relacionados con la ciberseguridad, algunos bastante conocidos y usados como el CVE, CVSS o el CWE.

Desde cierta perspectiva y aunando todos los frameworks que dirige, los distintos proyectos nos permiten clasificar y ordenar casi todo evento, vulnerabilidad, riesgo, amenaza, amenaza persistente avanzada (APT) y otras muchas definiciones con las que tenemos que lidiar a diario en el campo de la ciberseguridad.

Hace unos años, y debido sobre todo al auge de las operaciones APT de distintos actores, nació ATT&CK, un framework que se caracteriza por modelar las diferentes tácticas y técnicas empleadas en estas operaciones por grupos que casi siempre suelen tener un trasfondo político y un uso avanzado en los métodos que emplean.

Así, ATT&CK nos proporciona una visión de conjunto de toda técnica empleada en las APT, organizada por tácticas y a un nivel superior, distinguiendo tres dominios: Enterprise, Mobile y ICS. Lo que se traduce en: servidores y estaciones de trabajo, dispositivos móviles y sistemas de control industrial. Aunque el más usado y de referencia es el Enterprise, no podemos escatimar el uso de los otros dos dominios.



IMAGEN 12 PANORÁMICA DE MATRIX ENTERPRISE

8.2. Tácticas y técnicas

Una vez abordado un dominio, tenemos las diferentes tácticas. Verticales que nos proporcionan gradualmente todas las fases de un ataque orquestado y complejo; como son las APT.

Es decir, las diferentes tácticas descritas van desde el reconocimiento que se hace de los activos tecnológicos de una empresa hasta la exfiltración de datos y el impacto.

Enterprise tactics

Tactics represent the "why" of an ATT&CK technique or sub-technique. It is the adversary's tactical goal: the reason for performing an action. For example, an adversary may want to achieve credential access.

Enterprise Tactics: 14

ID	Name	Description
TA0043	Reconnaissance	The adversary is trying to gather information they can use to plan future operations.
TA0042	Resource Development	The adversary is trying to establish resources they can use to support operations.
TA0001	Initial Access	The adversary is trying to get into your network.
TA0002	Execution	The adversary is trying to run malicious code.
TA0003	Persistence	The adversary is trying to maintain their foothold.
TA0004	Privilege Escalation	The adversary is trying to gain higher-level permissions.
TA0005	Defense Evasion	The adversary is trying to avoid being detected.
TA0006	Credential Access	The adversary is trying to steal account names and passwords.
TA0007	Discovery	The adversary is trying to figure out your environment.
TA0008	Lateral Movement	The adversary is trying to move through your environment.
TA0009	Collection	The adversary is trying to gather data of interest to their goal.
TA0011	Command and Control	The adversary is trying to communicate with compromised systems to control them.
TA0010	Exfiltration	The adversary is trying to steal data.
TA0040	Impact	The adversary is trying to manipulate, interrupt, or destroy your systems and data.

IMAGEN 13 TÁCTICAS DESCRITAS EN EL DOMINIO ENTERPRISE

Es importante que nos quedemos con el esquema de la numeración, evidentemente no memorizarlo sino acostumbrarte a ver TA00…(tácticas), T…(técnicas), S…(software), M…(mitigación), DS…(detección), etc. y saber o poder intuir a qué se está refiriendo.

Bien, pero ¿Qué es una táctica? Vale, pues es un conjunto de técnicas que se les ha visto a los grupos que realizan APT con una finalidad determinada. Es esa finalidad la que da nombre al conjunto.

Por ejemplo, vamos a tomar la táctica de "Reconnaissance" que es, obviamente: reconocimiento. Es decir, la parte inicial de una campaña donde los grupos cibercriminales realizan una labor de campo orientada a obtener toda la información posible de su objetivo con el fin de poder planificar como, donde, cuando y qué técnicas emplear.

Una de las técnicas vistas a esos grupos es la de "Active Scanning": el escaneo activo de recursos del objetivo o más terrenal con un ejemplo: pasarle un escáner de puertos directamente a un servidor de la empresa a atacar.

ID	Name	Description
T1595	Active Scanning	Adversaries may execute active reconnaissance scans to gather information that can be used during targeting. Active scans are those where the adversary probes victim infrastructure via network traffic, as opposed to other forms of reconnaissance that do not involve direct interaction.
.001	Scanning IP Blocks	Adversaries may scan victim IP blocks to gather information that can be used during targeting. Public IP addresses may be allocated to organizations by block, or a range of sequential addresses.
.002	Vulnerability Scanning	Adversaries may scan victims for vulnerabilities that can be used during targeting. Vulnerability scans typically check if the configuration of a target host/application (ex: software and version) potentially aligns with the target of a specific exploit the adversary may seek to use.
.003	Wordlist Scanning	Adversaries may iteratively probe infrastructure using brute-forcing and crawling techniques. While this technique employs similar methods to Brute Force, its goal is the identification of content and infrastructure rather than the discovery of valid credentials. Wordlists used in these scans may contain generic, commonly used names and file extensions or terms specific to a particular software. Adversaries may also create custom, target-specific wordlists using data gathered from other Reconnaissance techniques (ex: Gather Victim Org Information, or Search Victim-Owned Websites).

De estos “escaneos activos” se han visto tres subtécnicas:

- Escaneo de bloques IP (lanzar un nmap sobre un bloque de direcciones IP)
- Escaneo de vulnerabilidades (lanzar un OpenVAS sobre dicho bloque de IP)
- Escaneo con wordlists (lanzar “cansina” sobre un sitio web)

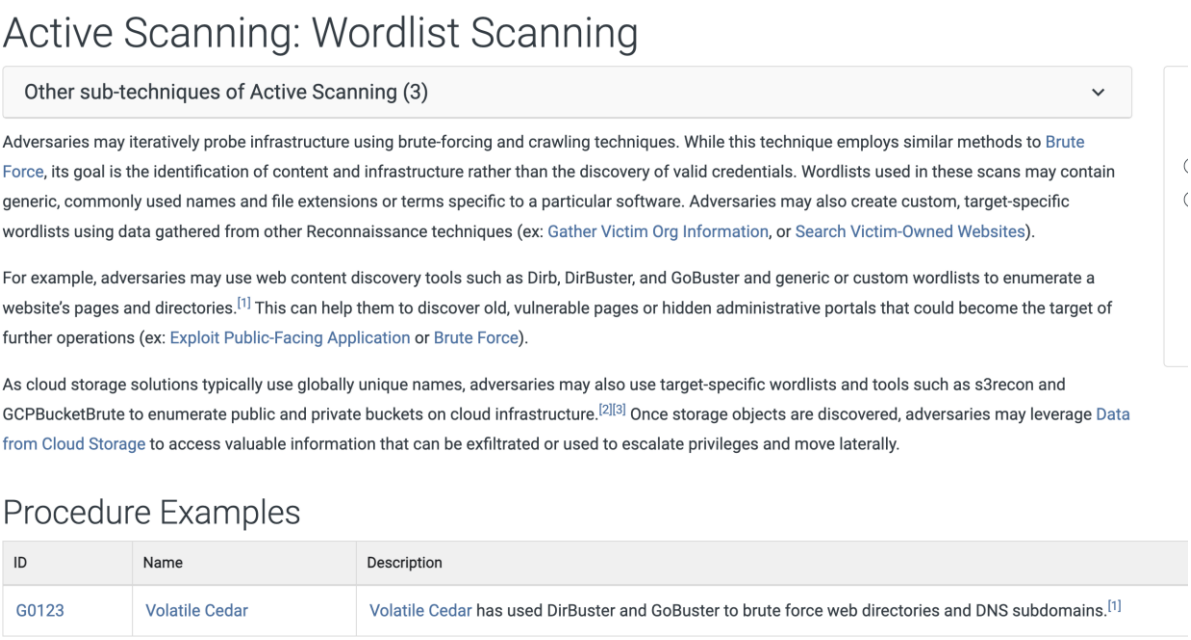
Como vemos, cada vez que bajamos más de nivel dentro de la jerarquía llegamos incluso a ejemplos concretos de “cómo” lo hacen y con “qué” herramientas.

8.3. Ejemplos, mitigaciones y detección

Una vez tenemos descritas las técnicas empleadas, dentro de las diferentes subtécnicas encontramos varios apartados de sumo interés. Estos son los **ejemplos**, los cuales nos ayudan a entender las técnicas con descripciones de usos reales, las **mitigaciones**, que son las contramedidas que un Blue Team emplearía para abordar la defensa y la **detección**, que nos permiten imponer controles que nos ayuden a detectar un posible uso de dicha técnica.

8.3.1. Ejemplos

Usando la técnica anterior de escaneo activo, tenemos un ejemplo con el cual podemos indagar sobre como usan en el terreno práctica la técnica (La "G" es de "group" o grupo APT que ha sido visto usándola):



Active Scanning: Wordlist Scanning

Other sub-techniques of Active Scanning (3)

Adversaries may iteratively probe infrastructure using brute-forcing and crawling techniques. While this technique employs similar methods to Brute Force, its goal is the identification of content and infrastructure rather than the discovery of valid credentials. Wordlists used in these scans may contain generic, commonly used names and file extensions or terms specific to a particular software. Adversaries may also create custom, target-specific wordlists using data gathered from other Reconnaissance techniques (ex: Gather Victim Org Information, or Search Victim-Owned Websites).

For example, adversaries may use web content discovery tools such as Dirb, DirBuster, and GoBuster and generic or custom wordlists to enumerate a website's pages and directories.[1] This can help them to discover old, vulnerable pages or hidden administrative portals that could become the target of further operations (ex: Exploit Public-Facing Application or Brute Force).

As cloud storage solutions typically use globally unique names, adversaries may also use target-specific wordlists and tools such as s3recon and GCPBucketBrute to enumerate public and private buckets on cloud infrastructure.[2][3] Once storage objects are discovered, adversaries may leverage Data from Cloud Storage to access valuable information that can be exfiltrated or used to escalate privileges and move laterally.

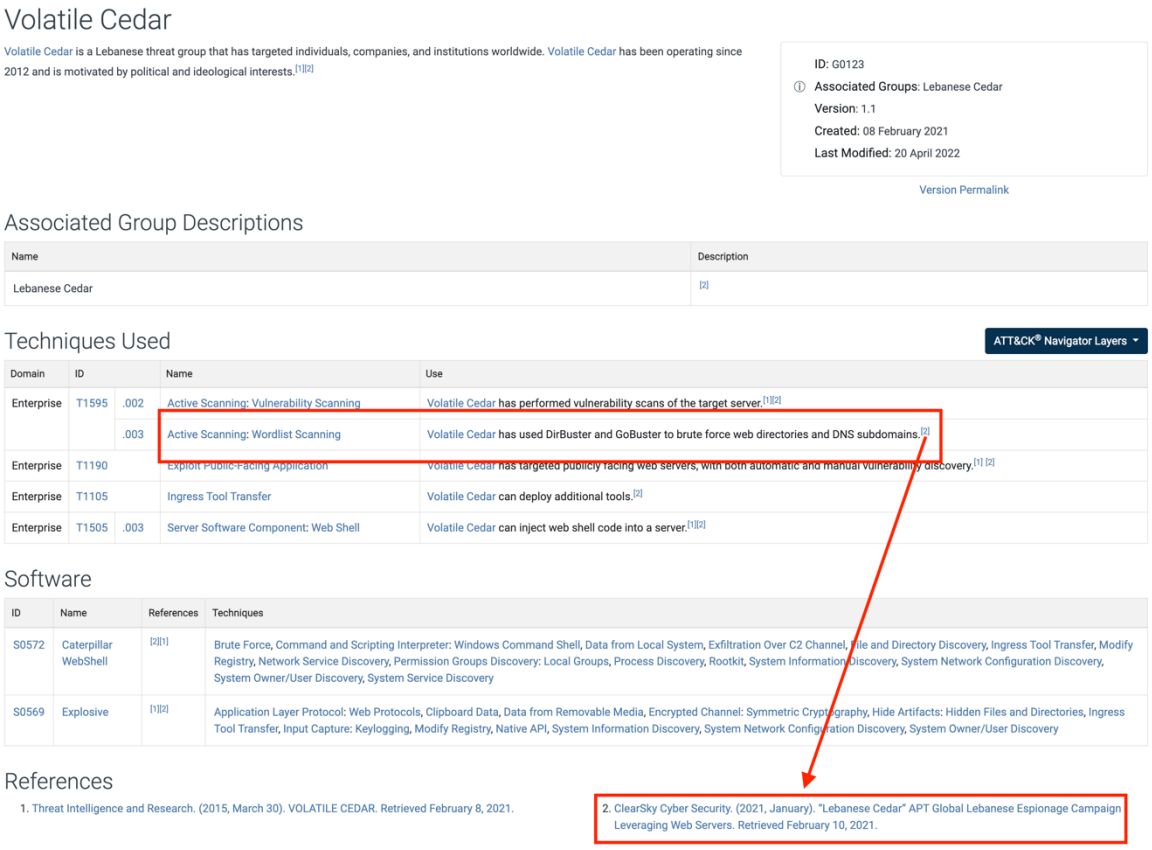
Procedure Examples

ID	Name	Description
G0123	Volatile Cedar	Volatile Cedar has used DirBuster and GoBuster to brute force web directories and DNS subdomains.[1]

IMAGEN 14 EJEMPLO DE TÉCNICA PUESTA EN PRÁCTICA POR EL GRUPO "VOLATILE CEDAR"

Si observamos la descripción nos comenta que dicho grupo ha sido visto usando las herramientas "DirBuster" y "GoBuster" para realizar una enumeración de subdominios y de contenido web.

Donde tenemos que ir una vez dentro de la página dedicada al grupo es a las referencias:



Volatile Cedar

Volatile Cedar is a Lebanese threat group that has targeted individuals, companies, and institutions worldwide. Volatile Cedar has been operating since 2012 and is motivated by political and ideological interests.[1][2]

ID: G0123
Associated Groups: Lebanese Cedar
Version: 1.1
Created: 08 February 2021
Last Modified: 20 April 2022

Version Permalink

Associated Group Descriptions

Name	Description
Lebanese Cedar	[2]

Techniques Used

ATT&CK® Navigator Layers

Domain	ID		Name	Use
Enterprise	T1595	.002	Active Scanning: Vulnerability Scanning	Volatile Cedar has performed vulnerability scans of the target server.[1][2]
		.003	Active Scanning: Wordlist Scanning	Volatile Cedar has used DirBuster and GoBuster to brute force web directories and DNS subdomains.[2]
Enterprise	T1190		Exploit Public-Facing Application	Volatile Cedar has targeted publicly facing web servers, with both automatic and manual vulnerability discovery.[1] [2]
Enterprise	T1105		Ingress Tool Transfer	Volatile Cedar can deploy additional tools.[2]
Enterprise	T1505	.003	Server Software Component: Web Shell	Volatile Cedar can inject web shell code into a server.[1][2]

Software

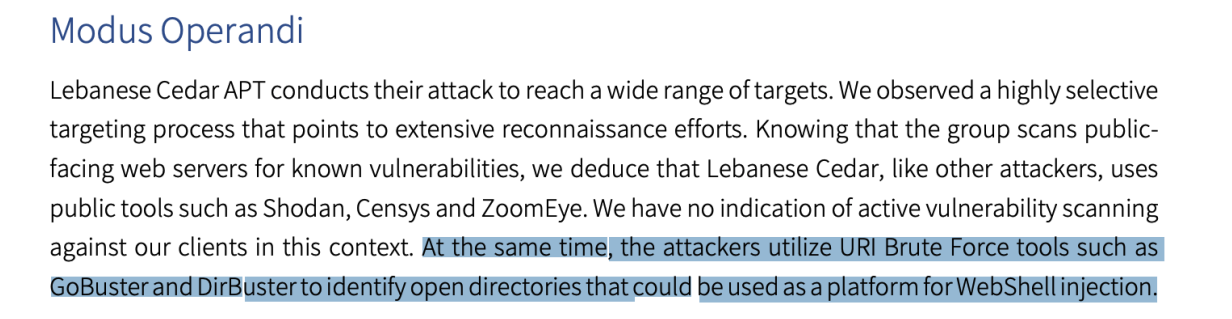
ID	Name	References	Techniques
S0572	Caterpillar WebShell	[2][1]	Brute Force, Command and Scripting Interpreter: Windows Command Shell, Data from Local System, Exfiltration Over C2 Channel, File and Directory Discovery, Ingress Tool Transfer, Modify Registry, Network Service Discovery, Permission Groups Discovery: Local Groups, Process Discovery, Rootkit, System Information Discovery, System Network Configuration Discovery, System Owner/User Discovery, System Service Discovery
S0569	Explosive	[1][2]	Application Layer Protocol: Web Protocols, Clipboard Data, Data from Removable Media, Encrypted Channel: Symmetric Cryptography, Hide Artifacts: Hidden Files and Directories, Ingress Tool Transfer, Input Capture: Keylogging, Modify Registry, Native API, System Information Discovery, System Network Configuration Discovery, System Owner/User Discovery

References

1. Threat Intelligence and Research. (2015, March 30). VOLATILE CEDAR. Retrieved February 8, 2021.
2. ClearSky Cyber Security. (2021, January). "Lebanese Cedar" APT Global Lebanese Espionage Campaign Leveraging Web Servers. Retrieved February 10, 2021.

IMAGEN 15 REFERENCIAS WEB DONDE PODEMOS INVESTIGAR EL USO PRÁCTICO DE LA TÉCNICA

Dicho enlace nos lleva a un completo informe en PDF donde se estudia la APT. Un extracto donde se comenta el uso indicado anteriormente:



Modus Operandi

Lebanese Cedar APT conducts their attack to reach a wide range of targets. We observed a highly selective targeting process that points to extensive reconnaissance efforts. Knowing that the group scans public-facing web servers for known vulnerabilities, we deduce that Lebanese Cedar, like other attackers, uses public tools such as Shodan, Censys and ZoomEye. We have no indication of active vulnerability scanning against our clients in this context. At the same time, the attackers utilize URI Brute Force tools such as GoBuster and DirBuster to identify open directories that could be used as a platform for WebShell injection.

IMAGEN 16 EXTRACTO DEL INFORME APT DONDE SE RESEÑA LA TÉCNICA OBSERVADA POR LOS INVESTIGADORES

Atención analistas: Para estar al día de todo lo que utilizan o hacen los grupos APT y que estemos alerta para cuando nos toque identificar una operación, debemos leer este tipo de informes. Es tarea del profesional suscribirse, buscar, guardar, estudiar y sacar

conclusiones de este tipo de informes que enriquecen y comparten el conocimiento adquirido en el campo de batalla.

Hay numerosas fuentes de recopilación de informes APT, un ejemplo:

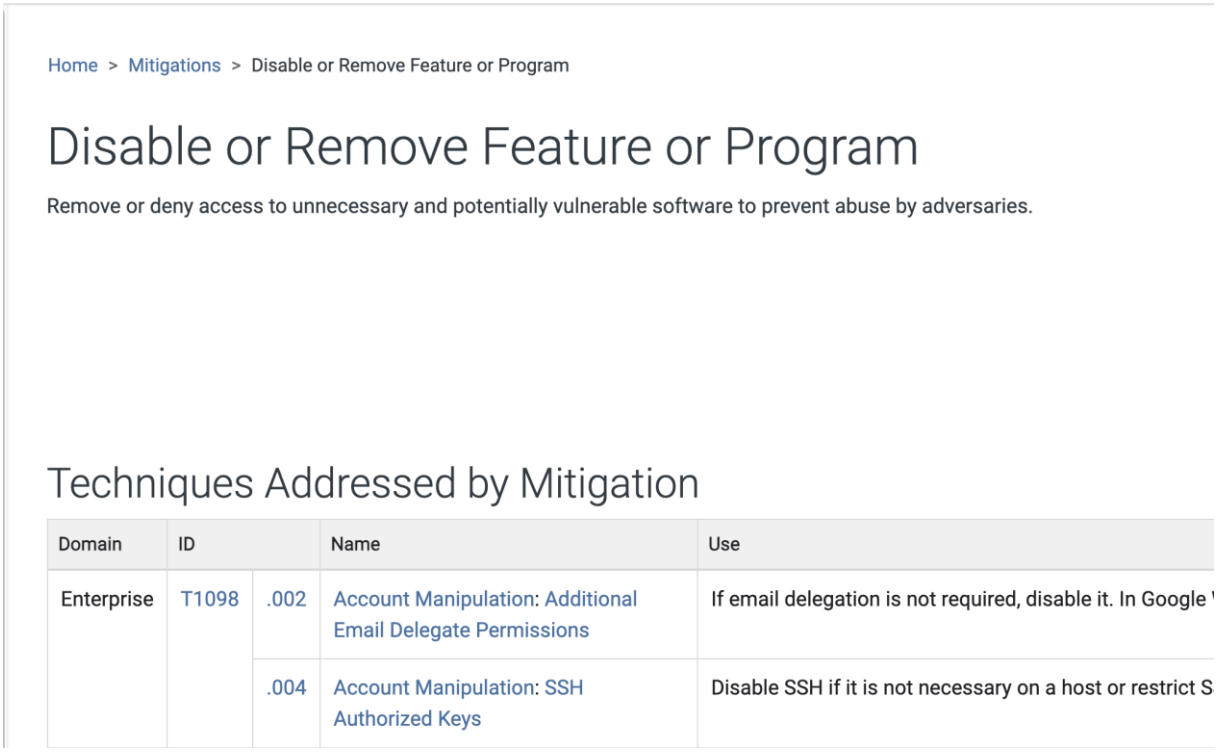
https://github.com/blackorbird/APT_REPORT

8.3.2. Mitigaciones

Sin más, son recomendaciones sobre cómo mitigar una técnica. Las mitigaciones llevan la letra "M" en el identificador.

Dos advertencias, porque aquí es donde más flaquea el framework: lo primero es que las mitigaciones suelen ser genéricas y a veces carecen de sustancia o granularidad. Por ejemplo: "está técnica afecta a este programa y la solución es desinstalar el programa"; es decir, son mitigaciones muy generalistas que no tienen en cuenta las particularidades de cada organización o sistema. Por último, las mitigaciones recogen varias técnicas, por el mismo motivo de que sean generalistas.

En realidad, ATT&CK, en el estado en el que se encuentra, no es precisamente un lugar donde mirar mitigaciones (puede que lo sea en un futuro, eso estaría por ver). Por ejemplo, no es comparable a la información que podríamos encontrar en la OWASP si estamos sopesando defender un ecosistema web (también es cierto que OWASP se especializa en esto mismo).



Home > Mitigations > Disable or Remove Feature or Program

Disable or Remove Feature or Program

Remove or deny access to unnecessary and potentially vulnerable software to prevent abuse by adversaries.

Techniques Addressed by Mitigation

Domain	ID		Name	Use
Enterprise	T1098	.002	Account Manipulation: Additional Email Delegate Permissions	If email delegation is not required, disable it. In Google
		.004	Account Manipulation: SSH Authorized Keys	Disable SSH if it is not necessary on a host or restrict S

IMAGEN 17 EJEMPLO DE LA BREVEDAD Y GENERALIDAD DE UNA MITIGACIÓN EN ATT&CK

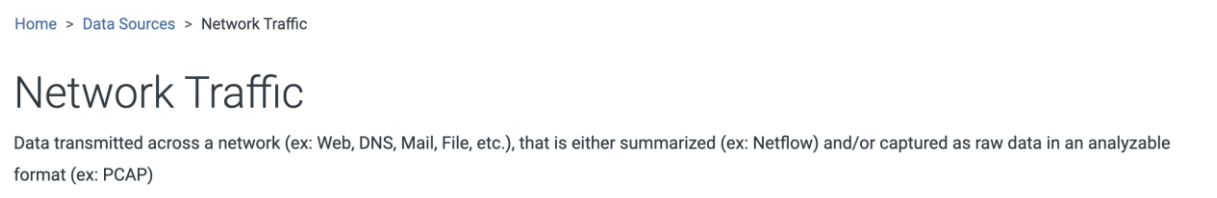
8.3.3. Detección

Si la mitigación ayuda a paliar el riesgo, la detección nos proporciona información acerca de cómo detectar una activación de una amenaza en los sistemas protegidos.

Las detecciones llevan la nomenclatura "DS".

Es fácil confundir una técnica o control de detección de una mitigación. Piensa esto: mitigar el peligro de fuego retirando material inflamable de una fuente de calor es una **mitigación**, puesto que impide que se inicie el fuego. Instalar un detector de humos es un control de **detección**, puesto que nos permite alertarnos de que existe un conato de incendio activado. Mejor ahora, ¿verdad?

El resto de lo comentado para las mitigaciones es válido aquí, por lo que no vamos a repetirnos.



Home > Data Sources > Network Traffic

Network Traffic

Data transmitted across a network (ex: Web, DNS, Mail, File, etc.), that is either summarized (ex: Netflow) and/or captured as raw data in an analyzable format (ex: PCAP)

IMAGEN 18 CONTROL DE DETECCIÓN GENÉRICO, VÁLIDO PARA VARIAS TÉCNICAS

En el ejemplo que ilustra la imagen superior lo que se aconseja para la técnica de enumeración por fuerza bruta web es que se monitoree el tráfico en busca de componentes maliciosos. Demasiado breve y concisa quizás.

8.4. ¿Para qué y cómo se usa ATT&CK?

ATT&CK es un convenio. Un marco de trabajo (indistintamente, en inglés: framework). Es un lenguaje común que proporciona la capacidad de entendimiento entre analistas y laboratorios que colaboran entre sí.

Si yo digo que una muestra de malware que hemos estudiado hace uso de T1189, T1059.001, T1068 tú te diriges al ATT&CK y encuentras en esas definiciones que la muestra se distribuye infectando a usuarios que visitan un sitio web, que hace uso de PowerShell y termina ejecutando un exploit para elevar privilegios.

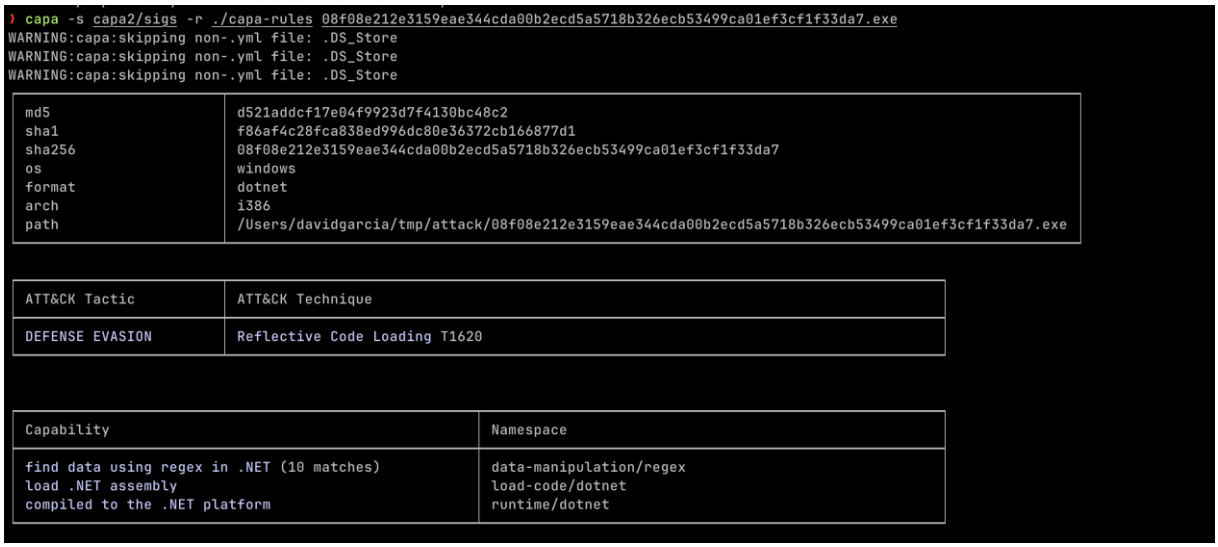
Mucha información en unas pocas referencias. Eso es ATT&CK y por eso es tan útil y usado.

¿Pero quién decide qué poner y dónde? Pues tú mismo como analista que estás viendo cómo se comporta la muestra, tanto en el análisis manual como en el entorno automatizado donde controlas que si aparecen ciertos eventos (individualmente o por conjunto) terminas etiquetando dicho comportamiento dentro de las categorías que proporciona ATT&CK.

Un ejemplo de uso que funciona muy bien dentro de la cadena de nuestro sistema de automatización es el proyecto CAPA de la compañía "Mandiant".

Dicho proyecto nos permite escanear un binario (PE, .Net, ELF...) de forma estática y a través de sus "capabilities" nos mapea directamente a ATT&CK.

Un ejemplo ilustrativo:



Una vez instalado capa, sus reglas y firmas, podemos escanear un binario (en particular ha sido este) y capa nos dará un mapeo a ATT&CK de aquello que vaya detectando.

Hay que indicar que se trata de una herramienta y como tal no es infalible, lo que quiere decir es que puede tener tanto falsos positivos como negativos (que detecte algo que no es y que pase por alto algo que sí lo es).

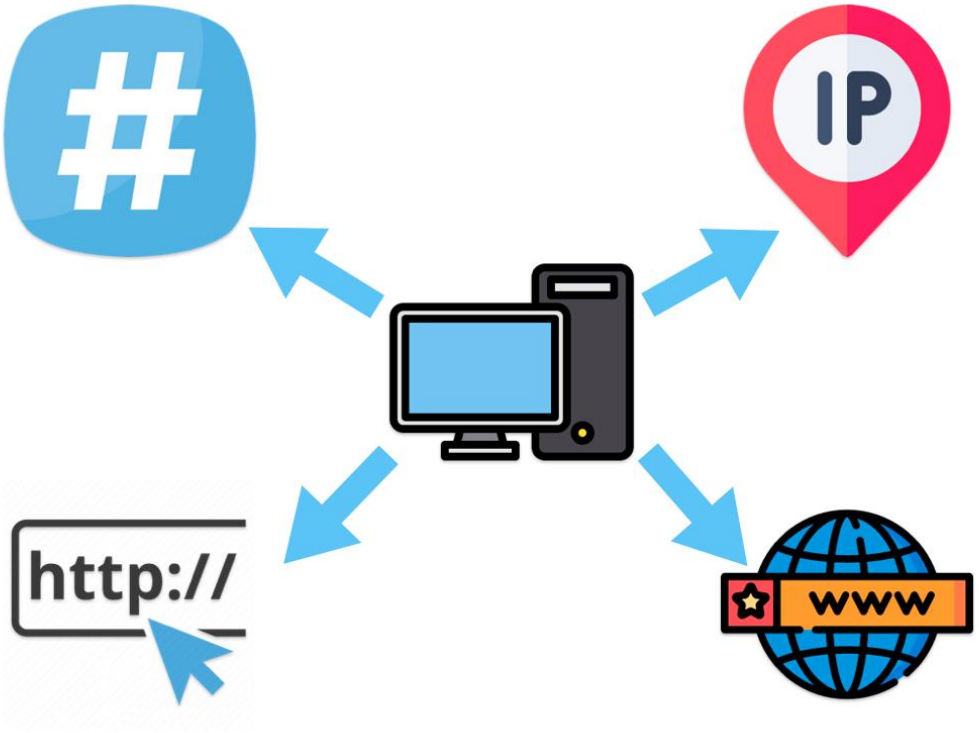
Ojo. Hay que tener en cuenta que, como hemos dicho, realiza un escaneo estático, por lo que los resultados ya de por sí van a ser exiguos si los comparamos con un análisis dinámico. No obstante, se trata de una herramienta para tener en cuenta para nuestra cadena de análisis porque nos aporta información muy útil para ir haciendo una preclasificación.

9. INDICADORES DE COMPROMISO (IOC)

9.1. ¿Qué son?

Es fácil entenderlo con un ejemplo. Supón que analizamos un malware que se comunica con el dominio "ejemplo7654.io". Días después, nos llega un evento de los sistemas de un cliente alertando de que uno de los ordenadores de cierto departamento ha intentado resolver dicho dominio para establecer una comunicación.

Es evidente, que dicho sistema tiene una enorme probabilidad de estar infectado. Es decir, el intento de conexión con ese dominio es un **indicador de compromiso**.



9.2. Tipos principales

No hay una lista cerrada, pero si un consenso. IoC es todo aquello que si es observado significa que puede existir un compromiso del sistema, dado que se le atribuye (a la aparición de ese IoC) indicios de maliciosidad.

Los principales son:

- Dominios
- Direcciones IP
- URLs
- Hashes de archivos
- Puertos o sockets

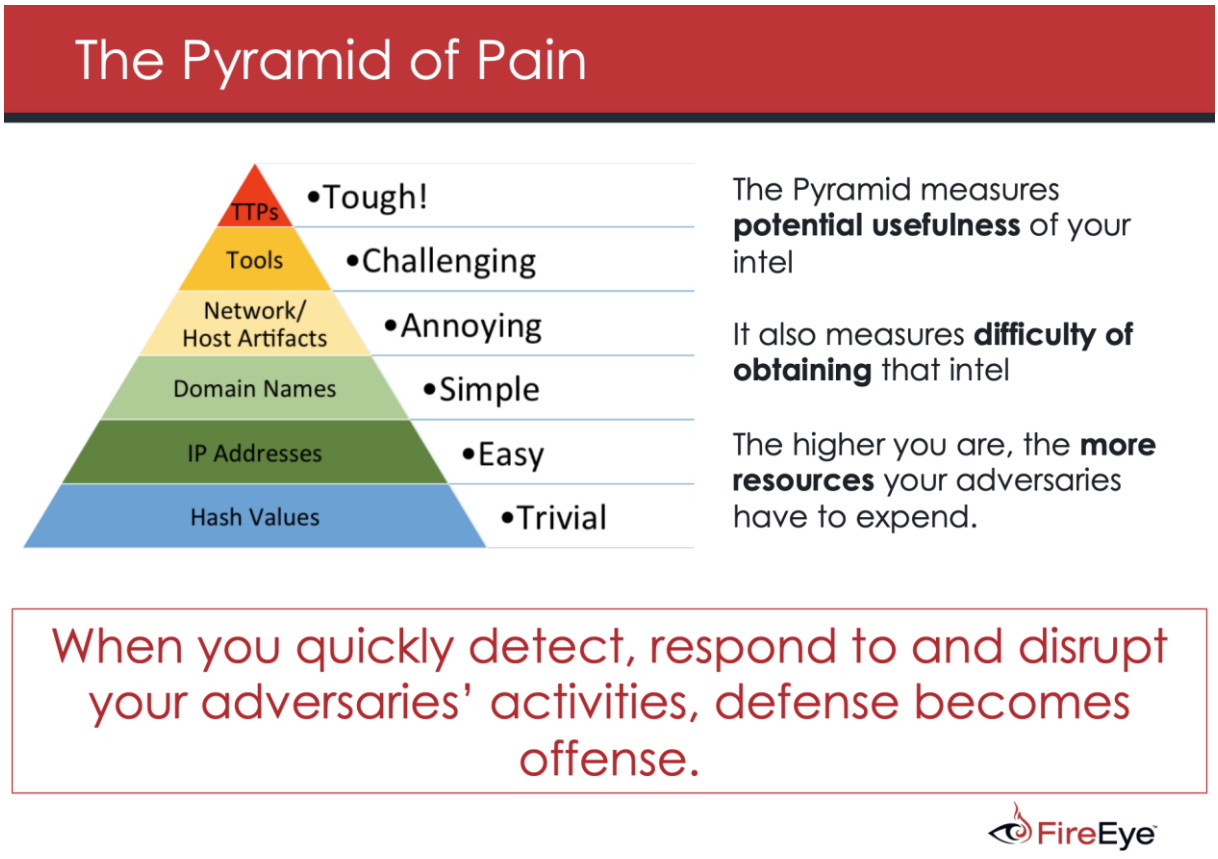
- Mutex
- Cadenas de texto o secuencias de bytes concretas

Recordemos, puede ser cualquier objeto cuya aparición podría indicar indicios de compromiso o infección del sistema que lo emite o produce o del cual ha sido registrado (eventos observables)

9.3. La Pirámide del Dolor

Aunque su denominación puede causar cierta confusión, se trata de una clasificación ad-hoc de la dificultad vs utilidad de los IOCs desde el punto de vista de la inteligencia de amenazas.

Observemos:



Fuente y recomendable lectura: https://rvasec.com/slides/2014/Bianco_Pyramid%20of%20Pain.pdf

La "Pirámide del Dolor" nos muestra lo difícil que es para un adversario (cibercriminales) cambiar ciertos IOCs. Por ejemplo, si tenemos un hash de una muestra, les basta con cambiar un byte para que dicho hash ya no sea útil. El esfuerzo es trivial. Cambiar una dirección IP que ha sido "fichada" es una cuestión de cambiar de máquina o resetear un router, por ejemplo.

Con el resto de IOCs pasa lo mismo, a medida que subimos la pirámide los IOCs son más complejos de cambiar y por lo tanto requerirán de un esfuerzo siempre mayor al adversario.

En el pico de la pirámide vemos las TTP y las herramientas. Respecto a esta última categoría lo cierto es que no vemos todos los días nuevas herramientas publicadas (ni mucho menos) sino que aprovechan una y otra vez las mismas y de vez en cuanto sale una nueva. Con las TTPs, el pico o cima, es más complejo aun: ¿Cada cuánto tiempo sale una técnica o táctica nueva? Puede que años...

9.4. ¿Para qué se usan?

Por supuesto, son inmensamente versátiles. Imaginemos el caso de los hashes. Una herramienta que calcule el hash de un archivo y lo compare con una lista de hashes que se saben, pertenecen a malware.

Otro caso más ajustado a nuestra descripción sería la de los dominios. Si a un sistema se le observa resolviendo un dominio concreto es posible alertar de un compromiso o infección. Sería el caso de análisis dinámico realizado por nuestra sandbox durante la detonación.

Pero es que si ese mismo dominio se encuentra en el cuerpo de un ejecutable, el análisis estático del mismo nos alertaría de que se ha encontrado dicho dominio en el ejecutable objeto de estudio. Ni tan siquiera haría falta que pasase a la fase de detonación y ya tendríamos un positivo por confirmar.

Podríamos generar una regla YARA, SIGMA, listas negras para filtrado de IPs o dominios, etc. con esos IoCs.

Pero el mayor uso de estos se da en entornos de colaboración. La mayoría de IoCs está interrelacionados formando una estructura de grafo dirigido. Por ejemplo, un dominio resuelve a una o varias direcciones IP, las direcciones IP se agrupan en redes, los dominios poseen un servidor de nombres autoritativo, etc.

Todas estas relaciones están modeladas y poseen un formato que facilita el intercambio. Lo veremos en su apartado correspondiente, son los formatos **STIX** y **TAXII**.

9.5. ¿Cómo extraemos los IoC?

Cuando sabemos que un ejecutable es malicioso simplemente vamos tomando objetos producto de su detonación y que nuestra sandbox habrá ido tomando: el hash del archivo, IP a las que se conecta, dominios que resuelve, etc. Toda esa lista de IoCs nos van a decir que si se observa en otros sistemas hay, como venimos diciendo, indicios de

compromiso o infección.

Es una tarea tanto automatizada como también puede ser generada a mano por un analista cuando tiene un caso particular entre manos y fruto del análisis va extrayendo IOCs para completarlos en el informe.

9.6. ¿Qué significa "defang"?

Una URL que conduzca a un recurso infectado o parte de la infraestructura no debe ser visitado fuera de un entorno controlado.

Si compartiésemos las URL maliciosa como tales: https://www.example.com, cualquier sistema que procese enlaces daría por válida esta y no distinguiría si se debe o no visitar.

Es un riesgo enorme. Podemos, desde infectar un sistema de manera involuntaria, hacer que alguien se infecte al compartir el enlace, desactivar una infraestructura que solo permita una visita por dirección IP o similar, etc. Además, muchos filtros en sistemas de mensaje o almacenamiento impedirán una URL maliciosa si está en listas negras.

Es decir, son muchas las razones por las que no podemos compartir ni hacer referencia (salvo en el contexto apropiado) una URL como tal.

Para ello, lo habitual es "camuflar" las URL cambiando parte del esquema para "desactivar" el procesamiento de éstas y que tomen forma de texto simple.

El convenio entre la comunidad es cambiar ese http por hxxp. Así, nuestra URL quedaría, por ejemplo, como:

hxxp://www.sitiopeligroso.com

Las **direcciones IP** también poseen un formato "defang":

192[.]168[.]1[.]0

Los **correos electrónicos** son particularmente curiosos, puesto que al tener varios componentes se deben tratar dos veces (usuario AT dominio [.] com):

testATtest[.]com

"**defang**" es un verbo inglés que entre sus acepciones significa: "hacer menos peligroso". Una interpretación directa en castellano y muy libre por parte del autor de este material sería: "**descolmillar**". Pero, por favor, usad "defang".

10. EL ESTÁNDAR STIX Y TAXII

10.1. Introducción

Los laboratorios de malware han compartido información desde los albores de la lucha contra el cibercrimen, incluso cuando no había grupos tan organizados y profesionalizados como hoy en día.

Existen múltiples acuerdos, consorcios, fundaciones, etc. De hecho, Virustotal, el conocido servicio nacido de la empresa española Hispasec (y posteriormente vendido a Google) es tierra común entre las marcas de soluciones de detección de malware, en la que se comparten las muestras (y todo archivo que se suba a la plataforma, sea malware o no) entre todos aquellos que poseen un acuerdo con Virustotal de integración de su tecnología.

Al principio, no existía un formato establecido de intercambio de esta información. La cual era compartida de manera desestructurada y en formatos de archivo y transmisión genéricos: csv, texto plano, etc. El más simbólico de todos ellos es el clásico .zip con la contraseña "infected" subido a un FTP. Gloriosos años 90; no obstante, el zip con esa contraseña sigue siendo usado como el primer día.

Estaba bastante claro que con el volumen in crescendo del malware y unos sistemas cada vez más modernos de ingesta y detonación, la industria exigía de facto un formato unificado para que el entendimiento entre las partes pudiera ser más fructífero y formal.

De la necesidad por protocolizar y formalizar estos intercambios nació el formato STIX y TAXII. Ambos tienen su origen en un grupo de trabajo del consorcio OASIS, motivado por el departamento de Interior de Estados Unidos. La primera versión de STIX y TAXII fue publicada en 2012.

10.2. Un lenguaje para modelar amenazas

STIX es eso, un lenguaje donde podemos expresar con bastante detalle una amenaza hasta crear, con las herramientas que nos proporciona, un modelo completo. De hecho, posee un rico sistema de tipos, patrones y objetos que, usados convenientemente, nos permiten capturar una foto con detalles.

No te quedes con la idea de que STIX solo describe "el malware que se está analizando". No, STIX va mucho más allá. Te permite expresar una campaña ATP e incluso atribución a un grupo cibercriminal determinado.

STIX posee un visualizador online en el que podemos plasmar un archivo JSON (procedente de la serialización del paquete STIX (que veremos cómo hacer más abajo)) y ver el grafo completo de objetos, observables y relaciones:

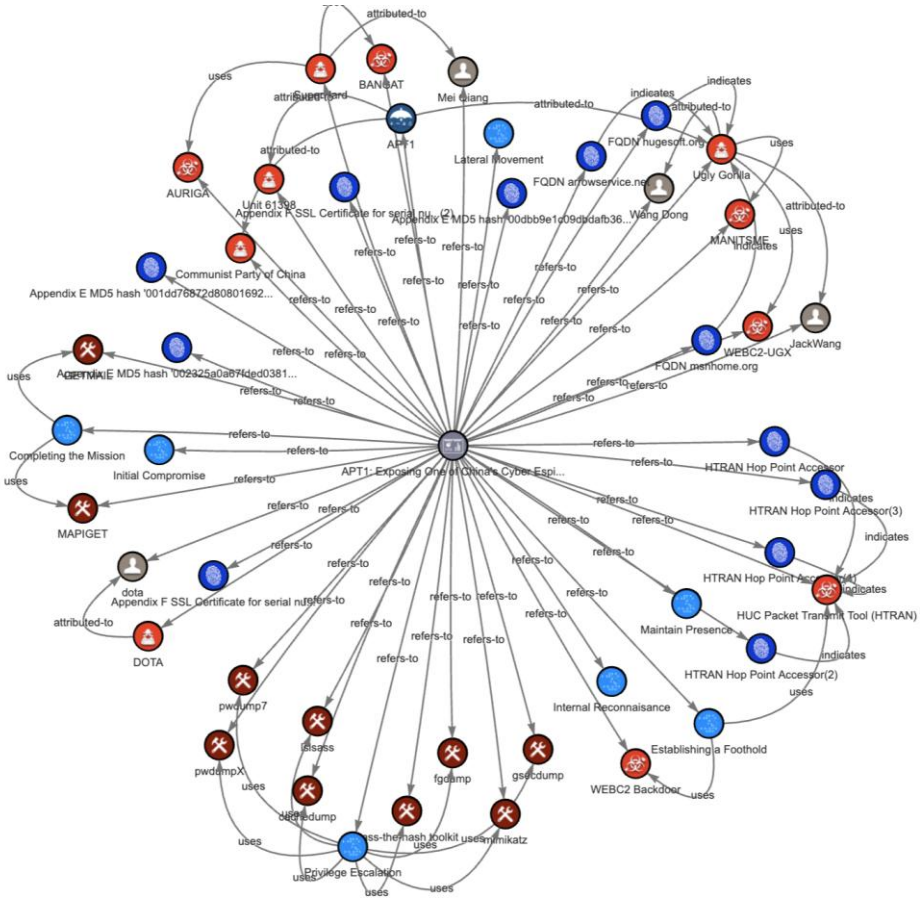


IMAGEN 19 REPRESENTACIÓN DEL GRAFO DE AMENAZA ATP-1 DE MANDIANT

10.2.1. Una advertencia respecto al formato y versiones de STIX

STIX, anteriormente usaba XML y existen muchos ejemplos y librerías asociadas a este formato y versiones anteriores de la especificación.

En este material el formato de salida o serialización es JSON y la versión STIX empleada es la 2.1.

10.3. ¿Qué aspecto tiene un paquete STIX?

El formato STIX, cuyas siglas significan "Structured Threat Information eXpression" define como debemos estructurar la información disponible de una amenaza concreta.

Como dijimos anteriormente, los indicadores de compromiso forman un grafo dirigido de relaciones entre ellos. STIX modela ese grafo en un formato de texto que describe cada nodo con atributos y relaciones. Todo ello forma lo que se conoce como paquete STIX preparado para ser intercambiado y procesado.

Es en lo que se convertirá una detonación de una muestra en la sandbox. Una vez se recogen y procesa el tráfico, eventos, registros y observaciones, irá a formar parte de un

paquete STIX. Ese paquete podrá almacenarse o enviarse a colaboradores o socios, con los que tendremos acuerdos de intercambio de muestras.

Los acuerdos entre firmas y laboratorios pueden ser variados. Por ejemplo, suministrar muestras con su paquete STIX correspondiente o las muestras en crudo. A veces, incluso puede haber acuerdos para intercambiar archivos de captura de red (pcaps), registro de eventos, etc.

Cuando hablamos de almacenar muestras, es posible almacenarla con el paquete STIX. De hecho, es práctica habitual que la muestra esté acompañada por el STIX correspondiente y podamos trazar relaciones entre estos; de ahí surge la inteligencia, además de la capacidad de agrupar y relacionar muestras en operaciones y estas a su vez, señalar (atribuciones) a grupos APT particulares.

Nota importante: En castellano, los analistas hablan de "paquete" STIX, pero en el contexto anglosajón el término es **bundle**. Recuerda, "STIX bundle" o un "paquete STIX"; depende del contexto en el que te encuentres.

Si lo abres en un editor de texto te encuentras con esto:

```
{
    "type": "bundle",
    "id": "bundle--56be2a3b-1534-4bef-8fe9-602926274089",
    "objects": [
        {
            "type": "indicator",
            "spec_version": "2.1",
            "id": "indicator--d81f86b9-975b-4c0b-875e-810c5ad45a4f",
            "created": "2014-06-29T13:49:37.079Z",
            "modified": "2014-06-29T13:49:37.079Z",
            "name": "Malicious site hosting downloader",
            "description": "This organized threat actor group operates to create profit from all types of crime.",
            "indicator_types": [
                "malicious-activity"
            ],
            "pattern": "[url:value = 'http://x4z9arb.cn/4712/']",
            "pattern_type": "stix",
            "valid_from": "2014-06-29T13:49:37.079Z"
        },
        {
            "type": "malware",
            "spec_version": "2.1",
            "id": "malware--162d917e-766f-4611-b5d6-652791454fca",
            "created": "2014-06-30T09:15:17.182Z",
            "modified": "2014-06-30T09:15:17.182Z",
            "name": "x4z9arb backdoor",
            "description": "This malware attempts to download remote files after establishing a foothold as a backdoor
            "malware_types": [
                "backdoor",
                "remote-access-trojan"
            ],
            "is_family": false,
            "kill_chain_phases": [
                {
                    "kill_chain_name": "mandiant-attack-lifecycle-model",
                    "phase_name": "establish-foothold"
                }
            ]
        },
        {
            "type": "relationship",
            "spec_version": "2.1",
            "id": "relationship--864af2ea-46f9-4d23-b3a2-1c2adf81c265",
            "created": "2020-02-29T18:03:58.029Z",
            "modified": "2020-02-29T18:03:58.029Z",
            "relationship_type": "indicates",
            "source_ref": "indicator--d81f86b9-975b-4c0b-875e-810c5ad45a4f",
            "target_ref": "malware--162d917e-766f-4611-b5d6-652791454fca"
        }
    ]
}
```


Un archivo JSON. Así de simple. No tiene más complicación. Es un diccionario típico con clave-valor. Ahora pasaremos a comentar que contiene.

10.4. Componiendo un paquete STIX básico

Antes de nada. Los STIX no se componen (escriben) a mano. Sería un trabajo extremadamente laborioso. STIX está pensado para operar a través de una librería que ofrece funciones para componer el STIX y finalizar entregando el paquete o bundle.

Es decir, STIX es un formato de intercambio de información sobre el estándar json para facilitar la serialización. Está pensado para que pueda ser leído por humanos por ser texto estructurado, pero no para que sea escrito por éstos.

Por ejemplo, para Python tenemos la librería STIX2.

Como hemos dicho, es a través de las librerías como nuestro sistema va incorporando objetos al STIX y tejiendo relaciones entre ellos. La idea es que con cada herramienta o análisis sobre la muestra, iremos encontrando artefactos (IoCs) y componiendo el "mapa" de esa muestra en particular que estamos estudiando. Al finalizar el proceso tenemos el STIX completo, lo cual se traduce en tener una fotografía panorámica de todo lo que abarca el malware.

Un ejemplo ilustrado en este esquema:

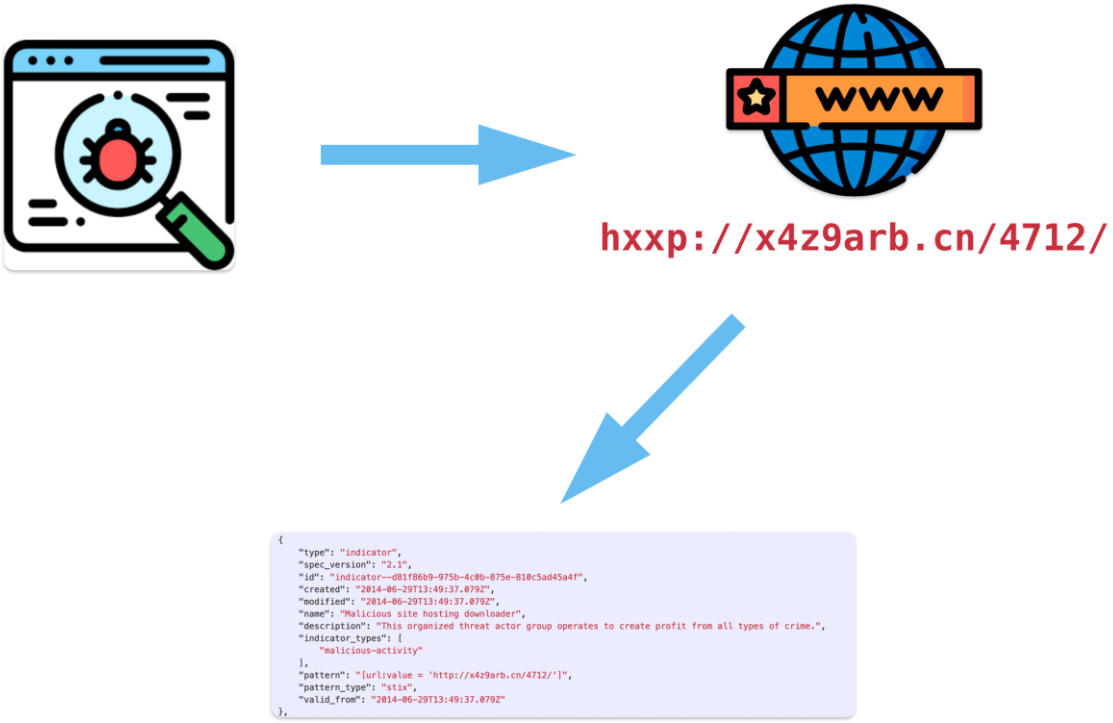


IMAGEN 20 FLUJO DE TRABAJO CON STIX

Vamos a explicar el flujo que vemos en la imagen:

Durante el análisis de una muestra, ésta intenta acceder al recurso sito en la URL hxxp://x4z9arb.cn/4712/. Imagina que hemos capturado esa URL porque tenemos un script que detecta URLs en archivos pcaps procedentes de la captura de red de la detonación.

Como consideramos esa URL parte de la infraestructura del malware, es un indicador de compromiso y por lo tanto formará parte del STIX que estamos componiendo.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Así, con el resto de IoC que vayamos encontrando. De hecho, si descomponemos la URL en dominio y, a su vez, dirección IP resuelta, tendremos más objetos que añadir al grafo. Incluso podemos adjuntar la geolocalización o los sistemas autónomos y redes empleadas. Tan profundo como queramos y podamos. A estas operaciones derivadas las llamamos **enriquecimiento**.

10.4.1. Añadiendo un objeto

Vamos a verlo de forma programática usando la librería comentada:

```python
from stix2 import Indicator

indicator = Indicator(
    description="Domain recorded during detonation of sample [captured by URLScrapper.py]",
    name="Malicious site hosting downloader",
    indicator_types=["malicious-activity"],
    pattern_type="stix",
    pattern="[url:value = 'http://x4z9arb.cn/4712/']",
)

print(indicator)
```

```
test1.py
❯ python test1.py | jq
{
  "type": "indicator",
  "spec_version": "2.1",
  "id": "indicator--9d3e7f61-2b18-4afc-b789-0db5553ffef0",
  "created": "2023-10-22T18:13:47.518089Z",
  "modified": "2023-10-22T18:13:47.518089Z",
  "name": "Malicious site hosting downloader",
  "description": "Domain recorded during detonation of sample [captured by URLScrapper.py]",
  "indicator_types": [
    "malicious-activity"
  ],
  "pattern": "[url:value = 'http://x4z9arb.cn/4712/']",
  "pattern_type": "stix",
  "pattern_version": "2.1",
  "valid_from": "2023-10-22T18:13:47.518089Z"
}
```

IMAGEN 21 USO EN PYTHON PARA CREAR UN INDICADOR STIX

Si dividimos la imagen en dos partes podemos ver en la de arriba el script Python y abajo la ejecución. Hay que tener en cuenta que lo que vemos no es un STIX completo, es solo un objeto y se trata de un paquete parcial, por terminar.

Lo importante aquí es ver como en un script ponemos los datos que disponemos y que describen el objeto (una URL) y la librería nos los transforma a un objeto STIX añadiendo campos que de otra forma serían muy engorroso añadirlos (fechas, UUIDs, etc.).

10.4.2. Estableciendo una relación

Además de añadir un segundo objeto, ahora modelaremos una relación entre ellos.

En primer lugar, añadiremos un malware, el que nuestro sistema estaría detonando. El orden en el que lo añadamos no es importante; recordemos que estamos tejiendo un grafo.

```
from stix2 import Indicator, Malware

indicator = Indicator(
    description="Domain recorded during detonation of sample [captured by URLScrapper.py]",
    name="Malicious site hosting downloader",
    indicator_types=["malicious-activity"],
    pattern_type="stix",
    pattern="[url:value = 'http://x4z9arb.cn/4712/']",
)

malware = Malware(
    name="x4z9arb backdoor",
    description="This malware attempts to download remote files after establishing a foothold as a backdoor.",
    malware_types=["backdoor", "remote-access-trojan"],
    is_family=False,
)
```

IMAGEN 22 AÑADIENDO UN MALWARE A STIX

Con ello, ya tenemos un objeto malware y un indicador. De momento están aislados, independientes. Lo único que poseen en común es estar en el mismo archivo fuente. Para ello, vamos a definir una relación, que será un nuevo objeto solo que su tipo y función es precisamente servir de nexo de conexión entre los dos anteriores. En lenguaje matemático sería la "arista" que une los dos nodos del grafo.

Veamos la imagen con el código y la ejecución:

```python
relationship = Relationship(
    relationship_type="indicates", source_ref=indicator.id, target_ref=malware.id
)

print(relationship.serialize(pretty=True))
```

```
test1.py
) python test1.py | jq
{
  "type": "relationship",
  "spec_version": "2.1",
  "id": "relationship--57ad1acf-1b7c-438f-88a9-49eadf011b78",
  "created": "2023-10-22T19:22:42.695882Z",
  "modified": "2023-10-22T19:22:42.695882Z",
  "relationship_type": "indicates",
  "source_ref": "indicator--563f108f-ed91-43b7-af3f-9b3642aee415",
  "target_ref": "malware--3876e97b-d2e5-4f1c-8458-53bd9438a572"
}
```

IMAGEN 23 DEFINICIÓN DE UNA RELACIÓN EN FORMATO STIX

Es bastante sencilla. Se invoca un objeto de la clase "Relationship" (previamente, importamos dicha clase), se indica el tipo de relación y, como hemos dicho: grafo dirigido, el origen y destino. En esta ocasión, la URL lleva al malware. Si fuese el malware el que usase la URL para descargar, por ejemplo, un archivo de configuración sería al revés.

10.4.3. Creación y uso del bundle STIX

El paso final. Ya solo tenemos que meterlo todo en un bundle o paquete STIX y estaría listo para ser compartido o almacenado.

Usando la librería STIX2 es una tarea bastante sencilla, como casi todo lo que hemos visto hasta ahora.

Importamos la clase "Bundle" y en el constructor pasamos todos los objetos que hemos ido creando. El resultado final, la instancia de la clase Bundle, contendrá todos los objetos y relaciones entre ellos.

```python
from stix2 import Bundle, Indicator, Malware, Relationship

indicator = Indicator(
    description="Domain recorded during detonation of sample [captured by URLScrapper.py]",
    name="Malicious site hosting downloader",
    indicator_types=["malicious-activity"],
    pattern_type="stix",
    pattern="[url:value = 'http://x4z9arb.cn/4712/']",
)

malware = Malware(
    name="x4z9arb backdoor",
    description="This malware attempts to download remote files after establishing a foothold as a backdoor.",
    malware_types=["backdoor", "remote-access-trojan"],
    is_family=False,
)

relationship = Relationship(
    relationship_type="indicates", source_ref=indicator.id, target_ref=malware.id
)

bundle = Bundle(indicator, malware, relationship)

print(bundle.serialize(pretty=True))
```

IMAGEN 24 CONSTRUCCIÓN FINAL DEL BUNDLE STIX

Ejecutando el script veremos la salida en formato JSON, el cual contendrá lo siguiente:


```json
{
  "type": "bundle",
  "id": "bundle--acf2c254-9b62-4656-9ca4-81864fbbf988",
  "objects": [
    {
      "type": "indicator",
      "spec_version": "2.1",
      "id": "indicator--b6ee2421-f812-43fc-a25b-040175b2e830",
      "created": "2023-10-22T19:45:51.781548Z",
      "modified": "2023-10-22T19:45:51.781548Z",
      "name": "Malicious site hosting downloader",
      "description": "Domain recorded during detonation of sample [captured by URLScrapper.py]",
      "indicator_types": [
        "malicious-activity"
      ],
      "pattern": "[url:value = 'http://x4z9arb.cn/4712/']",
      "pattern_type": "stix",
      "pattern_version": "2.1",
      "valid_from": "2023-10-22T19:45:51.781548Z"
    },
    {
      "type": "malware",
      "spec_version": "2.1",
      "id": "malware--03eedb54-7851-4109-814a-7f32dd3c42cd",
      "created": "2023-10-22T19:45:51.785922Z",
      "modified": "2023-10-22T19:45:51.785922Z",
      "name": "x4z9arb backdoor",
      "description": "This malware attempts to download remote files after establishing a foothold as a backdoor.",
      "malware_types": [
        "backdoor",
        "remote-access-trojan"
      ],
      "is_family": false
    },
    {
      "type": "relationship",
      "spec_version": "2.1",
      "id": "relationship--1619951c-c283-4e1b-95a7-69db8af341c4",
```


IMAGEN 25 PAQUETE STIX SERIALIZADO EN JSON

Ya está. Justo con eso ya podríamos compartirlo.

Respecto al almacenamiento, se usa la serialización. Al estar en formato JSON es posible tratarlo directamente como un objeto JSON genérico (en Python sería un diccionario) o deserializarlo y volver a usarlo en el contexto de la biblioteca STIX2. Veamos:

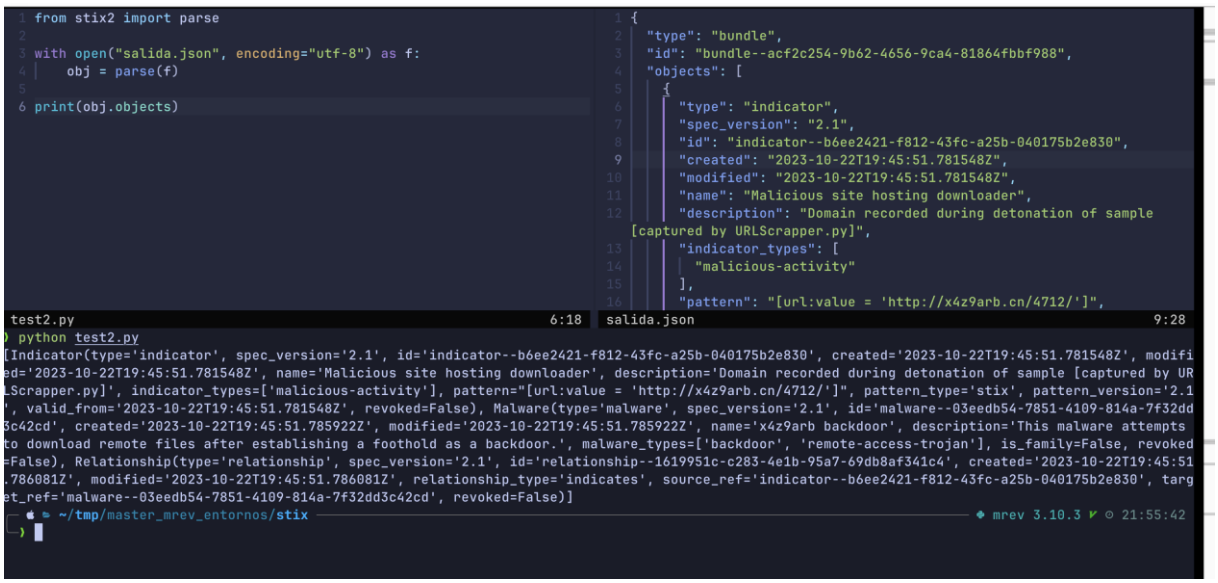


IMAGEN 26 DESERIALIZANDO UN PAQUETE STIX

Lo que estamos viendo es un pequeño script que carga un archivo JSON que es una serialización (justo la que hemos hecho más arriba) del paquete STIX creado. A la derecha, vemos dicho archivo de texto en formato JSON y abajo la ejecución del script, que muestra una cadena con los objetos de la librería cargados y listos para manipular de forma programática de nuevo.

10.4.4. Acerca de los UUID

El UUID, identificador único universal o "Universally Unique Identificator", es un identificador estandarizado que sirve para crear una referencia única (bueno, con el permiso de las colisiones).

Lo de universal es que al ser un estándar permite replicar el proceso de producción de estos UUID en diferentes plataformas o medios disponibles, además de poder detectarlos (con una simple expresión regular) y procesarlos sin sorpresas.

Único es, como hemos explicado y similar a los hashes, que tenga ciertas propiedades que permitan crear identificadores de forma continua y que no existan duplicados o si estos pueden darse, que posean una probabilidad reducida.

Hay varias versiones de UUID, dependiendo de cómo se crean, pero el formato es común a todos ellos y viene definido por la forma, en dígitos hexadecimales de:

8-4-4-12

Es mejor verlo con un ejemplo. Como en otras ocasiones, en Python, usando la librería estándar 'uuid':

```
>>> import uuid
>>> uuid.uuid1()
UUID('b165f156-711c-11ee-bf6a-5a93f4345d40')
>>> uuid.uuid4()
UUID('cb5c113c-7b41-4794-866e-e5d9bc56c04e')
>>>
```

IMAGEN 27 GENERANDO UUID EN PYTHON

Hemos generado dos UUID de versiones diferentes, pero el formato es idéntico.

Sus aplicaciones son inmensas y muy extendidas. Al ser un estándar nos permite identificarlos al vuelo y ya de por sí, su presencia indica para que sirven: dotar a un objeto con un identificador.

Huelga decir que no solo se usan, evidentemente, en ciberseguridad, sus usos están extendidos a todos los campos de la informática.

Los UUID en STIX poseen una cadena prefijo que acompaña al identificador y que permite reseñar que tipo de objeto es.

10.5. Uso avanzado de STIX

Hasta ahora hemos presentado STIX y un uso básico para entender cómo funciona. Para ello, hemos hecho uso de la librería Python STIX2 para no perder de vista el aspecto práctico. A partir de la introducción, ahora estamos en disposición de internarnos en los aspectos más avanzados. En realidad, avanzados no sería la palabra exacta, sino más bien profundos por la diversidad que incluye.

Vamos a situarnos a un nivel de abstracción apropiado para presentar los tipos y objetos que posee STIX y que son los que nos permiten modelar con detalle una amenaza. Hasta ahora, hemos visto como modelar una URL, el propio malware o un dominio asociados entre sí y producir un paquete STIX. Pero eso solo son objetos básicos.

En primer lugar, hablemos de como organiza STIX los tipos y objetos.

10.5.1. SDO (STIX Domain Objects)

STIX, en su versión 2.1, define 18 SDO's (STIX Domain Objects). Con estos objetos nos servimos, como hemos hecho antes, para representar todo lo relacionado con una amenaza. Son estos:

TABLA 1 TABLA DE LOS SDO DEFINIDOS POR STIX 2.1

Object	Name	Description
	Attack Pattern	A type of TTP that describe ways that adversaries attempt to compromise targets.
	Campaign	A grouping of adversarial behaviors that describes a set of malicious activities or attacks (sometimes called waves) that occur over a period of time against a specific set of targets.
	Course of Action	A recommendation from a producer of intelligence to a consumer on the actions that they might take in response to that intelligence.
	Grouping	Explicitly asserts that the referenced STIX Objects have a shared context, unlike a STIX Bundle (which explicitly conveys no context).
	Identity	Actual individuals, organizations, or groups (e.g., ACME, Inc.) as well as classes of individuals, organizations, systems or groups (e.g., the finance sector).
	Indicator	Contains a pattern that can be used to detect suspicious or malicious cyber activity.
	Infrastructure	Represents a type of TTP and describes any systems, software services and any associated physical or virtual resources intended to support some purpose (e.g., C2 servers used as part of an attack, device or server that are part of defence, database servers targeted by an attack, etc.).
	Intrusion Set	A grouped set of adversarial behaviors and resources with common properties that is believed to be orchestrated by a single organization.
	Location	Represents a geographic location.
	Malware	A type of TTP that represents malicious code.
	Malware Analysis	The metadata and results of a particular static or dynamic analysis performed on a malware instance or family.
	Note	Conveys informative text to provide further context and/or to provide additional analysis not contained in the STIX Objects, Marking Definition objects, or Language Content objects which the Note relates to.
	Observed Data	Conveys information about cyber security related entities such as files, systems, and networks using the STIX Cyber-observable Objects (SCOs).
	Opinion	An assessment of the correctness of the information in a STIX Object produced by a different entity.
	Report	Collections of threat intelligence focused on one or more topics, such as a description of a threat actor, malware, or attack technique, including context and related details.
	Threat Actor	Actual individuals, groups, or organizations believed to be operating with malicious intent.
	Tool	Legitimate software that can be used by threat actors to perform attacks.
	Vulnerability	A mistake in software that can be directly used by a hacker to gain access to a system or network.

(fuente: https://oasis-open.github.io/cti-documentation/stix/intro)

No es descartable que aparezcan más en un futuro, por supuesto. En un campo tan cambiante como es el desarrollo de malware y nuevas campañas, se produce una constante innovación de técnicas que sugieren, a su vez, nuevos planteamientos y definiciones.

No tendría sentido alguno ir desgranando mecánicamente cada una de las entradas de esa tabla. Es mejor explicar cómo usarla y dar indicaciones generales de como agregar un objeto y utilizar el vocabulario provisto por STIX.

Como venimos diciendo, STIX no solo modela aquello que observamos y analizamos durante una detonación sino que va más allá. Podemos expresar toda una campaña APT y relacionarla incluso con artículos escritos sobre cierto grupo, etc.

10.5.2. SRO (STIX Relationship Object)

Los SRO son las aristas entre el resto de componentes. Existen dos tipos de relaciones.

10.5.2.1. RELATIONSHIP

Modelan las relaciones entre los SDO o SCO, pero lo hacen a un nivel muy genérico. STIX deja claro aquí que son las aristas del grafo dirigido. Y es cierto, cuando veamos una representación gráfica solo este tipo de relación se verá representada gráficamente.

10.5.2.2. LAS RELACIONES DE ESTE TIPO VIENEN INDICADAS EN EL APARTADO CORRESPONDIENTE A CADA SDO O SCO EN LA DOCUMENTACIÓN.



Relationship	Used to link together two SDOs or SCOs in order to describe how they are related to each other.

En líneas generales, **pero siempre de forma orientativa**, se usa cuando en la documentación encontramos un caso de uso especificado. Un ejemplo, respecto del SDO "Attack pattern":

```
[
 {
  "type": "attack-pattern",
  "spec_version": "2.1",
  "id": "attack-pattern--7e33a43e-e34b-40ec-89da-36c9bb2cacd5",
  "created": "2016-05-12T08:17:27.000Z",
  "modified": "2016-05-12T08:17:27.000Z",
  "name": "Spear Phishing as Practiced by Adversary X",
  "description": "A particular form of spear phishing where the attacker claims that the target had won a contest, including personal details, to get them to click on a link.",
  "external_references": [
   {
    "source_name": "capec",
    "external_id": "CAPEC-163"
   }
  ]
 },
 {
  "type": "relationship",
  "spec_version": "2.1",
  "id": "relationship--57b56a43-b8b0-4cba-9deb-34e3e1faed9e",
  "created": "2016-05-12T08:17:27.000Z",
  "modified": "2016-05-12T08:17:27.000Z",
  "relationship_type": "uses",
  "source_ref": "intrusion-set--0c7e22ad-b099-4dc3-b0df-2ea3f49ae2e6",
  "target_ref": "attack-pattern--7e33a43e-e34b-40ec-89da-36c9bb2cacd5"
 },
 {
  "type": "intrusion-set",
  "spec_version": "2.1",
  "id": "intrusion-set--0c7e22ad-b099-4dc3-b0df-2ea3f49ae2e6",
  "created": "2016-05-12T08:17:27.000Z",
  "modified": "2016-05-12T08:17:27.000Z",
  "name": "Adversary X"
 }
]
```


Como vemos, existe un "Intrusion-set" (herramientas usadas en una intrusión) que son empleadas de forma ordenada y muy característica. Es decir, están **relacionadas con un patrón de ataque**. Fijaos que la misma frase lo dice y la documentación lo apoya, por lo tanto es una relación establecida por el objeto **Relationship**.

10.5.2.3. SIGHTING

El segundo tipo de relación hace honor a la frase: " el objeto X ha sido visto con el objeto Y". No posee arista gráfica en un grafo representado.



Sighting Denotes the belief that something in CTI (e.g., an indicator, malware, tool, threat actor, etc.) was seen.

Este tipo de relación nos permite aportar más datos y además, tan solo necesita el origen de la referencia "sighting_of_ref" y si acaso, los observables referenciados en "observed_data_ref".

Puede llegar a ser complicado discernir en un caso cuál de las dos relaciones es la adecuada. Básicamente, si no está documentado en Relationship entonces es Sighting. También, tener en cuenta que la clave es "observación" en el sentido de "avistamiento".

10.5.3. SCO (STIX Cyber-observable Object)

Afortunadamente, es fácil saber cuáles son los cyber-observables: Son objetos concretos que encontramos en un análisis: ficheros, hashes, dominios, direcciones IP. Objetos prácticos y que no poseen la genericidad y abstracción que sí poseen los SDO.

Una característica de los SCO es que casi siempre complementan a un SDO. Enriquecen su conjunto de datos y propiedades extendiendo de forma externa su significado.

10.6. Caso práctico con STIX

Supongamos que durante la detonación de un nuevo malware, éste descarga un archivo "malicious_payload.exe" que resulta tener el mismo hash que un archivo relacionado con un grupo APT en anteriores operaciones.

También vamos a suponer que ese hash y ese grupo están sacados de un informe de un laboratorio y fue publicado hace unos meses. Es decir, estamos complementando dicha investigación añadiendo datos de nuestra cosecha.

Existen varias formas de modelar estas relaciones.

En primer lugar vamos a definir los objetos involucrados:

- Malware en estudio
- Hash del archivo descargado
- Artículo publicado
- Grupo APT

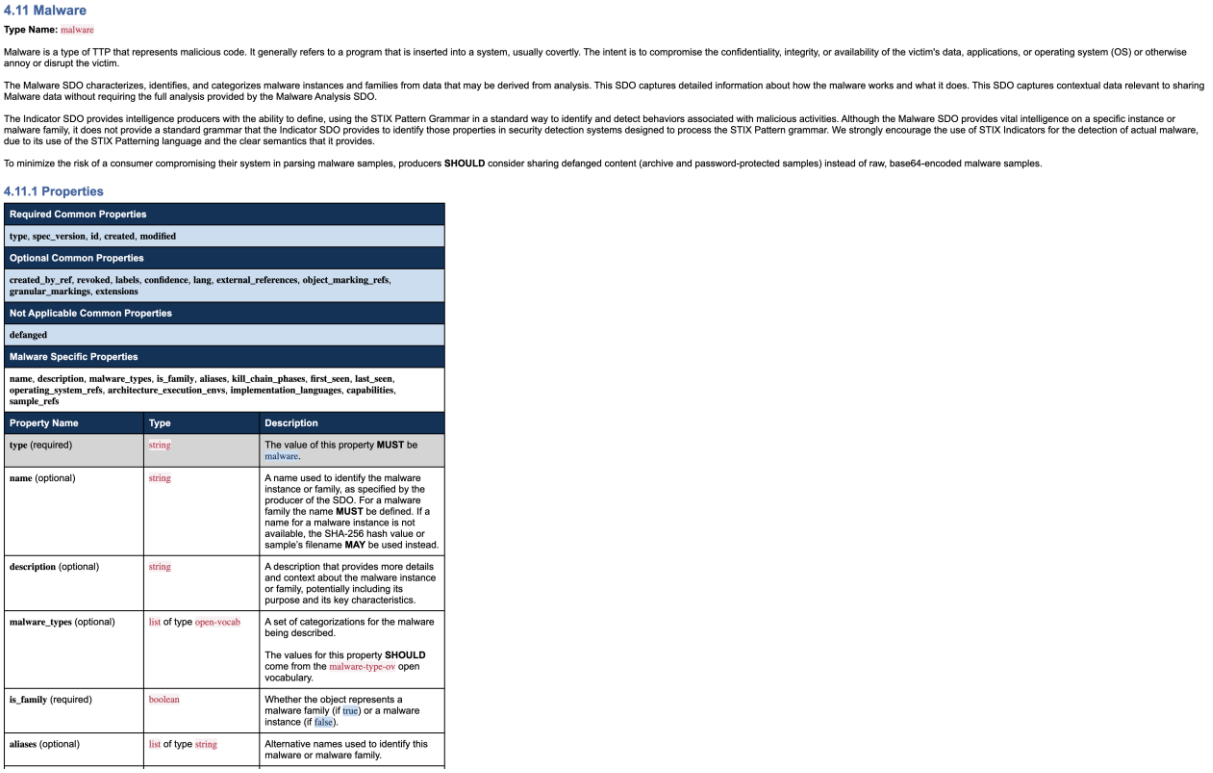
Podríamos profundizar mucho más. Por ejemplo, añadiendo los indicadores del nombre de dominio y el propio servidor de descarga. No vamos a complicar el ejemplo.

Para el malware, hacemos uso del SDO apropiado:



Malware A type of TTP that represents malicious code.

En la documentación de la especificación del formato STIX 2.1 nos encontramos con esto:



4.11 Malware

Type Name: malware

Malware is a type of TTP that represents malicious code. It generally refers to a program that is inserted into a system, usually covertly. The intent is to compromise the confidentiality, integrity, or availability of the victim's data, applications, or operating system (OS) or otherwise annoy or disrupt the victim.

The Malware SDO characterizes, identifies, and categorizes malware instances and families from data that may be derived from analysis. This SDO captures detailed information about how the malware works and what it does. This SDO captures contextual data relevant to sharing Malware data without requiring the full analysis provided by the Malware Analysis SDO.

The Indicator SDO provides intelligence producers with the ability to define, using the STIX Pattern Grammar in a standard way to identify and detect behaviors associated with malicious activities. Although the Malware SDO provides vital intelligence on a specific instance or malware family, it does not provide a standard grammar that the Indicator SDO provides to identify those properties in security detection systems designed to process the STIX Pattern grammar. We strongly encourage the use of STIX Indicators for the detection of actual malware, due to its use of the STIX Patterning language and the clear semantics that it provides.

To minimize the risk of a consumer compromising their system in parsing malware samples, producers **SHOULD** consider sharing defanged content (archive and password-protected samples) instead of raw, base64-encoded malware samples.

4.11.1 Properties

Required Common Properties		
type, **spec_version**, **id**, **created**, **modified**		
Optional Common Properties		
created_by_ref, **revoked**, **labels**, **confidence**, **lang**, **external_references**, **object_marking_refs**, **granular_markings**, **extensions**		
Not Applicable Common Properties		
defanged		
Malware Specific Properties		
name, **description**, **malware_types**, **is_family**, **aliases**, **kill_chain_phases**, **first_seen**, **last_seen**, **operating_system_refs**, **architecture_execution_envs**, **implementation_languages**, **capabilities**, **sample_refs**		
Property Name	**Type**	**Description**
type (required)	string	The value of this property **MUST** be malware.
name (optional)	string	A name used to identify the malware instance or family, as specified by the producer of the SDO. For a malware family the name **MUST** be defined. If a name for a malware instance is not available, the SHA-256 hash value or sample's filename **MAY** be used instead.
description (optional)	string	A description that provides more details and context about the malware instance or family, potentially including its purpose and its key characteristics.
malware_types (optional)	list of type open-vocab	A set of categorizations for the malware being described. The values for this property **SHOULD** come from the malware-type-ov open vocabulary.
is_family (required)	boolean	Whether the object represents a malware family (if true) or a malware instance (if false).
aliases (optional)	list of type string	Alternative names used to identify this malware or malware family.

Es una definición del SDO en sí y sus propiedades. Las propiedades son los datos que aportamos a ese SDO para darle cuerpo. Un objeto vacío no nos sirve de mucho, pero curiosamente sí que podemos emplear SDO's con un mínimo de propiedades.

De hecho, en el caso de malware, tan solo es requerida la presencia obligatoria de dos propiedades: "type" y "is_family". La primera de ellas es simplemente el nombre del SDO, por lo que no nos dará mucho trabajo. La segunda propiedad, "is_family", si está a "true" indica que estamos hablando de una familia de malware en el plano abstracto, mientras que si lo ponemos a "false" es que estamos refiriéndonos a una muestra concreta. Es como si con el primer hablamos del ransomware Tesla en general y con "is_family" a "false" estaríamos hablando de un malware detonado que resulta ser una instancia de dicha familia.

¿Se entiende? ¿No? Venga, un ejemplo más terrenal. Imagina que hablamos de coches. En concreto vamos a referirnos a los SEAT 124, ninguno en particular, sino en general. Eso haría que pusiéramos "is_family" a "true". Pero si hablo del SEAT 124 que tiene Paco, natural de Calzadilla de los Barros y que tiene matrícula BA-12345 (todo datos inventados), estoy hablando de un coche en concreto...aunque es familia de los míticos 124 de SEAT. En tal caso, "is_family" a "false".

Continuando con el uso de la librería STIX2 en Python:

```python
from stix2 import Malware

malware = Malware(is_family=False)
```

Con eso ya tendríamos nuestro malware. Pero claro, no posee mucha información, solo la obligatoria (por cierto: el "type" ya nos lo pone automáticamente la librería)

¿Por qué podemos añadir SDO's sin mucha información? Es lógico que pueda hacerse. Imaginad que en ese momento no tenemos esa información, nos faltan datos, etc. El SDO formará parte del STIX y completaremos la información a medida que llegue. Pero, por la falta de información, no deberíamos tener impedimentos para añadir un nuevo nodo. Simplemente, se queda pendiente.

Es habitual, en el caso del malware, que todavía no tengamos un nombre (propiedad "name") y se le ponga de forma provisional el hash SHA1 del binario. Posteriormente, si se le da nombre, se le cambiaría.

De hecho, a posteriori, añadamos el nombre del malware.

```
from stix2 import Malware

malware = Malware(is_family=False)
malware = malware.new_version(name="LOWBALL")

print(malware.serialize(pretty=True))
```

```
test3.py                                                    6:35
❯ python test3.py
{
    "type": "malware",
    "spec_version": "2.1",
    "id": "malware--6635c2a6-bd6c-4944-bd0f-8a216aee1af0",
    "created": "2023-10-24T12:59:18.482585Z",
    "modified": "2023-10-24T12:59:18.482807Z",
    "name": "LOWBALL",
    "is_family": false
}
 ~/tmp/master_mrev_entornos/stix                    mrev 3.10.3  14:59:18
❯
```

Sí, lo que ves. Cada vez que modifiquemos un SDO en la librería debemos crear un nuevo objeto con las nuevas propiedades añadidas. No obstante, si lo tuviéramos serializado y lo recuperamos deserializándolo, una vez modificado los cambios se plasmarían, obviamente, en la nueva serialización.

10.6.1. Descubrimos que el malware es del tipo ransomware y emplea técnicas rookit

Vamos a ponernos en el caso. Estamos detonando y tenemos una baliza (beacon) que vigila si se ha cifrado un grupo de ciertos archivos puestos ahí a propósito. Si la baliza da positivo entonces es ransomware. Además, vemos por un detector heurístico que contiene rutinas para instalar un rootkit en el sistema. Esto es automático. Así que procedemos a modificar nuestro malware y añadir un par de datos bastante informativos.

Estamos seguros de que debe haber un campo para ello ¿verdad? Buscamos entre las propiedades (en la documentación STIX) y efectivamente lo encontramos:

malware_types (optional)	list of type open-vocab	A set of categorizations for the malware being described. The values for this property **SHOULD** come from the malware-type-ov open vocabulary.

¿Se hace directamente? ¿Qué tipo ponemos? No podemos poner un valor cualquiera en ese campo. Nos lo está diciendo la documentación. Debe ser una lista de valores de "open-vocab".

Ya dijimos anteriormente que STIX detalla muchísimo el nivel de granularidad con el que podemos detallar o modelar una operación APT. Así que para ciertos valores debemos atenernos a los valores posibles de su vocabulario (que se denomina "open-vocab").

En concreto, para el caso de "tipo de malware" debemos buscar en "malware-type-ov". Viene en la especificación de STIX, así que no tenemos que viajar mucho para encontrarlo:

10.15 Malware Type Vocabulary

Vocabulary Name: malware-type-ov

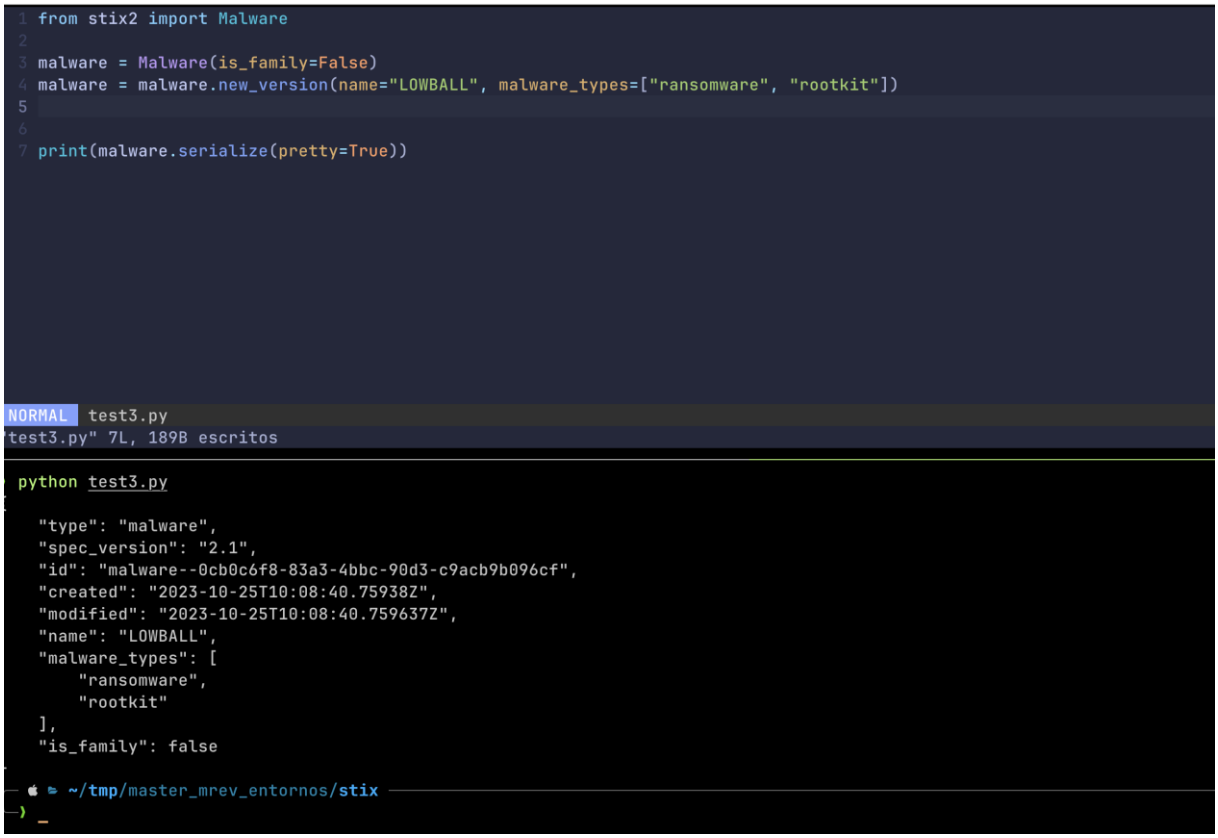
The malware type vocabulary is currently used in the following SDO(s):

- Malware

Malware type is an open vocabulary that represents different types and functions of malware. Malware types are not mutually exclusive; for example, a malware instance can be both spyware and a screen capture tool.

Vocabulary Summary	
adware, backdoor, bot, bootkit, ddos, downloader, dropper, exploit-kit, keylogger, ransomware, remote-access-trojan, resource-exploitation, rogue-security-software, rootkit, screen-capture, spyware, trojan, unknown, virus, webshell, wiper, worm	
Vocabulary Value	**Description**
adware	Any software that is funded by advertising. Adware may also gather sensitive user information from a system.
backdoor	A malicious program that allows an attacker to perform actions on a remote system, such as transferring files, acquiring passwords, or executing arbitrary commands [NIST800-83].
bot	A program that resides on an infected system, communicating with

Así que esos son los valores apropiados (la captura de pantalla es parcial) que podemos añadir en el malware objeto de análisis. Vámonos al editor de código y lo vemos de forma práctica:



Como vemos, el campo correspondiente "malware_types" y su valor sería una lista con los valores apropiados y ya definidos en "malware_types_ov". Aunque podemos añadir cualquier cadena, no es conveniente salirnos de las definiciones de STIX salvo cuando se indique.

10.6.2. Modelando el archivo descargado

Supongamos ahora que el malware, durante su ejecución ha descargado un archivo y tenemos su hash. Siguiendo el hilo de la historia, ese hash está referenciado a un artículo publicado por un laboratorio en el que atribuyen la autoría a un grupo APT. Vamos a integrar el hash del archivo en nuestro modelo.

Según nuestra herramienta (en nuestro sistema, vale con las utilidades de línea de comando "md5sum" y "sha1sum"), los hashes correspondientes del archivo descargado son (añadimos md5 por si nos resulta útil):

sha1: 3b3d6a02d0963b6adb60522ad38f6169562f2e38
md5: 2c4b6bfc7912b5fd26ff61ed4ff0a6a7

Ahora bien, si buscamos una categoría adecuada a "archivo" dentro de los SDO no vamos a encontrarla. Es decir, en los SDO no hay un "File".

El archivo, al igual que otros muchos recursos, es un **objeto observado durante la ejecución**. Algo así como si tuviésemos un bloc de notas y un cronómetro durante la detonación y dijésemos: "Vale, 23 segundos después de su ejecución, se **observa** la descarga un archivo. Anoto hora y hash del archivo descargado".

Así, pues, procedemos a agregar un objeto del tipo "ObservedData", que requiere varias propiedades para su creación, como los campos "first_observed", "last_observed" y "number_observed":

```python
from datetime import datetime

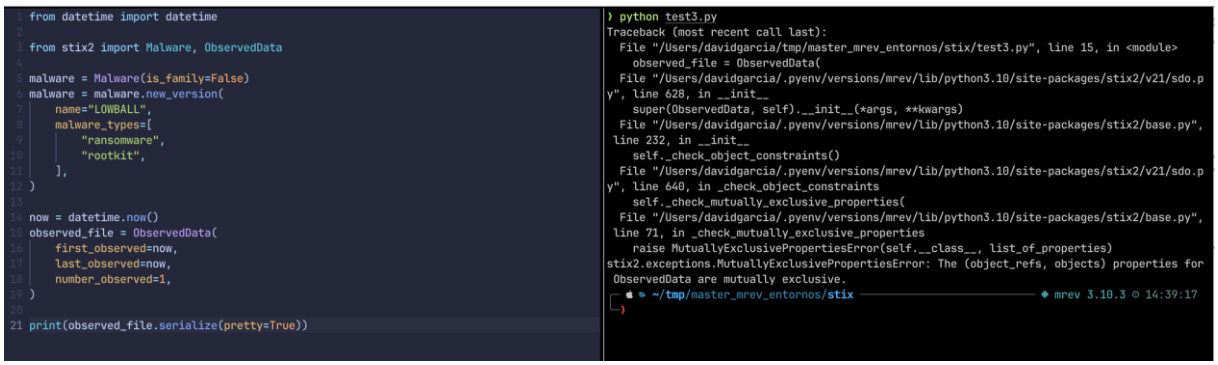
from stix2 import Malware, ObservedData

malware = Malware(is_family=False)
malware = malware.new_version(
    name="LOWBALL",
    malware_types=[
        "ransomware",
        "rootkit",
    ],
)

now = datetime.now()
observed_file = ObservedData(
    first_observed=now,
    last_observed=now,
    number_observed=1,
)

print(observed_file.serialize(pretty=True))
```

Nos falta uno, ahora sabremos por qué:



Un SDO "Observed Data" no puede subsistir sin referencias. Es como si dijéramos que se ha observado algo en algún momento **¿Pero qué exactamente?**

Lo veremos justo ahora. Pero antes, debemos comprender que si un SDO es demasiado genérico, entonces es que debemos enriquecerlo con referencias externas que otorguen

más información. En este caso, los "datos observados" nos dicen que "algo" ha sido observado y "cuándo".

Ahora, vamos a hacer uso de los SCO para ser más concretos.

Especificando más datos sobre el archivo observado (usando SCO)

Los SCO, como ya hemos comentado en su apartado correspondiente, nos permiten ser más concretos con los indicadores y observaciones que se hacen en los SDO. Si hemos observado un archivo descargado, entonces hay un objeto SCO que nos permite modelar eso. Fijémonos:

6.7 File Object

Type Name: file

The File object represents the properties of a file. A File object **MUST** contain at least one of **hashes** or **name**.

Ahora sí que podemos modelar con todos los datos técnicos el archivo descargado:



El objeto "File" de la librería STIX2 es el SCO apropiado y en el constructor podemos poner todas las propiedades que queramos y que tengamos. Cuantas más, obviamente, mejor; más definido estará el objeto. En este caso hemos añadido los hashes porque los tenemos y es fácil calcularlos, además del tamaño del archivo.

10.6.3. Añadiendo referencias

Es evidente que el objeto "ObservedData" SDO y el objeto "file" SCO no poseen ninguna relación. Pero además, no la vamos a definir con los "relationship" del tipo SRO.

Con ellos, al referirse al mismo archivo físico, tanto el SDO como el SCO deben poseer lo que STIX denomina **referencias**, que es de lo que se quejaba cuando hemos intentado instanciar un objeto "ObservedData". Recordemos:

```
❯ python test3.py
Esquema
ack (most recent call last):
  File "/Users/davidgarcia/tmp/master_mrev_entornos/stix/test3.py", line 15, in <module>
    observed_file = ObservedData(
  File "/Users/davidgarcia/.pyenv/versions/mrev/lib/python3.10/site-packages/stix2/v21/sdo.py", line 628, in __init__
    super(ObservedData, self).__init__(*args, **kwargs)
  File "/Users/davidgarcia/.pyenv/versions/mrev/lib/python3.10/site-packages/stix2/base.py", line 232, in __init__
    self._check_object_constraints()
  File "/Users/davidgarcia/.pyenv/versions/mrev/lib/python3.10/site-packages/stix2/v21/sdo.py", line 640, in _check_object_constraints
    self._check_mutually_exclusive_properties(
  File "/Users/davidgarcia/.pyenv/versions/mrev/lib/python3.10/site-packages/stix2/base.py", line 71, in _check_mutually_exclusive_properties
    raise MutuallyExclusivePropertiesError(self.__class__, list_of_properties)
stix2.exceptions.MutuallyExclusivePropertiesError: The (object_refs, objects) properties for ObservedData are mutually exclusive.
```

Aunque es un galimatías, se queja de que el objeto "ObservedData" no posee referencias que lo sustenten.

Vamos a añadir al objeto del tipo "ObservedData" esa referencia en sus propiedades y ejecutar:

```python
from datetime import datetime

from stix2 import File, Malware, ObservedData

malware = Malware(is_family=False)
malware = malware.new_version(
    name="LOWBALL",
    malware_types=[
        "ransomware",
        "rootkit",
    ],
)

file_observed = File(
    name="malicious_payload.exe",
    size=23456,
    mime_type="application/octet-stream",
    hashes={
        "SHA1": "3b3d6a02d0963b6adb60522ad38f6169562f2e38",
        "MD5": "2c4b6bfc7912b5fd26ff61ed4ff0a6a7",
    },
)

now = datetime.now()
file_event = ObservedData(
    object_refs=[file_observed],
    first_observed=now,
    last_observed=now,
    number_observed=1,
)


print(file_observed.serialize(pretty=True))
print(file_event.serialize(pretty=True))
```

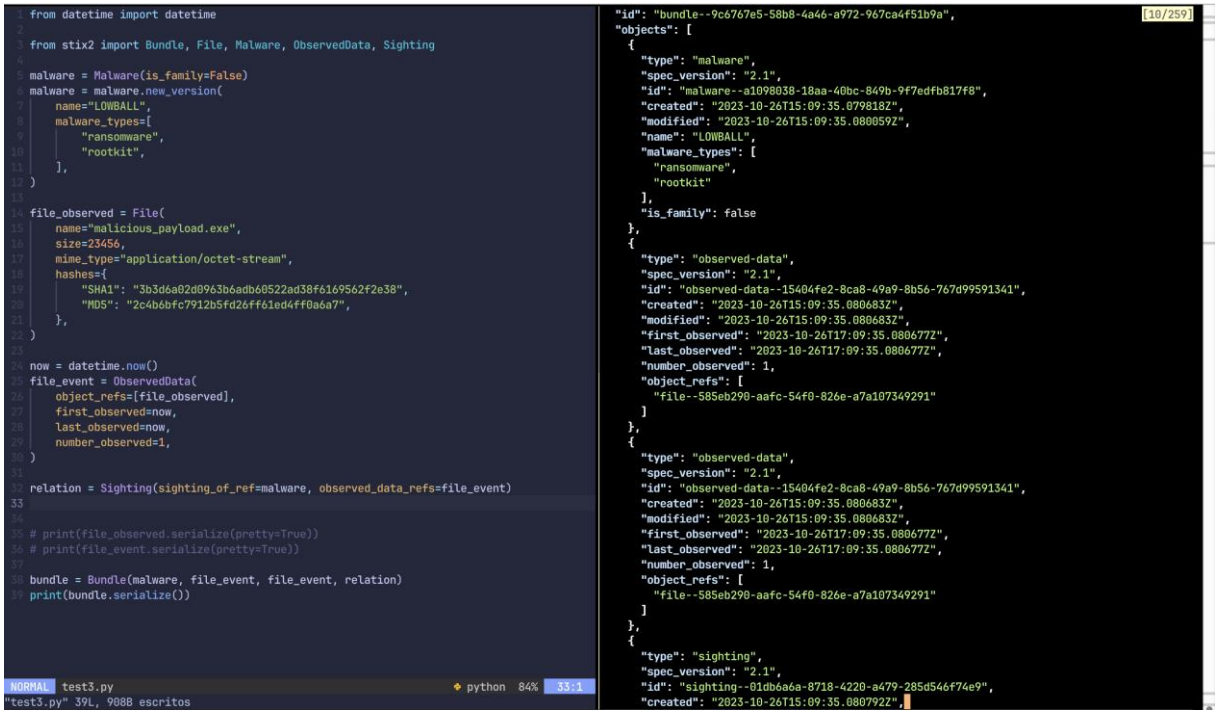
```
❯ python test3.py
{
    "type": "file",
    "spec_version": "2.1",
    "id": "file--585eb290-aafc-54f0-826e-a7a107349291",
    "hashes": {
        "MD5": "2c4b6bfc7912b5fd26ff61ed4ff0a6a7",
        "SHA-1": "3b3d6a02d0963b6adb60522ad38f6169562f2e38"
    },
    "size": 23456,
    "name": "malicious_payload.exe",
    "mime_type": "application/octet-stream"
}
{
    "type": "observed-data",
    "spec_version": "2.1",
    "id": "observed-data--517bc521-abec-44dd-8d42-8e3cd55c4877",
    "created": "2023-10-26T12:48:09.822538Z",
    "modified": "2023-10-26T12:48:09.822538Z",
    "first_observed": "2023-10-26T14:48:09.822532Z",
    "last_observed": "2023-10-26T14:48:09.822532Z",
    "number_observed": 1,
    "object_refs": [
        "file--585eb290-aafc-54f0-826e-a7a107349291"
    ]
}
 ~/tmp/master_mrev_entornos/stix ──────────── mrev 3.10.3 14:48:09
❯
```

Ya tenemos los dos SDO (Malware y ObservedData) y un SCO (File). Vamos a establecer la relación que une el malware al observable creado. Esto, evidentemente, lo hacemos con un SRO.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

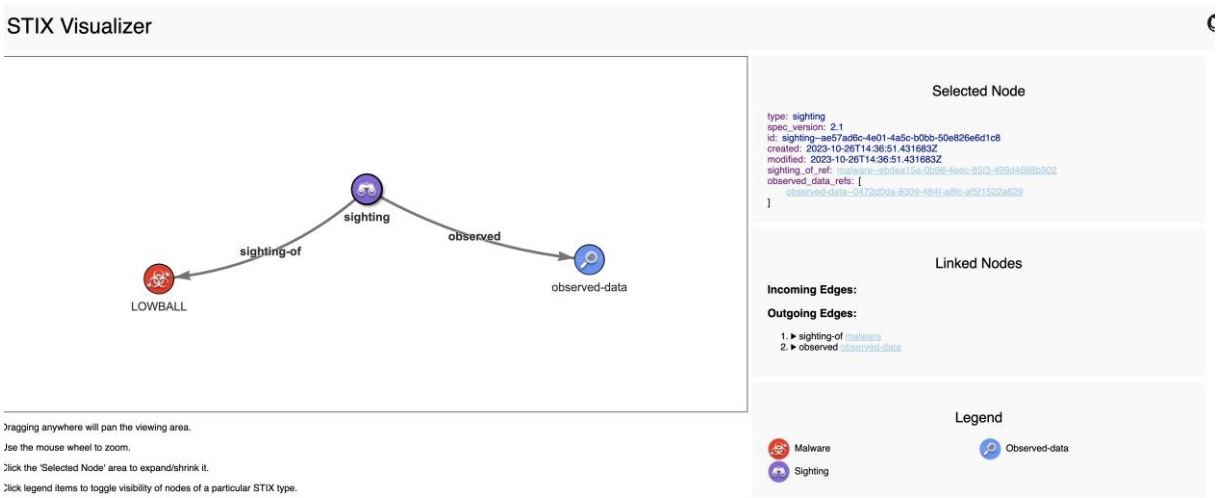
Tenemos dos tipos de SRO, como ya vimos: las **relaciones** (Relationships) y las observaciones (**Sightings**). En nuestro caso, es una observación. Es como decir: "He visto a aquel troyano descargar este archivo".

Empleamos el objeto "Sighting". A nivel elemental, solo tenemos que cumplimentar las propiedades "sighting_of_ref" y observed_data_refs" con los objetos SDO ya creados:



Podemos añadir todas las referencias de observaciones en el campo "observed_data_refs", pero solo puede haber un SDO en la propiedad "sighting_of_ref".

El grafo correspondiente al bundle que estamos creando pintaría así:



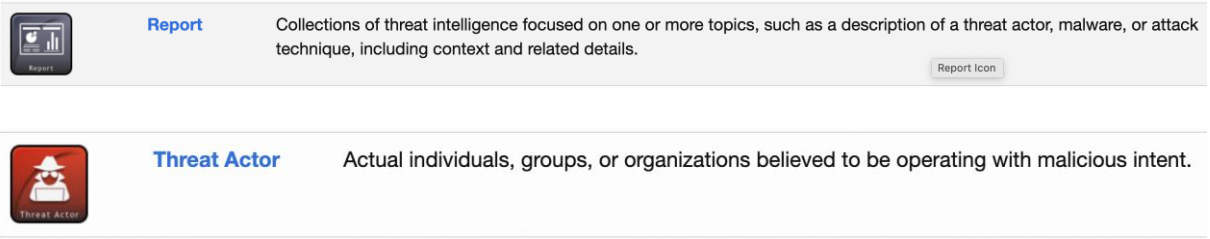
Como vemos, ya tenemos modelado el hecho de que cierto malware descarga cierto archivo.

10.6.4. Añadiendo el grupo APT

Como dijimos en el caso práctico, el hash del archivo es lo que hizo saltar las alarmas al estar relacionado con un hash visto durante el análisis.

Ahora tenemos tres objetos que podríamos relacionar y añadir: el hash, el informe y el grupo APT si queremos.

Para el informe y el grupo APT tenemos dos SDO muy claros:



El informe es este: https://unit42.paloaltonetworks.com/gamaredon-primitive-bear-ukraine-update-2021/ (es un ejemplo, no tengáis en cuenta si aparece o no el hash, archivo, etc.). En él, aparece como atribución de las operaciones al grupo Gamaredon.

Veamos en código como introducimos el Threat Actor y lo relacionamos con el malware:

```
group = ThreatActor(name="gamaredon")

rel_grp_mal = Relationship(
    relationship_type="uses", source_ref=group, target_ref=malware
)
```

Muy simple. De hecho no sería tan simple en un caso real, sino que estaría acompañado de muchas más propiedades e incluso el objeto Threat Actor, si estaba definido ya en nuestra base de datos sería una deserialización.

La relación, en este caso, es un SRO "Relationship" de SDO a SDO. Si nos fijamos en el tipo de relación es "uses" dado que es uno de los tipos de relaciones "de-hacia" admitidos por el SDO Threat Actor:

threat-actor	uses	attack-pattern, infrastructure, malware, tool	This Relationship describes that attacks carried out as part of the Threat Actor typically use the related Attack Pattern, Infrastructure, Malware, or Tool. For example, a uses Relationship from the jay-sm17h Threat Actor to the xInject Malware indicates that xInject is often used by John Smith. A campaign, threat actor, intrusion set, malware, or tool takes infrastructure and compromises and/or uses it for their own.

Si nos fijamos bien, también vemos como Threat Actor posee una "referencia inversa":

Reverse Relationships			
campaign, intrusion-set	attributed-to	threat-actor	See forward relationship for definition.
malware	authored-by	threat-actor	See forward relationship for definition.
indicator	indicates	threat-actor	See forward relationship for definition.

Es decir, el grupo APT usa un malware ¿Pero y si ese malware ha sido creado por ese grupo? Entonces podemos modelar esta autoría estableciendo esa relación:

```
rel_mal_grp = Relationship(
    relationship_type="authored-by", source_ref=malware, target_ref=group
)
```

Y así pinta nuestro grafo:



10.6.5. Añadiendo el informe y su URL de consulta

Como dijimos, con el informe lo tenemos claro. Se trata del SDO "Report" y no contiene muchos campos requeridos, aunque siempre es conveniente añadir lo que tengamos a mano para enriquecer nuestro paquete STIX.

El objeto Report en código queda así:

```
published_time = datetime(year=2021, month=2, day=3)
report = Report(
    name="gamaredon-primitive-bear-ukraine-update-2021",
    published=published_time,
    object_refs=[malware, group, file_event],
    report_types=["campaign"],
    external_references=[url_report],
)
```

Es decir, el nombre, fecha de publicación, tipo de informe (definido por el vocabulario STIX correspondiente) y ahora veremos que es el resto.

Aquí tenemos un buen ejemplo para el uso tanto de relaciones como de referencias externas. Un informe puede ser consultado por URL, es lo habitual. ¿Pero para que añadir una URL como SDO o SCO sino interviene en la infección?

Para ello tenemos las referencias externas. Y en el caso del informe, vemos que disponemos del campo "external_references" para enlazarla. Luego, creamos esa referencia externa con un objeto de la clase "ExternalReference" y estaríamos listos.

```
url_report = ExternalReference(
    source_name="unit42",
    url="https://unit42.paloaltonetworks.com/gamaredon-primitive-bear-ukraine-update-
2021/",
)
```

Una vez creada, como hemos visto en las propiedades del informe, se añade:

```
external_references=[url_report],
```

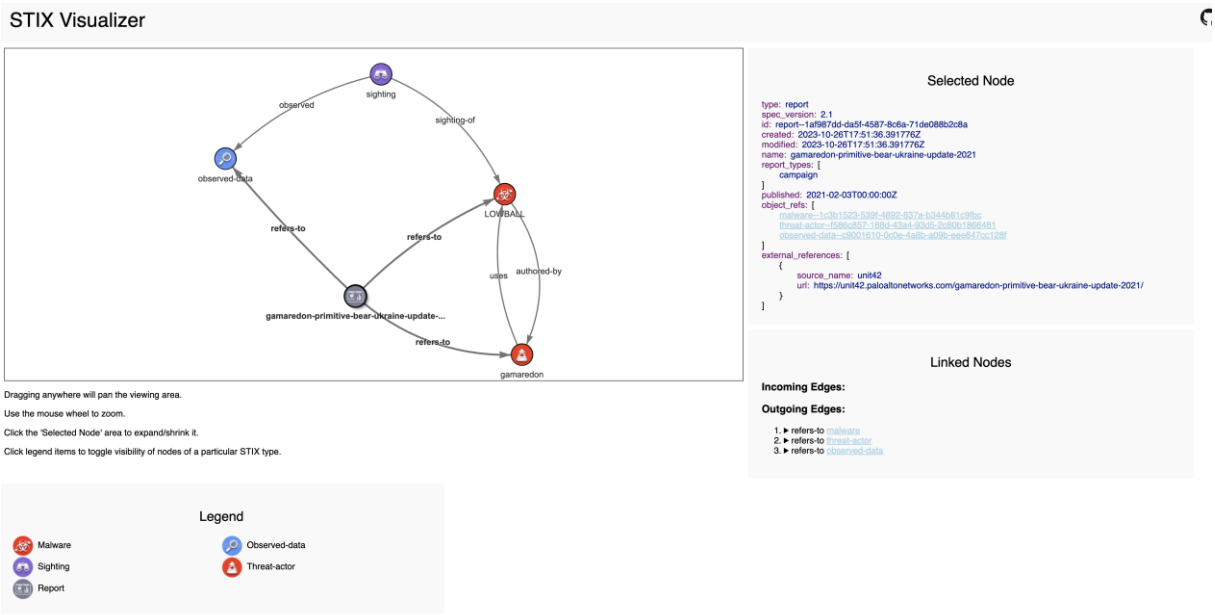
Ya tenemos ese enlace hecho. Ahora el informe tiene su URL pero no se representa como un nodo en el grafo.

Deteniéndonos en las propiedades del informe vemos también otro campo de referencias:

```
object_refs=[malware, group, file_event],
```

Esto se traduce en relaciones incrustadas o embebidas. Es decir, el informe hace "referencia" al malware, al grupo APT y al archivo (su hash). Pero no se modelan como Relationship ni como Sighting porque da igual como lo intentemos no le veríamos la lógica. Por eso existen las llamadas referencias y que hemos estado viendo.

Volvemos a visualizar el grafo:

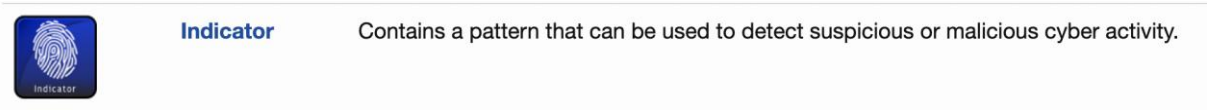


Ya está el nodo del informe con sus referencias anotadas cuando seleccionamos el nodo.

10.6.6. Añadiendo un indicador, el hash visto

Ya tendríamos un paquete STIX bastante completo (aunque básico), pero vamos a añadir un nuevo componente. Como el hash tiene bastante peso y es un indicador de la infección por el malware, vamos a darle protagonismo.

Podemos hacerlo a través de un SDO que nos permite modelar IoCs. "Indicator":



Haciendo uso de nuestra librería STIX2 como siempre, instanciamos un objeto de la clase Indicator.

```
mal_hash = Indicator(
    name="hash",
    description="hashes of downloaded file by malware",
    indicator_types=["malicious-activity"],
    pattern_type="stix",
    pattern="[file:hash.SHA1=='3b3d6a02d0963b6adb60522ad38f6169562f2e38' OR file:hash.MD5=='2c4b6bfc7912b5fd26ff61ed4ff0a6a7']",
)
```

El primer campo, "indicator_types" está bastante claro. Es una actividad maliciosa puesto que tenemos certeza, por el hash, que el objeto descargado es malicioso. Los tipos del indicador vienen definidos por su vocabulario; en la documentación STIX bajo "indicator-type-ov".

Los patrones y lenguaje de patrones

Los campos "pattern" y "pattern_type" son algo más complejos de explicar, pero por su significado se intuye su función.

Vamos a empezar por el "pattern_type" porque nos va a despejar dudas antes de meternos con el patrón en sí.

El "pattern_type" indica el lenguaje de patrones usado. Es simple, STIX soporta: stix, pcre, sigma, snort, suricata o yara. Muchos de ellos ya los hemos introducido aquí, por lo que no deben representar ningún problema. Es habitual, no obstante, usar 'stix', pero por usar puede ser cualquiera. A conveniencia. De hecho, si ya tienes creada de antemano una regla en algún otro lenguaje, lo lógico es emplearla.

El patrón usado debe describir unívocamente el objeto modelado. Así, puedo poner el nombre del archivo y añadir el hash o cualquier otra propiedad que creamos conveniente. **La regla de oro es: que el patrón exprese la detección sin ambigüedades del objeto indicador**.

Bien, pues ese indicador ahora debemos relacionarlo con el malware, es simple, porque ya lo hemos estado viendo. Usaremos una Relationship:

```
hash_to_malware = Relationship(
    relationship_type="indicates", source_ref=mal_hash, target_ref=malware
)
```

Esa arista entra de forma perfecta en el cuadro porque ya tenemos bastantes elementos sobre el tapete.

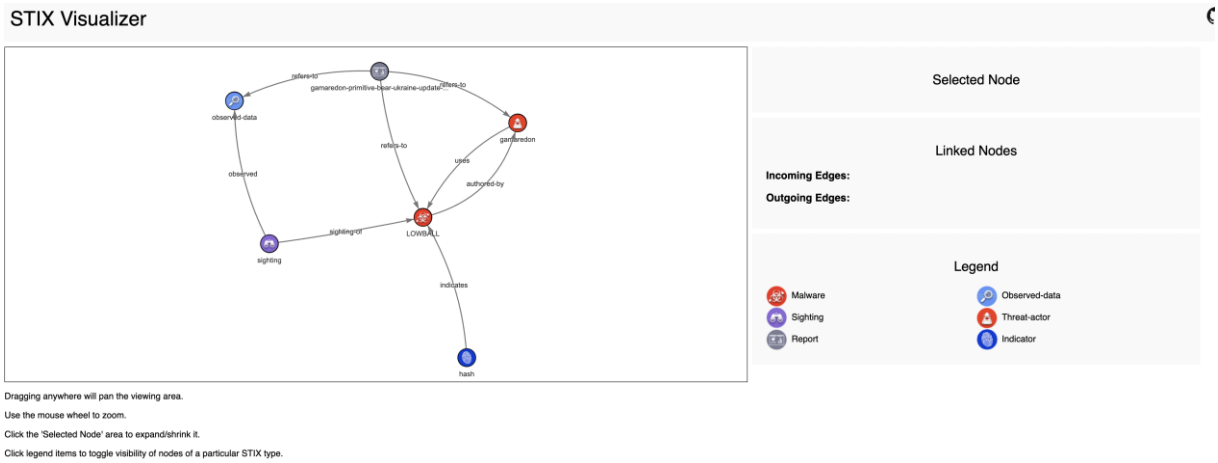
El tipo de relación viene dado por los que tenemos en el objeto SDO Indicator de la docuentación:

indicator	indicates	attack-pattern, campaign, infrastructure, intrusion-set, malware, threat-actor, tool	This Relationship describes that the Indicator can detect evidence of the related Attack Pattern, Campaign, Infrastructure, Intrusion Set, Malware, Threat Actor, or Tool. This evidence may not be direct: for example, the Indicator may detect secondary evidence of the Campaign, such as malware or behavior commonly used by that Campaign. For example, an indicates Relationship from an Indicator to a Campaign object representing Glass Gazelle means that the Indicator is capable of detecting evidence of Glass Gazelle, such as command and control IPs commonly used by that Campaign.

Es básico explicarlo: "Si ves este hash es que indica que te has infectado con el malware".

10.6.7. El cuadro final

Ya tendríamos todas las relaciones y objetos pertinentes. Nuestro grafo final queda así:



¿Podríamos haber añadido algo más? Por supuesto, podríamos profundizar tanto como quisiéramos y pudiésemos, siempre que existan datos e información disponibles.

Podríamos haber enlazado el hash, añadir a los creadores del informe e incluso crear un SDO campaña ya que es probable que la infección pertenezca a una orquestación de dicho grupo.

Aportamos el código final del bundle y su JSON:

```
from datetime import datetime

from stix2 import (Bundle, ExternalReference, File, Indicator, Malware,
                   ObservedData, Relationship, Report, Sighting, ThreatActor)

malware = Malware(is_family=False)
malware = malware.new_version(
    name="LOWBALL",
    malware_types=[
        "ransomware",
        "rootkit",
    ],
)

file_observed = File(
    name="malicious_payload.exe",
    size=23456,
    mime_type="application/octet-stream",
    hashes={
        "SHA1": "3b3d6a02d0963b6adb60522ad38f6169562f2e38",
        "MD5": "2c4b6bfc7912b5fd26ff61ed4ff0a6a7",
    },
)

now = datetime.now()
file_event = ObservedData(
    object_refs=[file_observed],
    first_observed=now,
    last_observed=now,
    number_observed=1,
)

sight_mal_file = Sighting(sighting_of_ref=malware, observed_data_refs=file_event)

group = ThreatActor(name="gamaredon")

mal_hash = Indicator(
    name="hash",
    description="hashes of downloaded file by malware",
    indicator_types=["malicious-activity"],
    pattern_type="stix",
    pattern="[file:hash.SHA1=='3b3d6a02d0963b6adb60522ad38f6169562f2e38' OR file:hash.MD5=='2c4b6bfc7912b5fd26ff61ed4ff0a6a7']",
)

hash_to_malware = Relationship(
    relationship_type="indicates", source_ref=mal_hash, target_ref=malware
)

rel_grp_mal = Relationship(
    relationship_type="uses", source_ref=group, target_ref=malware
```

```
)

rel_mal_grp = Relationship(
    relationship_type="authored-by", source_ref=malware, target_ref=group
)

url_report = ExternalReference(
    source_name="unit42",
    url="https://unit42.paloaltonetworks.com/gamaredon-primitive-bear-ukraine-update-2021/",
)

published_time = datetime(year=2021, month=2, day=3)
report = Report(
    name="gamaredon-primitive-bear-ukraine-update-2021",
    published=published_time,
    object_refs=[malware, group, file_event],
    report_types=["campaign"],
    external_references=[url_report],
)


bundle = Bundle(
    malware,
    file_event,
    file_event,
    sight_mal_file,
    group,
    rel_grp_mal,
    rel_mal_grp,
    report,
    mal_hash,
    hash_to_malware,
)
print(bundle.serialize())
```

Bundle en formato JSON:


```
{
  "type": "bundle",
  "id": "bundle--b2bcf88d-ad98-4a95-8582-6997790ab75c",
  "objects": [
    {
      "type": "malware",
      "spec_version": "2.1",
      "id": "malware--a9924e72-ed86-4387-9404-e6c6ee40ce81",
      "created": "2023-10-26T18:30:13.568793Z",
      "modified": "2023-10-26T18:30:13.569062Z",
      "name": "LOWBALL",
      "malware_types": [
        "ransomware",
        "rootkit"
      ],
      "is_family": false
    },
    {
      "type": "observed-data",
      "spec_version": "2.1",
      "id": "observed-data--292890c8-a543-40c7-a738-37647b42ba01",
      "created": "2023-10-26T18:30:13.569913Z",
      "modified": "2023-10-26T18:30:13.569913Z",
      "first_observed": "2023-10-26T20:30:13.569907Z",
      "last_observed": "2023-10-26T20:30:13.569907Z",
      "number_observed": 1,
      "object_refs": [
        "file--585eb290-aafc-54f0-826e-a7a107349291"
      ]
    },
    {
      "type": "observed-data",
      "spec_version": "2.1",
      "id": "observed-data--292890c8-a543-40c7-a738-37647b42ba01",
      "created": "2023-10-26T18:30:13.569913Z",
      "modified": "2023-10-26T18:30:13.569913Z",
      "first_observed": "2023-10-26T20:30:13.569907Z",
      "last_observed": "2023-10-26T20:30:13.569907Z",
      "number_observed": 1,
      "object_refs": [
        "file--585eb290-aafc-54f0-826e-a7a107349291"
      ]
    },
    {
      "type": "sighting",
      "spec_version": "2.1",
      "id": "sighting--a0ddd3e1-e984-4f5a-9e5d-28de8780c35b",
      "created": "2023-10-26T18:30:13.570034Z",
      "modified": "2023-10-26T18:30:13.570034Z",
      "sighting_of_ref": "malware--a9924e72-ed86-4387-9404-e6c6ee40ce81",
      "observed_data_refs": [
        "observed-data--292890c8-a543-40c7-a738-37647b42ba01"
      ]
```

```
  },
  {
    "type": "threat-actor",
    "spec_version": "2.1",
    "id": "threat-actor--0f2cc28c-106a-4bba-b951-e7ca753e90b4",
    "created": "2023-10-26T18:30:13.570235Z",
    "modified": "2023-10-26T18:30:13.570235Z",
    "name": "gamaredon"
  },
  {
    "type": "relationship",
    "spec_version": "2.1",
    "id": "relationship--fbf3ae92-5c25-4616-ada0-9ed9bd9f4ac4",
    "created": "2023-10-26T18:30:13.576029Z",
    "modified": "2023-10-26T18:30:13.576029Z",
    "relationship_type": "uses",
    "source_ref": "threat-actor--0f2cc28c-106a-4bba-b951-e7ca753e90b4",
    "target_ref": "malware--a9924e72-ed86-4387-9404-e6c6ee40ce81"
  },
  {
    "type": "relationship",
    "spec_version": "2.1",
    "id": "relationship--b279207f-beae-4346-aba0-a371154f7bec",
    "created": "2023-10-26T18:30:13.576123Z",
    "modified": "2023-10-26T18:30:13.576123Z",
    "relationship_type": "authored-by",
    "source_ref": "malware--a9924e72-ed86-4387-9404-e6c6ee40ce81",
    "target_ref": "threat-actor--0f2cc28c-106a-4bba-b951-e7ca753e90b4"
  },
  {
    "type": "report",
    "spec_version": "2.1",
    "id": "report--b96e1335-a2a9-4467-a1a4-cdf9258931dd",
    "created": "2023-10-26T18:30:13.576242Z",
    "modified": "2023-10-26T18:30:13.576242Z",
    "name": "gamaredon-primitive-bear-ukraine-update-2021",
    "report_types": [
      "campaign"
    ],
    "published": "2021-02-03T00:00:00Z",
    "object_refs": [
      "malware--a9924e72-ed86-4387-9404-e6c6ee40ce81",
      "threat-actor--0f2cc28c-106a-4bba-b951-e7ca753e90b4",
      "observed-data--292890c8-a543-40c7-a738-37647b42ba01"
    ],
    "external_references": [
      {
        "source_name": "unit42",
        "url": "https://unit42.paloaltonetworks.com/gamaredon-primitive-bear-ukraine-update-2021/"
      }
    ]
  },
  {
    "type": "indicator",
    "spec_version": "2.1",
    "id": "indicator--a43074dd-adf0-4c68-a500-01288cc00521",
    "created": "2023-10-26T18:30:13.570352Z",
    "modified": "2023-10-26T18:30:13.570352Z",
    "name": "hash",
    "description": "hashes of downloaded file by malware",
    "indicator_types": [
      "malicious-activity"
    ],
    "pattern": "[file:hash.SHA1=='3b3d6a02d0963b6adb60522ad38f6169562f2e38' OR file:hash.MD5=='2c4b6bfc7912b5fd26ff61ed4ff0a6a7']",
    "pattern_type": "stix",
    "pattern_version": "2.1",
    "valid_from": "2023-10-26T18:30:13.570352Z"
  },
  {
    "type": "relationship",
    "spec_version": "2.1",
    "id": "relationship--19d88382-a595-4ba4-96d1-5019549d1c78",
    "created": "2023-10-26T18:30:13.575907Z",
    "modified": "2023-10-26T18:30:13.575907Z",
    "relationship_type": "indicates",
    "source_ref": "indicator--a43074dd-adf0-4c68-a500-01288cc00521",
    "target_ref": "malware--a9924e72-ed86-4387-9404-e6c6ee40ce81"
  }
 ]
}
```

10.7. TAXII

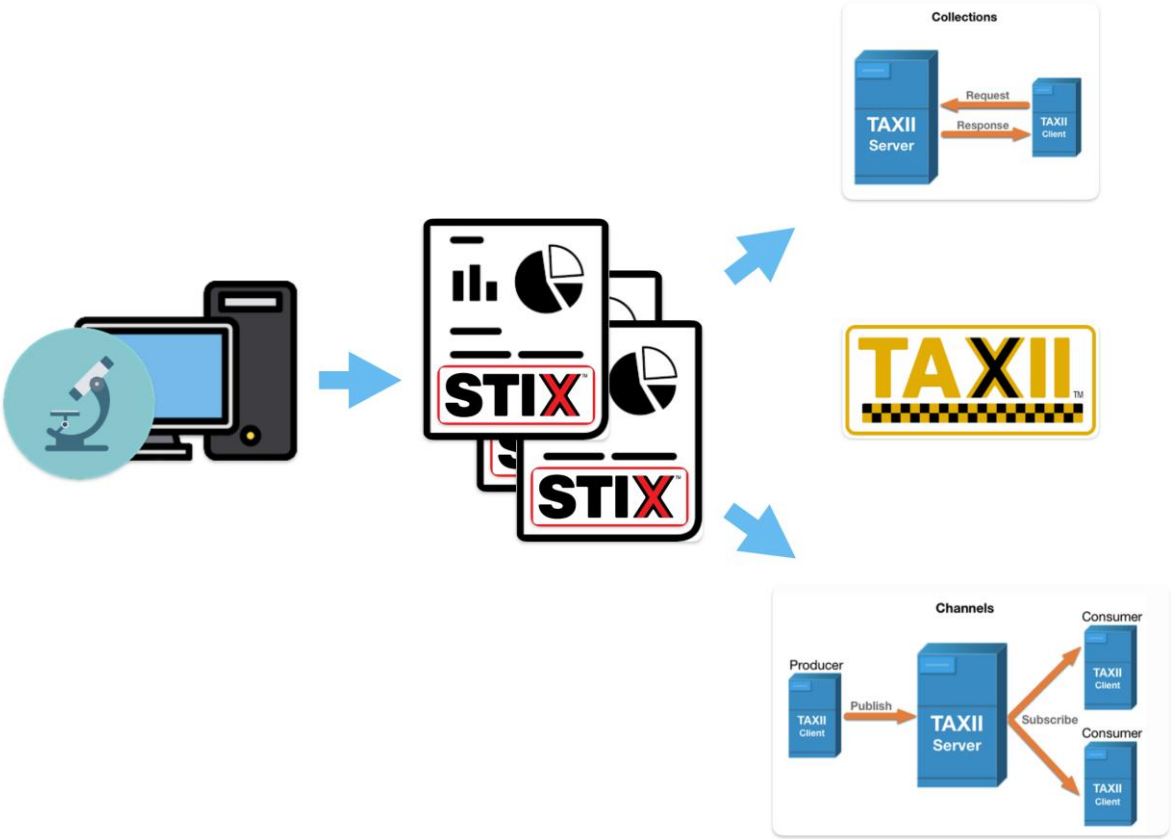
Sus siglas significan "Trusted Automated Exchange of Intelligence Information".

Definición rápida: TAXII es el protocolo de transporte diseñado expresamente para transmitir STIX.

TAXII está especificado aquí.

TAXII sirve para que dos o más socios intercambien paquetes STIX de forma automatizada.

Pero, como casi todo, mejor un gráfico:



A la izquierda nuestra sandbox, analizando y produciendo informes como paquetes (bundles) STIX. Como vemos sigue siendo un proceso mecanizado o industrializado.

Esos paquetes STIX serán transmitidos a nuestros socios, clientes u otros equipos. Para ello, el transporte, surgió TAXII.

Las dos formas que vemos se corresponden con las modalidades que hay para transmitir: a través de canales o colecciones.

No tiene mucho secreto. El esquema de colecciones es simple, almacenamos los STIX en un servidor TAXII y ya irán los clientes TAXII a recuperarlos a través de un índice. Por contra, el esquema de canales va por subscripción. Muchos clientes pueden subscribirse a nuestros canales y en cuanto publiquemos (nuestro cliente TAXII publicando STIX en dicho servidor) se creará un evento push hacia los clientes subscritos y estos recuperarán el objeto u objetos (paquetes STIX) disponibles.

TAXII no solo define los esquemas de comunicación sino que nos dice como debemos implementar una API específica para el intercambio de STIX, los tipos de datos, el formato (JSON) o el protocolo (HTTPS), además de que emplear para la autenticación, etc.

No vamos a detenernos mucho porque TAXII es un ámbito en el que deben de profundizar más los equipos de Devops y desarrollo, que serán los que pongan a punto el sistema.

Como analistas nos sirve conocer su existencia, para qué sirve y poco más.

11. CYBER KILL CHAIN

Si has leído con atención el apartado de STIX verás que ha aparecido el concepto de Cyber-Kill-Chain. Esta denominación viene del ámbito militar y se trasladó (como infinidad de términos) al ámbito de la ciberseguridad por parte de la compañía Lockheed Martin.

La kill-chain es un conjunto de fases o etapas que modelan un ataque. Desde el comienzo, cuando el grupo APT está recolectando información, hasta el ataque inicial, explotación, infiltración, etc. Es decir, toda la **cadena** que recorre un grupo hasta la consecución de sus objetivos.

Otra kill-chain conocida también ha sido introducida, la de ATT&CK que posee más fases y estrictamente no indica que se deban dar todas las fases que define en un ataque en particular.

11.1. ¿Para qué sirve en ciberseguridad?

Nos permite definir en qué fase se encuentra un ataque. O, dado un artefacto que encontremos (troyano, exploit, comunicación, etc.) durante un análisis, podemos darle cabida en alguna de las fases de la cadena dependiendo de las acciones observadas.

De hecho, STIX utiliza kill-chains en algunas propiedades, lo que permite que definamos cuando ocurre un evento. Si estamos analizando un archivo de red y observamos un exploit, está claro que podría tener cabida en la etapa 4 de "Explotación"; siempre atendiendo al contexto y objetivos con los que, el grupo cibercriminal, lo utilice.

Por ejemplo, el SDO de STIX "Attack-Pattern" posee una propiedad específica:

kill_chain_phases (optional)	list of type kill-chain-phase	The list of Kill Chain Phases for which this Attack Pattern is used.

También la poseen otros SDO como "Indicator", "Infraestructure", "Malware", etc. Obra decir, que se puede utilizar cualquier otra clasificación, pero como canon está muy reconocida la de Lockheed Martin.

Al final, lo que tratamos de transmitir es "dónde" se encuentra dentro de la cadena el objeto descrito, en el caso de STIX.

11.2. Fases de las Cyber-Kill-Chain

En el modelo de Lockheed Martin son 7 etapas.

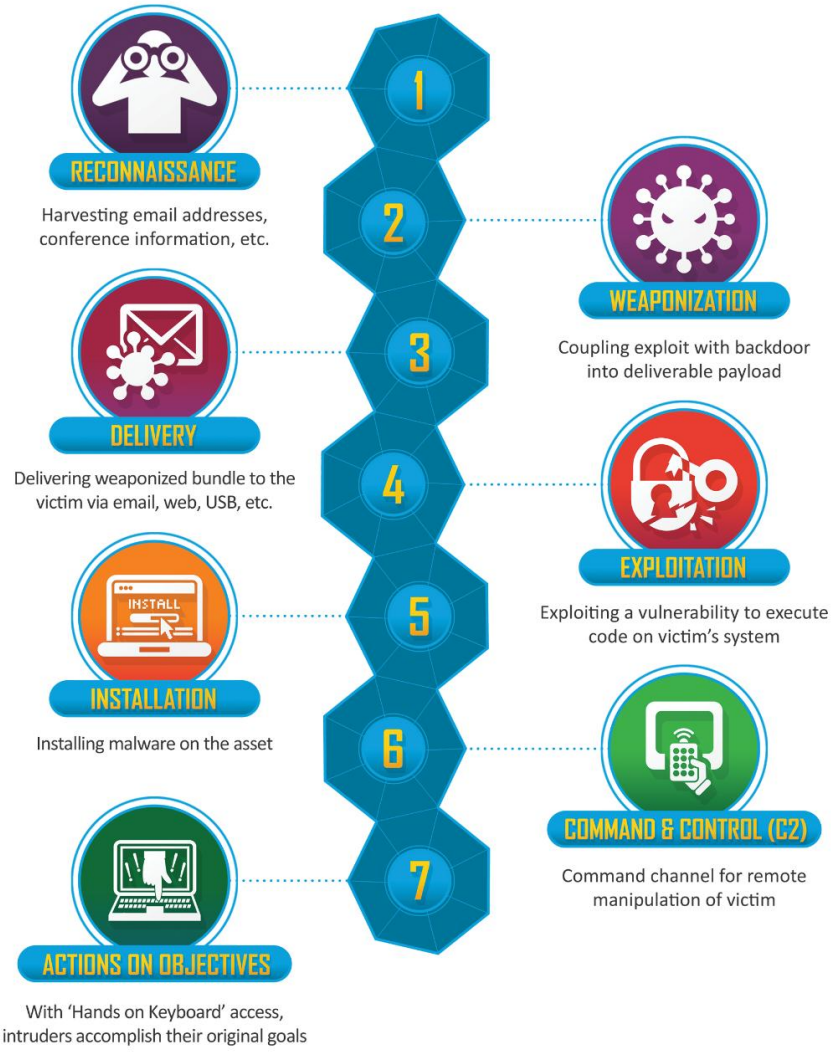


ILUSTRACIÓN 9 FASES EN LA CYBERKILLCHAIN (FUENTE: HTTPS://WWW.LOCKHEEDMARTIN.COM/EN-US/CAPABILITIES/CYBER/CYBER-KILL-CHAIN.HTML)

Coinciden muy de cerca con cualquier metodología Red Team o de Pentesting. Sobre todo en áreas de reconocimiento y explotación.

La idea es que un ataque se divide en estas fases (se deben cumplir todas en el modelo Lockheed Martin). Es decir, una APT comenzaría en la fase temprana de obtención de información, pasaría a la elección del arsenal a emplear (malware, exploits, phishings...), el envío de malware o explotación de vulnerabilidades, la instalación de puertas traseras y demás artefactos operativos, el control de los nodos infectados y el despliegue de acciones dentro del perímetro afectado: exploración de la red interna, persistencia, robo de información, etc.

El modelo de MITRE, visto en ATT&CK, posee muchas más fases, pero como se dijo, no todas van a encontrarse en la misma operación. La idea de ATT&CK es más bien identificar patrones que aglutinen bajo un mismo conjunto ciertas acciones. Ya es la propia idiosincrasia del grupo si las emplea o no.

De un vistazo, podemos ver:



Luego, a su vez, se despliegan en las técnicas que ya vimos en el capítulo dedicado a ATT&CK. No vamos a añadir nada más.

11.3. ¿Cómo usamos una Kill-Chain?

Es sencillo. Optar por un modelo u otro va a depender del contexto. STIX permite el uso de las dos, mientras que algunas sandbox comerciales o Virustotal suelen presentar resultados relacionados con ATT&CK. La ventaja de este último es que no solo agrupa un conjunto de técnicas bajo una misma denominación, sino que se para a detallarlas, identificarlas e incluso proponer mitigaciones y ejemplos. Digamos que ATT&CK posee mayor nivel de detalle (granularidad).

Usar una kill-chain es básicamente atribuir una fase o etapa a una acción. Esto se verá reflejado en nuestro informe, paquete STIX o detección de habilidades en una sandbox (algo que hemos visto usando la herramienta "capa")

El uso se encuentra en nuestra capacidad para discernir en que categoría o etapa entra un componente que queramos identificar. Por ejemplo, si analizamos el malware Mirai, veremos que en un momento dado se activa una rutina de escaneo de la red en busca de nuevos nodos para infectar.

Ese nuevo ataque, en ATT&CK, tendría la codificación T1595.001 "Scanning IP Blocks" dentro de la fase de "Reconnaissance" (reconocimiento) y coincide con la fase 1 de Lockheed Martin del mismo nombre.

Sin embargo, cuando dicho componente encuentra una máquina prometedora y comienza a lanzar ataques de diccionario y fuerza bruta. ¿Dónde ponemos dicha técnica? En ATT&CK, de nuevo, está perfectamente identificada: T1110.001 (Password guessing) mientras que en Lockheed Martin pasaríamos a "Exploitation" dado que se trata de "explotar" la "vulnerabilidad" (ambos términos discutibles) de dejar una cuenta con las credenciales por defecto o débiles.

Quedémonos con:

- Usaremos el framework adecuado basado en el contexto. Si podemos o nos dejan elegir: ATT&CK.
- Piensa que quiere conseguir el atacante y qué está empleando. En Lockheed es más filosófico, en ATT&CK es más práctico, como buscar en unas páginas amarillas

(o el índice de un libro si no sabes que son las páginas amarillas).

- El kill-chain proporciona muy buena información acerca de la fase en la que se encuentra un ataque cuando se observa la técnica descrita o empleada en ese momento.